Module-III

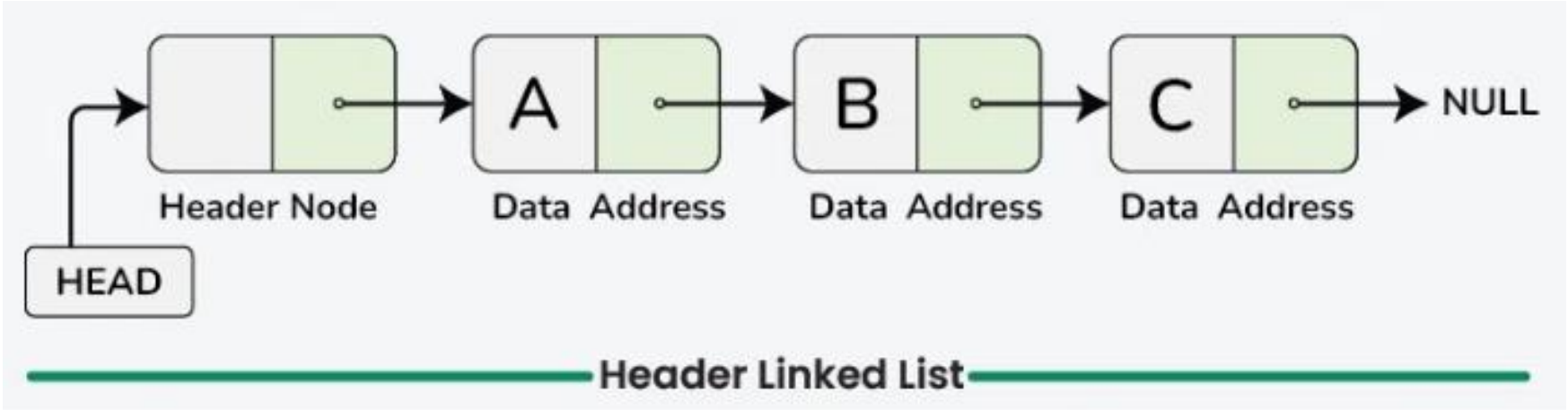
Advanced Linked Lists: Header linked lists, Doubly Linked lists Operations, Circular linked lists, Linked Stacks and Queues.

Applications of Linked Lists: Polynomials, Sparse matrix representation.

Trees: Terminologies, Binary Trees, Properties of Binary Trees, Array and linked representation of Binary Trees, Binary Tree Traversals, Threaded binary trees.

Header Linked List

- A Header Linked List is a special type of linked list where an extra node is placed at the beginning of the list.
- This extra node is called the header node or dummy node.



- Header node does NOT store actual data.
- It stores metadata such as:
  - count of nodes
  - pointers for circular links
  - temporary info
- Real data starts after the header node.

Why Use a Header Linked List?

- Simplifies insertion & deletion (no special case for head changes)
- Eliminates need for repeated if (head == NULL) checks
- Makes algorithms uniform and clean
- Good for lists where operations at the beginning happen often
- Header can store:
  - number of nodes
  - pointer to last node
  - type of list (sorted/unsorted)

Real-Time Use Cases

- Implementing polynomial expressions
- Maintaining symbol tables in compilers
- File directory structures
- Custom memory management blocks
- Student databases with dynamic size

The five primary types are categorized by their links:

- Linear: Includes the basic Singly (forward-only) and Doubly (forward/backward) lists, along with the Grounded type (ending in NULL).
- Cyclic: Includes the Circular list (tail links to header) and the advanced Circular Two-way list (fully looped and bidirectional).

Types of Header Linked List

1. Singly or Doubly Header Linked List
2. Singly or Doubly Circular Header Linked List
3. Ground Header Linked List
4. Two-way Header Linked List
5. Circular Two-way Header Linked List

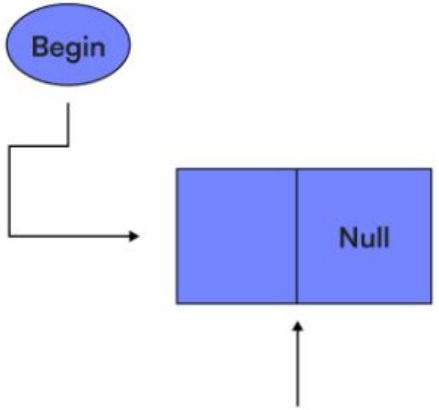
Singly or Doubly Header Linked List

- It is a list whose last node contains the NULL pointer.
- In the header linked list the start pointer always points to the header node.
- start -> next = NULL indicates that the header linked list is empty.
- The operations that are possible on this type of linked list are Insertion, Deletion, and Traversing.

- This structure can be implemented using a singly linked list or a doubly linked list.
- When using a doubly linked list, the header node simplifies bidirectional traversal and manipulation of the list, further enhancing the efficiency of operations like insertion and deletion.

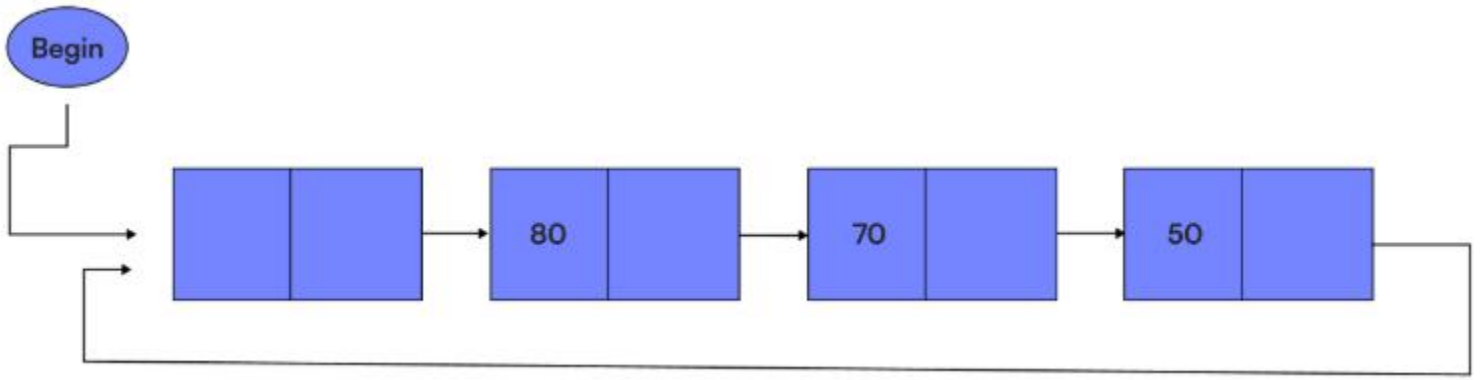
Ground Header Linked List

- A grounded header linked list is a linked list where the last node points to null.
- It is a type of header-linked list with a special node at the beginning, called the header node.
- The header node allows access to all nodes in the list and does not need to represent the same data type as other nodes.



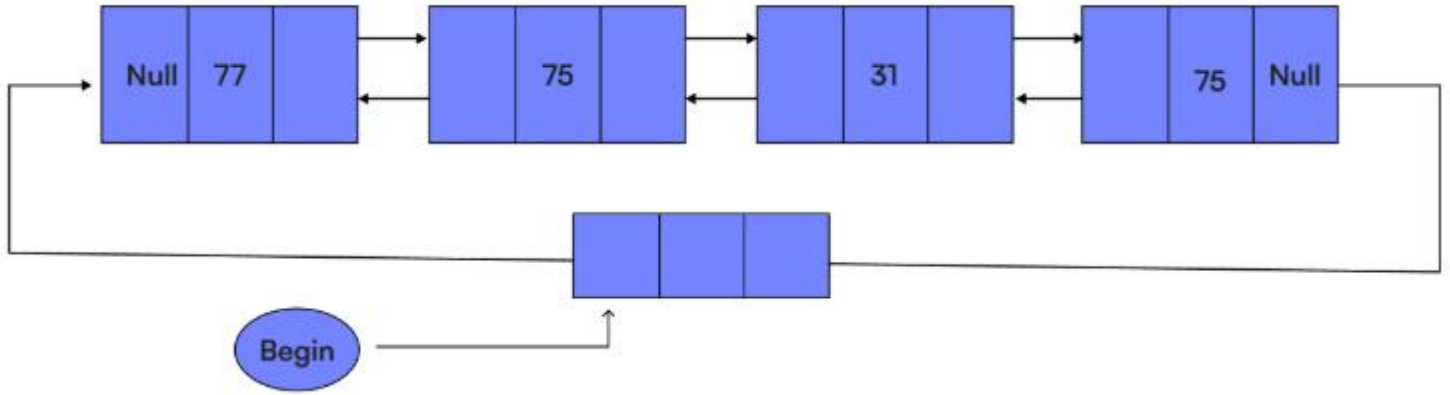
Circular Header Linked List

- A circular header linked list is a header-linked list that has a header node at the beginning of the list and the last node points to the header node.
- This header node is a special node the header node gives access to all the nodes in the linked list.



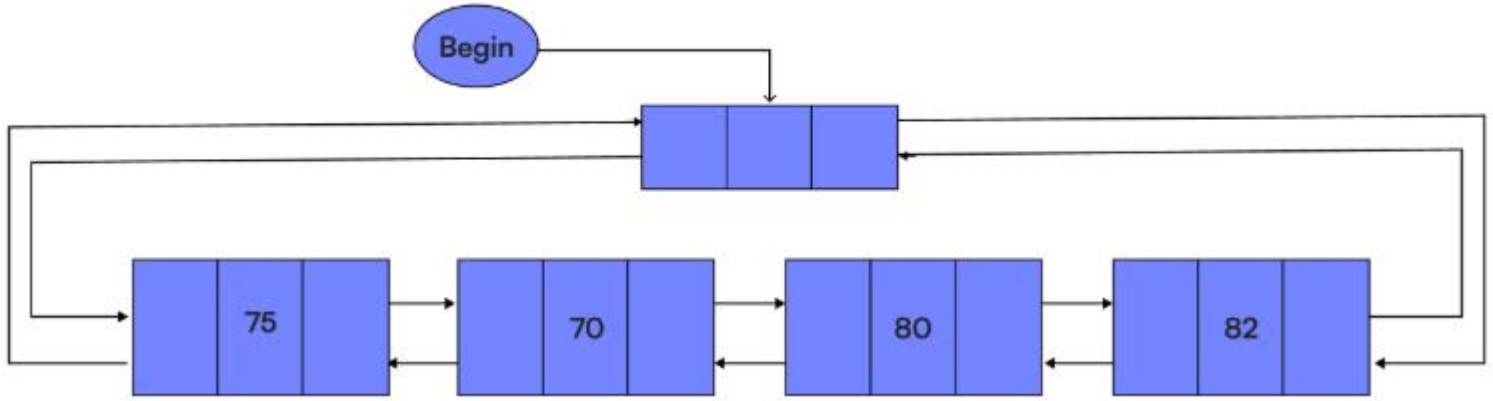
Two-way Linked List

- A Two-Way Header Linked List is a variation of the doubly linked list where there is a special header node that serves as an anchor for both directions (forward and backward) of traversal.



Circular Two-way Linked List

- A Circular Two-Way Header Linked List combines features from both circular lists and doubly linked lists.
- It has a header node, and both the next and previous pointers of the nodes are used.
- Additionally, the list is circular, meaning the last node's next pointer links back to the header, and the first node's previous pointer links back to the header as well



// C program to implement header linked list

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
struct Node* createNode(int newData);
```

// Inserts a node into the linked list

```
void insert(struct Node* header, int x) {

    struct Node* curr = header;
    while (curr->next != NULL)
        curr = curr->next;

    struct Node* newNode = createNode(x);

    curr->next = newNode;
}
```

```
void printList(struct Node* header) {
    struct Node* curr = header->next;
    while (curr != NULL) {
        printf("%d ", curr->data);
        curr = curr->next;
    }
    printf("\n");
}
```

```
struct Node* createNode(int newData) {
    struct Node* node =
    (struct Node*)malloc(sizeof(struct Node));
    node->data = newData;
    node->next = NULL;
    return node;
}
```

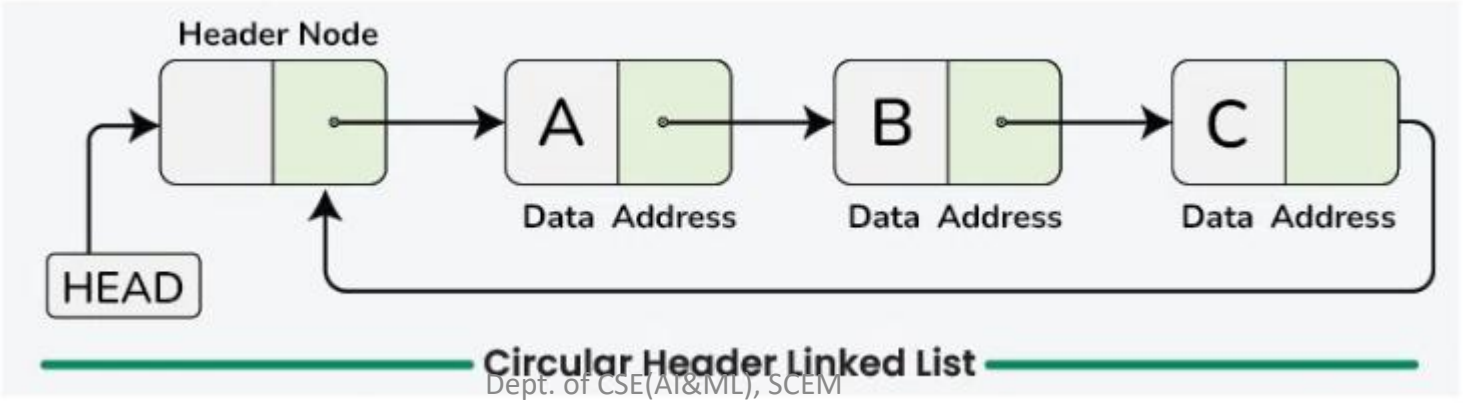
```
int main() {

    struct Node* header = createNode(0);
    insert(header, 1);
    insert(header, 2);
    insert(header, 3);
    insert(header, 4);

    printList(header);

    return 0;
}
```

Output

```
1 2 3 4
```

Singly or Doubly Circular Header Linked List

- A list in which the last node points back to the header node is called circular header linked list.
- The chains do not indicate first or last nodes.
- In this case, external pointers provide a frame of reference because last node of a circular linked list does not contain the NULL pointer.
- The possible operations on this type of linked list are Insertion, Deletion and Traversing.



Simple Representation:

```
struct Node {
    int data;
    struct Node *next;
};
```

Operations

- Insert at Beginning (with header)
- Traversal
- Deleting a Node (by value)

Creating Header List

```
struct Node {
    int data;
    struct Node *next;
};

// Create header node
struct Node* createHeader() {
    struct Node *header = malloc(sizeof(struct Node));
    header->data = 0;      // can store count
    header->next = NULL;
    return header;
}
```

Insert at Beginning

```
void insertBeginning(struct Node *header, int value) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = value;

    newNode->next = header->next;
    header->next = newNode;

    header->data++;   // maintain count
}
```

Traversal

```
void traverse(struct Node *header) {
    struct Node *temp = header->next;

    printf("List (%d nodes): ", header->data);

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

Deleting a Node (by value)

- Algorithm
- Start at header
- Traverse until node with data == key
- Relink: prev->next = temp->next
- Free deleted node
- Reduce count

```
void deleteValue(struct Node *header, int key) {
    struct Node *temp = header->next;
    struct Node *prev = header;

    while (temp != NULL && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) return;
    prev->next = temp->next;
    free(temp);

    header->data--;
}
```

header(data=3) → 10 → 20 → 30 → NULL

temp starts at first real node → 10
prev = header
10 ≠ 20 → move on.

temp moves to 20
prev moves to 10
Found the key!

Unlink the node:
10 → 30
Free the deleted node (20).

Decrease the count:
header->data becomes 2

C program to implement circular header linked list

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
struct Node* createNode(int x);
```

Inserts a node into the linked list.

```
void insert(struct Node* header, int x) {

   // If list is empty
   if (header->next == NULL) {
      struct Node* newNode = createNode(x);
      header->next = newNode;
      newNode->next = newNode;
      return;
   }
```

```
 struct Node* curr = header->next;
    while (curr->next != header->next)
        curr = curr->next;

    struct Node* newNode = createNode(x);

    // Update the previous and new Node
    // next pointers
    curr->next = newNode;
    newNode->next = header->next;
}
```

```
void printList(struct Node* header) {
    if (header->next == NULL)
        return;

    struct Node* curr = header->next;
    do {
        printf("%d ", curr->data);
        curr = curr->next;
    } while (curr != header->next);
    printf("\n");
}
```

```
struct Node* createNode(int x) {
    struct Node* node =
        (struct Node*)malloc(sizeof(struct Node));
    node->data = x;
    node->next = NULL;
    return node;
}
```

```
int main() {

    // Create a hard-coded circular header linked list:
    // header -> 1 -> 2 -> 3 -> 4
    struct Node* header = createNode(0);
    insert(header, 1);
    insert(header, 2);
    insert(header, 3);
    insert(header, 4);

    printList(header);

    return 0;
}
```

Output

```
1 2 3 4
```

Header Linked List in C- Both Insert and Delete

```
#include <stdio.h>
#include <stdlib.h>

// Definition of the node structure
typedef struct Node {
    int data;
    struct Node* next;
} Node;
```

```
// Definition of the linked list with a header node
typedef struct LinkedList {
    Node* header;
} LinkedList;
```

```
// Function to create a new node
Node* create_node(int data) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    if (new_node == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    new_node->data = data;
    new_node->next = NULL;
    return new_node;
}
```

```
LinkedList* create_list() {
    LinkedList* list = (LinkedList*)malloc(sizeof(LinkedList));
    if (list == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    // Creating a header node without any meaningful data (data = -1)
    list->header = create_node(-1);
    return list;
}
```

```
void insert_end(LinkedList* list, int data) {
    Node* new_node = create_node(data);
    Node* temp = list->header;

    // Traverse to the last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    // Insert the new node at the end
    temp->next = new_node;
}
```

Function to display the list

```
void display_list(LinkedList* list) {
    Node* temp = list->header->next;  // Skip the header node
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Function to delete a node by its value

```
void delete_node(LinkedList* list, int value) {
    Node* temp = list->header;
    while (temp->next != NULL && temp->next->data != value) {
        temp = temp->next;
    }
    if (temp->next != NULL) {
        Node* to_delete = temp->next;
        temp->next = temp->next->next;
        free(to_delete);
        printf("Node with value %d deleted.\n", value);
    } else {
        printf("Node with value %d not found.\n", value);
    }
}
```

```
// Function to free the entire list
void free_list(LinkedList* list) {
    Node* temp = list->header;
    while (temp != NULL) {
        Node* next = temp->next;
         free(temp);
        temp = next;
    }
    free(list);
}
```

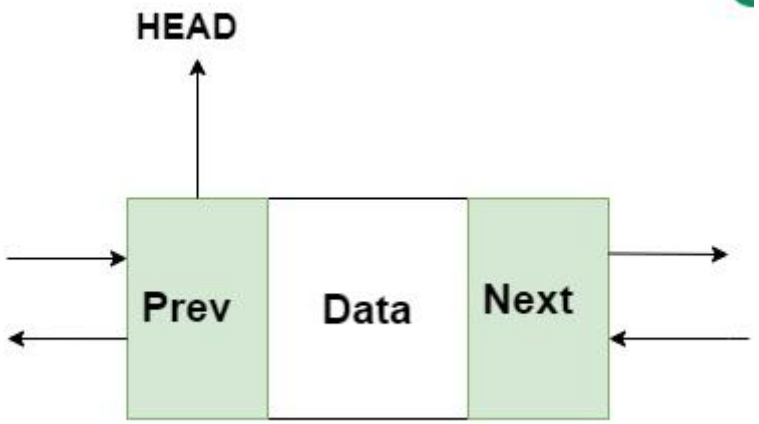
```
int main() {
    // Create a new list
    LinkedList* list = create_list();
    // Insert elements into the list
    insert_end(list, 10);
    insert_end(list, 30);
    insert_end(list, 40);
    insert_end(list, 50);
    // Display the list
    printf("Linked List: ");
    display_list(list);
    // Delete a node
    delete_node(list, 30);
    printf("Linked List after deletion: ");
    display_list(list);
    // Free the list memory
    free_list(list);
    return 0;
}
```

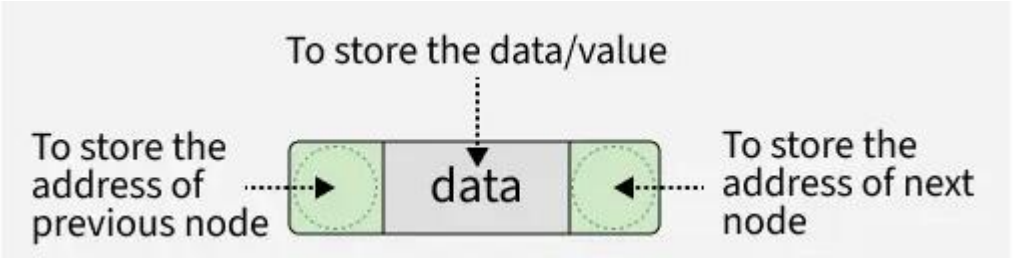
Output

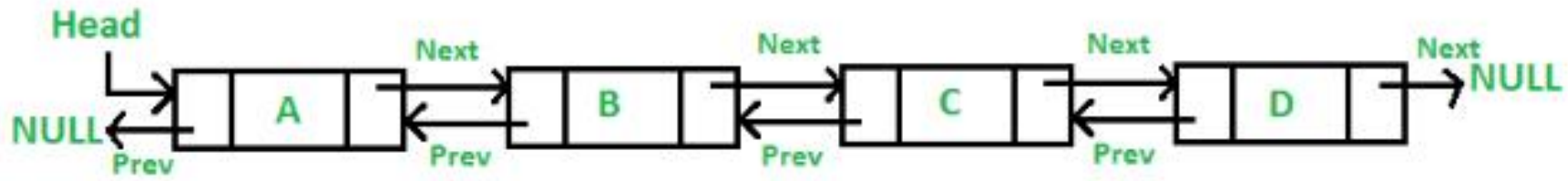
```
The Linked List: 10 -> 30 -> 40 -> 50 -> NULL
Node with value 30 deleted.
The Linked List after deletion: 10 -> 40 -> 50 -> NULL
```

Doubly Linked List

- A doubly linked list is a type of linked list in which each node contains 3 parts, a data part and two addresses, one points to the previous node and one for the next node.
- The main advantage of a doubly linked list is that it allows for efficient traversal of the list in both directions.
- This is because each node in the list contains a pointer to the previous node and a pointer to the next node.








```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
}
```

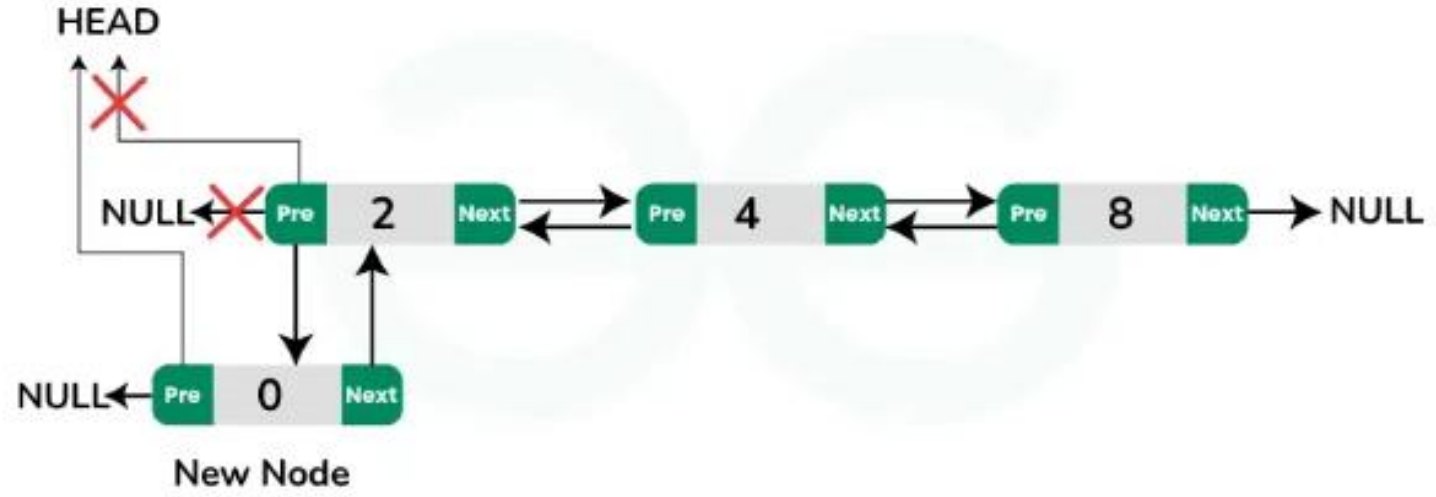
Basic Operations on C Doubly Linked List

- Insertion
- Deletion
- Traversal

Insertion in Doubly Linked List in C

- Insertion at the beginning of the list.
- Insertion at the end of the list.
- Insertion in the middle of the list.

Insertion at the Beginning of the Doubly Linked List

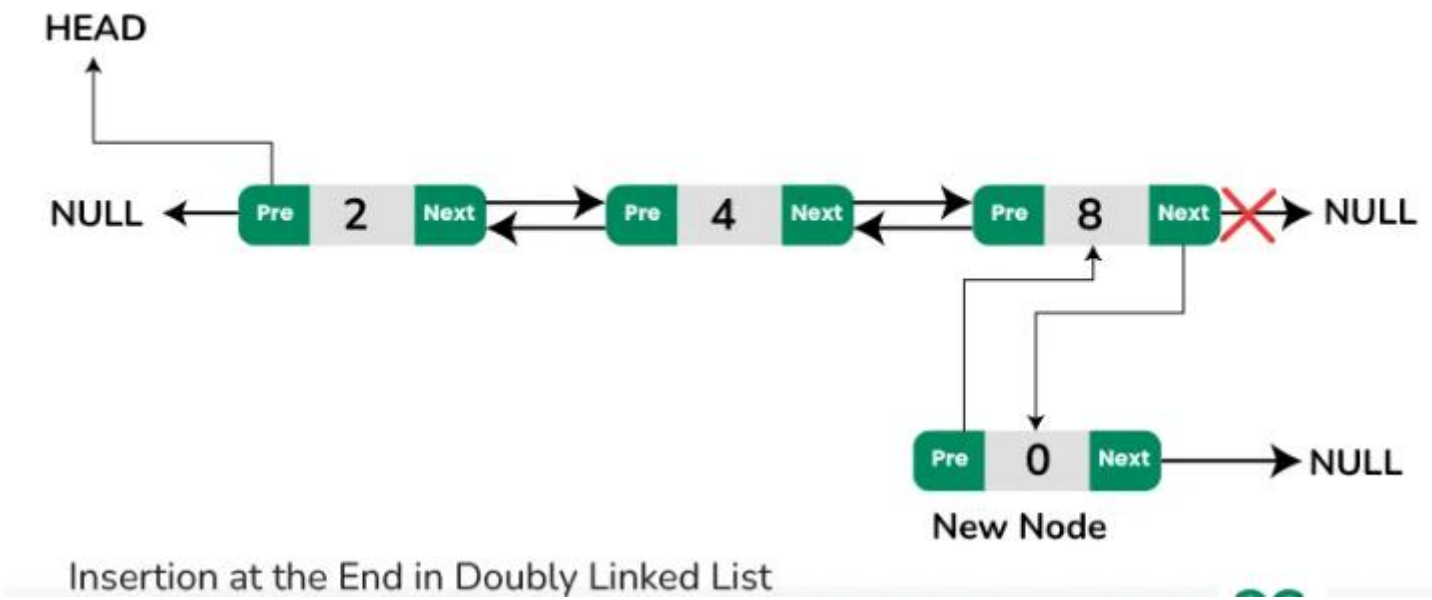


Insertion at the Beginning in Doubly Linked List

Approach:

- Create a newNode with data field assigned the given value.
- If the list is empty:
- Set newNode->next to NULL.
- Set newNode->prev to NULL.
- Update the head pointer to point to newNode.
- If the list is not empty:
- Set newNode->next to head.
- Set newNode->prev to NULL.
- Set head->prev to newNode.
- Update the head pointer to point to newNode.

Insertion at the End of the Doubly Linked List

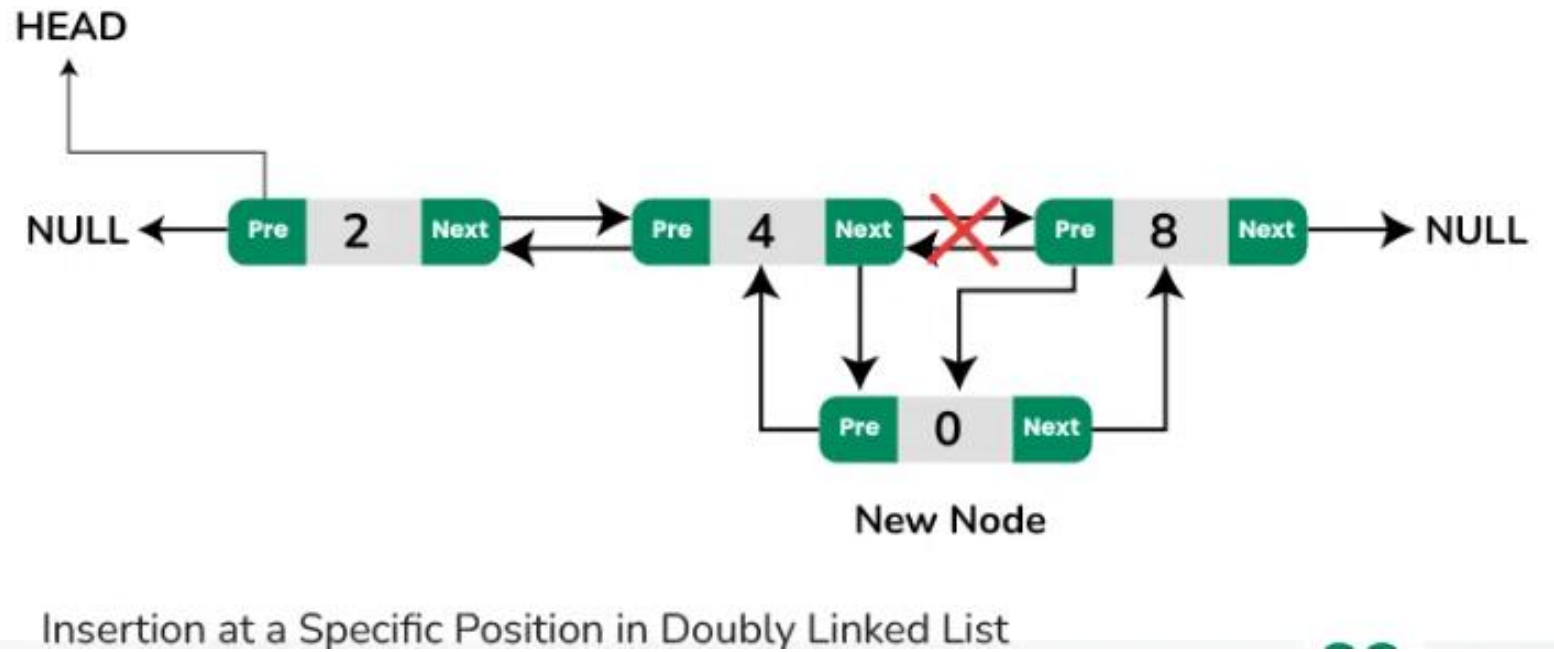


Insertion at the End in Doubly Linked List

Approach:

- Create a newNode with data field assigned the given value.
- If the list is empty:
- Set newNode->next to NULL.
- Set newNode->prev to NULL.
- Update the head pointer to point to newNode.
- If the list is not empty:
- Traverse the list to find the last node (where last->next == NULL).
- Set last->next to newNode.
- Set newNode->prev to last.
- Set newNode->next to NULL.

Insertion in the Middle of the Doubly Linked List



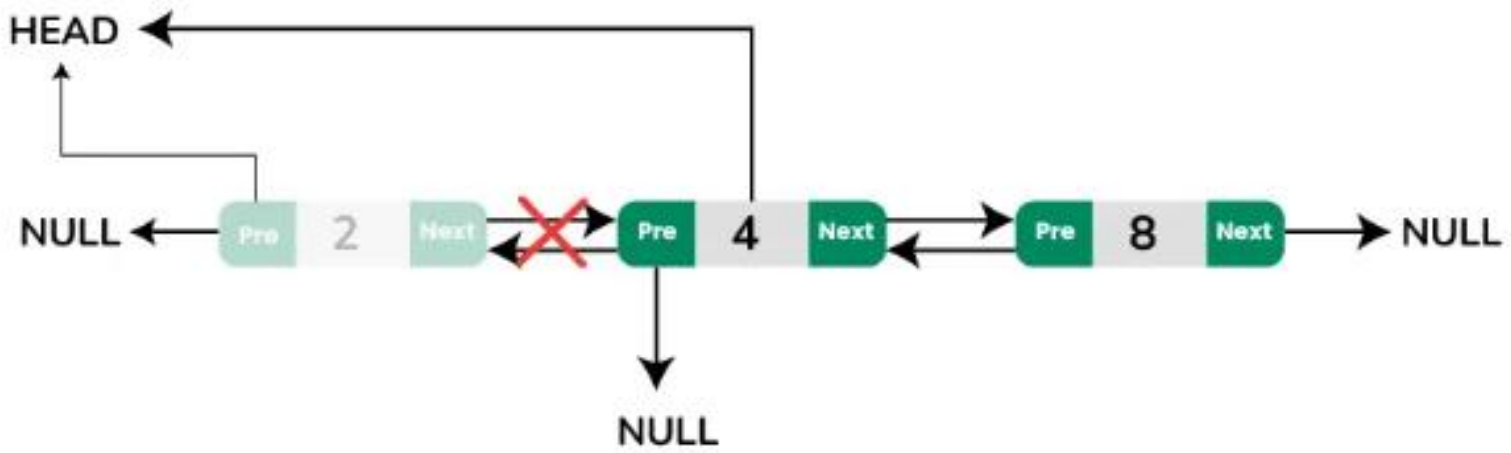
Approach:

- Create a newNode.
- Find the size of the list to ensure the position is within valid bounds.
- If the position is 0 (or 1 depending on your indexing):
- Use the insertion at the beginning approach.
- If the position is equal to the size of the list:
- Use the insertion at the end approach.
- If the position is valid and not at the boundaries:
- Traverse the list to find the node immediately before the desired insertion point (prevNode).
- Set newNode->next to prevNode->next.
- Set newNode->prev to prevNode.
- If prevNode->next is not NULL, set prevNode->next->prev to newNode.
- Set prevNode->next to newNode.

Deletion in a Doubly Linked List in C

- Deletion from the beginning of the list.
- Deletion from the end of the list.
- Deletion at specific position of the list.

Deletion at the Beginning of the Doubly Linked List

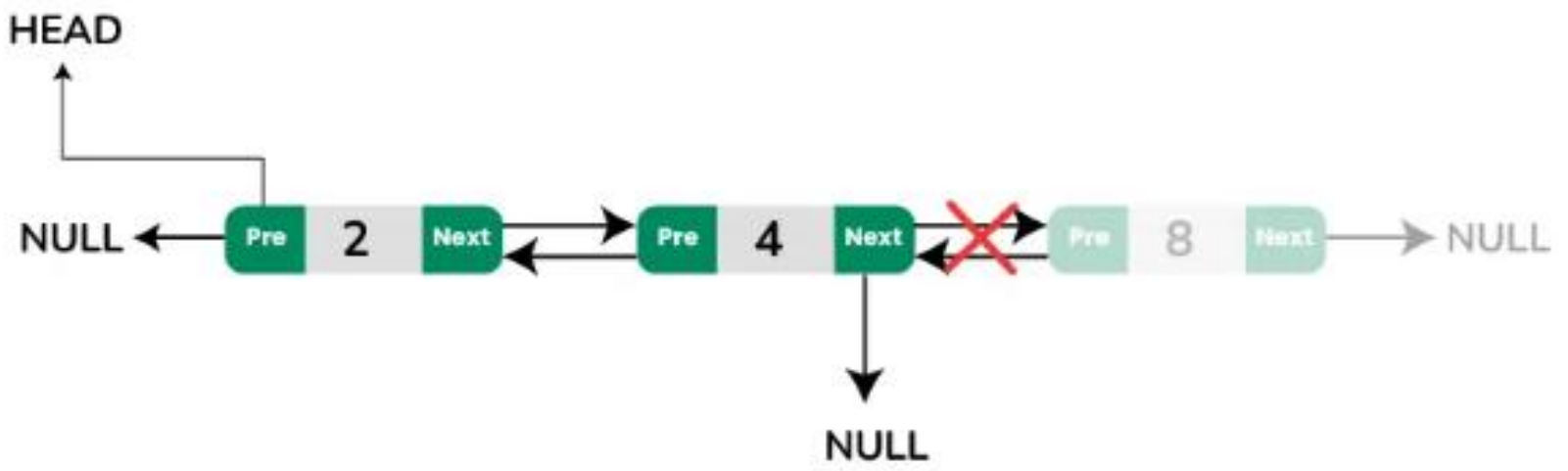


Deletion at the Beginning in Doubly Linked List

Approach:

- Check if the List is Empty:
- If true, there's nothing to delete.
- Check if the List Contains Only One Node:
- Set head to NULL.
- Free the memory of the node.
- If the List Contains More Than One Node:
- Update head to head->next.
- Set the prev of the new head to NULL.
- Free the memory of the old head.

Deletion at the End of the Doubly Linked List

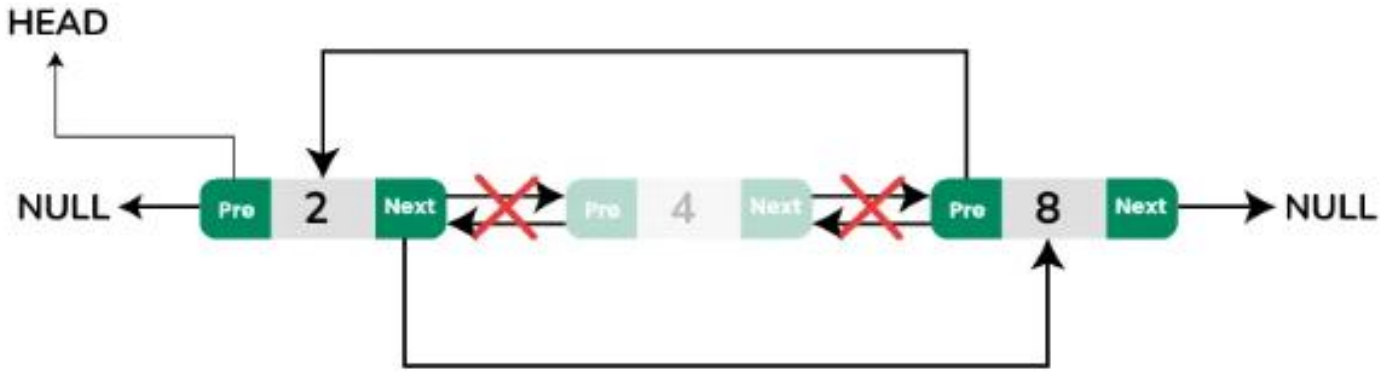


Deletion at the End in Doubly Linked List

Approach:

- Check if the List is Empty:
- If true, there's nothing to delete.
- If the List is Not Empty:
- Traverse to the last node using a loop.
- Check if the last node is the only node (head->next == NULL):
- Update head to NULL.
- If more than one node:
- Set the next of the second last node (last->prev) to NULL.
- Free the memory of the last node.

Deletion at a Specific Position in Doubly Linked List



Deletion at a Specific Position in Doubly Linked List

Approach:

- Check Position Validity:
- If position is 0 (or 1 depending on indexing), use deletion at the beginning.
- If position is the size of the list, use deletion at the end.
- Otherwise, proceed with the middle deletion.
- Traverse to the Specified Position:
- Use a loop to reach the node (delNode) at the desired position.
- If delNode is the first node, adjust head.
- If not, adjust delNode->prev->next and delNode->next->prev if delNode->next is not NULL.
- Free the Memory:
- Release the node to be deleted.

Traversal in a Doubly Linked List in C

- Traversal in a doubly linked list means visiting each node of the doubly linked list to perform some kind of operation like displaying the data stored in that node.
- Forward Traversal: From head node to tail node
- Reverse Traversal: From tail node to head node

Forward Traversal in Doubly Linked List

- Approach:

- Create a temporary pointer ptr and copy the head pointer into it.
- Traverse through the list using a while loop.
- Keep moving the value of temp pointer variable ptr to ptr->next until the last node whose next part contains null is found.
- During each iteration of the loop, perform the desired operation.

Reverse Traversal in Doubly Linked List

- Create a temporary pointer ptr and copy the tail pointer into it.
- Traverse through the list using a while loop.
- Keep moving the value of temp pointer variable ptr to ptr->prev until the first node whose prev part contains null is found.
- During each iteration of the loop, perform the desired operation .

C Program to Implement Doubly Linked List

```
#include <stdio.h>
#include <stdlib.h>

// defining a node
typedef struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
} Node;
```

```
// Function to create a new node with given value as data
Node* createNode(int data)
{
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}
```

// Function to insert a node at the beginning

```
void insertAtBeginning(Node** head, int data)
{
    // creating new node
    Node* newNode = createNode(data);

    // check if DLL is empty
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    newNode->next = *head;
    (*head)->prev = newNode;
    *head = newNode;
}
```

// Function to insert a node at the end

```
void insertAtEnd(Node** head, int data)
{
    // creating new node
    Node* newNode = createNode(data);
    // check if DLL is empty
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
```

// Function to insert a node at a specified position

```
void insertAtPosition(Node** head, int data, int position)
{
    if (position < 1) {
        printf("Position should be >= 1.\n");
        return;
    }

    // if we are inserting at head
    if (position == 1) {
        insertAtBeginning(head, data);
        return;
    }
```

```
Node* newNode = createNode(data);
    Node* temp = *head;
    for (int i = 1; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf(
            "Position greater than the number of nodes.\n");
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}
```

// Function to delete a node from the beginning

```
void deleteAtBeginning(Node** head)
{
    // checking if the DLL is empty
    if (*head == NULL) {
        printf("The list is already empty.\n");
        return;
    }
    Node* temp = *head;
    *head = (*head)->next;
    if (*head != NULL) {
        (*head)->prev = NULL;
    }
    free(temp);
}
```

// Function to delete a node from the end

```
void deleteAtEnd(Node** head)
{
    // checking if DLL is empty
    if (*head == NULL) {
        printf("The list is already empty.\n");
        return;
    }
```

```
Node* temp = *head;
    if (temp->next == NULL) {
        *head = NULL;
        free(temp);
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->prev->next = NULL;
    free(temp);
}
```

// Function to delete a node from a specified position

```
void deleteAtPosition(Node** head, int position)
{
    if (*head == NULL) {
        printf("The list is already empty.\n");
        return;
    }
    Node* temp = *head;
    if (position == 1) {
        deleteAtBeginning(head);
        return;
    }
```

```
for (int i = 1; temp != NULL && i < position; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position is greater than the number of "
               "nodes.\n");
        return;
    }
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    }
    free(temp);
}
```

// Function to print the list in forward direction

```
void printListForward(Node* head)
{
    Node* temp = head;
    printf("Forward List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

// Function to print the list in reverse direction

```
void printListReverse(Node* head)
{
    Node* temp = head;
    if (temp == NULL) {
        printf("The list is empty.\n");
        return;
    }
```

```
// Move to the end of the list
    while (temp->next != NULL) {
        temp = temp->next;
    }
    // Traverse backwards
    printf("Reverse List: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}
```

```
int main()
{
    Node* head = NULL;

    // Demonstrating various operations
    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtBeginning(&head, 5);
    insertAtPosition(&head, 15, 2); // List: 5 15 10 20

    printf("After Insertions:\n");
    printListForward(head);
    printListReverse(head);

    deleteAtBeginning(&head); // List: 15 10 20
    deleteAtEnd(&head); // List: 15 10
    deleteAtPosition(&head, 2); // List: 15

    printf("After Deletions:\n");
    printListForward(head);

    return 0;
}
```

Output

```
After Insertions:
Forward List: 5 15 10 20
Reverse List: 20 10 15 5
After Deletions:
Forward List: 15
```

Advantages of Doubly Linked List

- Unlike singly linked list, we can traverse in both the direction forward and reverse using doubly linked list which makes operations like reversing and searching from end easier and more efiicient.
- It is fast and easier to delete a node in doubly linked list, if the node to be deleted is given then we can search the previous node quickly and update it's links.
- Inserting a new node is also easier in doubly linked list, as we can simply insert a new node before a given node by updating the links.
- In a doubly linked list, we need not to keep track of the previous node during traversal that is very useful in data structures like the binary tree where we need to keep track of the parent node.

Disadvantages of Doubly Linked List

- Although doubly linked lists provide more flexibility and efficiency in certain operations compared to singly linked lists, they also consume more memory as they need to store an extra pointer for each node.
- The insertion and deletion code in a doubly linked list is more complex as we need to maintain the previous links too.
- For insertion and deletion at a specific position in a doubly linked list we need to traverse from a head node to the specified node that can be time-consuming in case of larger lists.
- For each node we have to maintain two pointers in a doubly linked list that leads to the overhead of maintaining and updating these pointers during insertions and deletions.

- Develop C functions for the following :
- i) Reverse a double Linked list
- ii) Count the number of nodes in the linked list

Develop C Functions for the following

- Inserting a node at the beginning of a Doubly linked list
- Deleting a node at the end of theDoubly linked list.

Circular linked lists

- A circular linked list is a data structure where the last node points back to the first node, forming a closed loop.
- Structure: All nodes are connected in a circle, enabling continuous traversal without encountering NULL.
- Uses: Ideal for tasks like scheduling and managing playlists

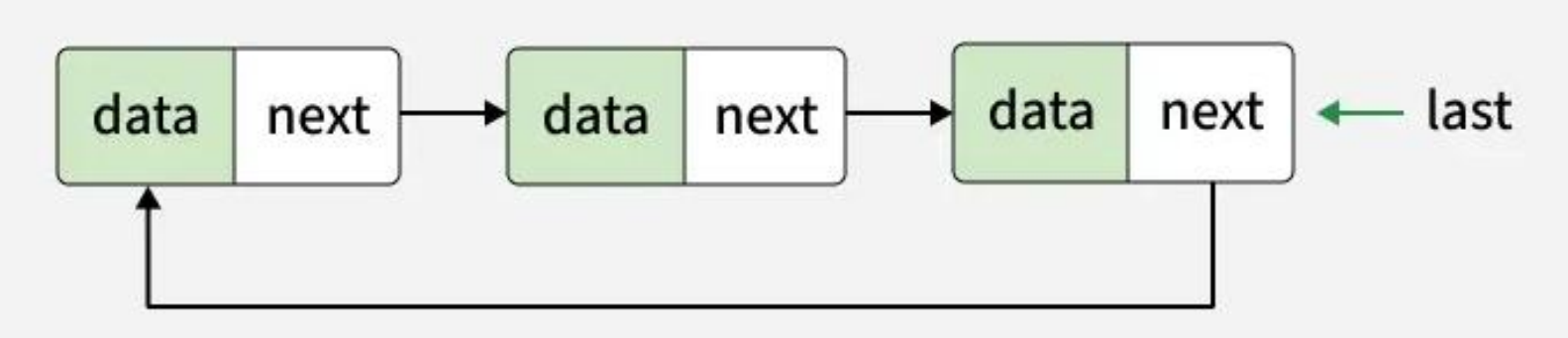
Types of Circular Linked Lists

Two types:

1. Circular Singly Linked List
2. Circular Doubly Linked List

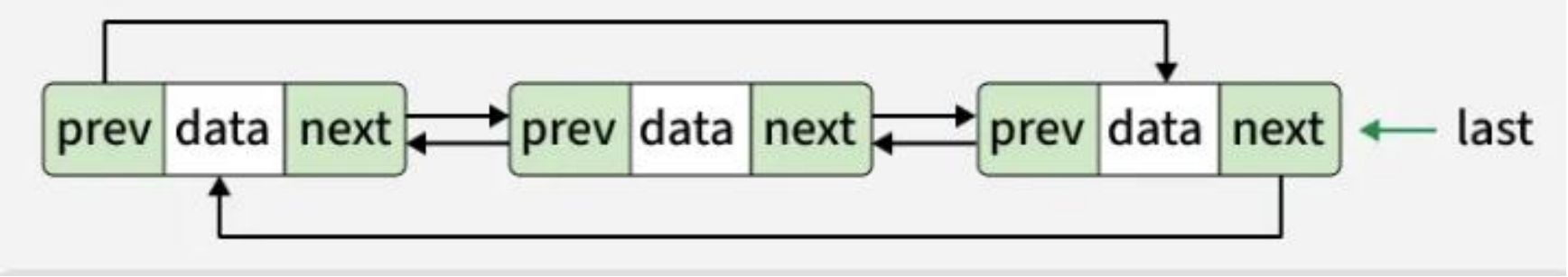
1. Circular Singly Linked List

- Each node has just one pointer called the "next" pointer.
- The next pointer of the last node points back to the first node and this results in forming a circle.
- In this type of Linked list, we can only move through the list in one direction.

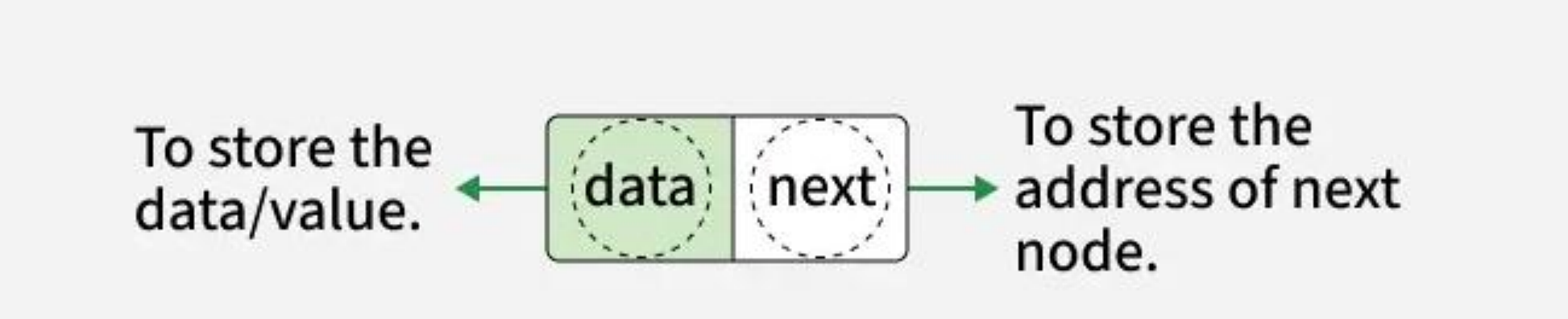


2. Circular Doubly Linked List

- Each node has two pointers prev and next, similar to doubly linked list.
- The prev pointer points to the previous node and the next points to the next node.
- Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.



Representation of a Circular Singly Linked List



```
struct Node {
    int data;
    struct Node* next;
};
```

Basic Operations on Circular Linked List

1. Insertion:
   - Insertion at the end
   - Insertion at the Beginning
   - Insertion in the Specific Position
2. Deletion:
   - Deletion from the Beginning
   - Deletion from the End
   - Deletion from the Specific Position
3. Traversal
4. Search

Implementation of InsertAtBegin

- To insert a node at the beginning of a circular linked list:

- Create a new node with the given data.
- If the list is empty, set the new node to itself and make it the head node.
- If the list is not empty, traverse to the last node and update its next pointer to point to the new node. Then update the head pointer to the new node.'

Implementation of InsertAtEnd

- To insert a node at the end of a circular linked list:

- Create a new node with the given data.
- If the list is empty, set the new node to itself and make it the head node.
- If the list is not empty, traverse to the last node.
- Set the last node's next pointer to the new node.
- Set the new node's next pointer to the head to maintain circularity.

Implementation of InsertAtSpecificPosition

- To insert a node at a specific position in a circular linked list:

- Create a new node with the given data.
- If the list is empty and position is 0, set the new node to itself and make it the head node.
- Traverse to the node just before the desired position.
- Insert the new node by adjusting pointers.

Implementation of DeleteFromBeginning

- To delete a node from the beginning of a circular linked list:

- If the list is empty, return.
- If the list has only one node, delete it and set head to NULL.
- Traverse to the last node and update its next pointer.
- Free the memory allocated to the node to be deleted.

- To delete a node from the end of a circular linked list:

- If the list is empty, return.
- If the list has only one node, delete it and set head to NULL.
- Traverse to the second last node and update its next pointer to NULL.
- Free the memory allocated to the last node.

- To delete a node from a specific position in a circular linked list:

- If the list is empty, return.
- Traverse to the node just before the desired position.
- Adjust pointers to skip the node to be deleted.
- Free the memory allocated to the node to be deleted.

Implementation of Traversal

- Check if list is empty then if it is return.
- Initialize the temporary pointer to the head.
- Loop through the list starting from head and print the each nodes data.
- Continue until the pointer reaches back to head.

- To search for a node with a given key in a circular linked list:

- Start from the head node.
- Traverse each node and compare its data with the given key until the key is found or you reach the head again.

Program to Implement Circular Linked List in C

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};
```

// Function to create a new node

```
struct Node* createNode(int data)
{
    struct Node* newNode
        = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

// Function to insert a node at the beginning

```
void insertAtBeginning(struct Node** head, int data)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
    }
```

```
else {
        struct Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = *head;
        *head = newNode;
    }
}
```

// Function to insert a node at the end

```
void insertAtEnd(struct Node** head, int data)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        newNode->next = *head;
    }
```

```
 else {
        struct Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = *head;
    }
}
```

// Function to insert a node at a specific position

```
void insertAtPosition(struct Node** head, int data, int position)
{
    struct Node* newNode = createNode(data);
    if (*head == NULL && position == 0) {
        *head = newNode;
        newNode->next = *head;
    }
    else if (position == 0) {
        insertAtBeginning(head, data);
    }
```

```
else {
        struct Node* temp = *head;
        int i = 0;
        while (i < position - 1) {
            temp = temp->next;
            i++;
        }
        newNode->next = temp->next;
        temp->next = newNode;
    }
}
```

// Function to delete a node from the beginning

```
void deleteFromBeginning(struct Node** head)
{
    if (*head == NULL) {
        return;
    }
    else if ((*head)->next == *head) {
        free(*head);
        *head = NULL;
    }
```

```
else {
        struct Node* temp = *head;
        while (temp->next != *head) {
            temp = temp->next;
        }
        temp->next = (*head)->next;
        struct Node* toDelete = *head;
        *head = (*head)->next;
        free(toDelete);
    }
}
```

// Function to delete a node from the end

```
void deleteFromEnd(struct Node** head)
{
    if (*head == NULL) {
        return;
    }
    else if ((*head)->next == *head) {
        free(*head);
        *head = NULL;
    }
```

```
else {
        struct Node* secondLast = *head;
        while (secondLast->next->next != *head) {
            secondLast = secondLast->next;
        }
        struct Node* last = secondLast->next;
        secondLast->next = *head;
        free(last);
    }
}
```

// Function to delete a node from a specific position

```
void deleteAtPosition(struct Node** head, int position)
{
    if (*head == NULL) {
        return;
    }
    else if (position == 0) {
        deleteFromBeginning(head);
    }
```

```
else {
        struct Node* temp = *head;
        int i = 0;
        while (i < position - 1) {
            temp = temp->next;
            i++;
        }
        struct Node* toDelete = temp->next;
        temp->next = temp->next->next;
        free(toDelete);
    }
}
```

// Function to traverse and print the circular linked list

```
void traverse(struct Node* head)
{
    if (head == NULL) {
        return;
    }
    struct Node* temp = head;
    do {
        printf("%d -> ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("HEAD\n");
}
```

// Function to search for a node with a given key

```
int search(struct Node* head, int key)
{
    if (head == NULL) {
        return 0;
    }
    struct Node* temp = head;
    do {
        if (temp->data == key) {
            return 1; // Key found
        }
        temp = temp->next;
    } while (temp != head);
    return 0; // Key not found
}
```

```
int main()
{
    struct Node* head = NULL;

    // Insertion
    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtBeginning(&head, 5);
    insertAtPosition(&head, 15, 2);
```

```
// Traversal
    printf("Circular Linked List: ");
    traverse(head);

    // Deletion
    deleteFromEnd(&head);
    deleteAtPosition(&head, 1);

// Traversal after deletion
    printf("Circular Linked List after deletion: ");
    traverse(head);
```

```
// Searching
    int key = 10;
    if (search(head, key)) {
        printf("Element %d is found in the linked list.\n",
               key);
    }
    else {
        printf(
            "Element %d is not found in the linked list.\n",
            key);
    }
    return 0;
}
```

Output

```
Circular Linked List: 5 -> 10 -> 15 -> 20 -> HEAD
Circular Linked List after deletion: 5 -> 15 -> HEAD
Element 10 is not found in the linked list.
```

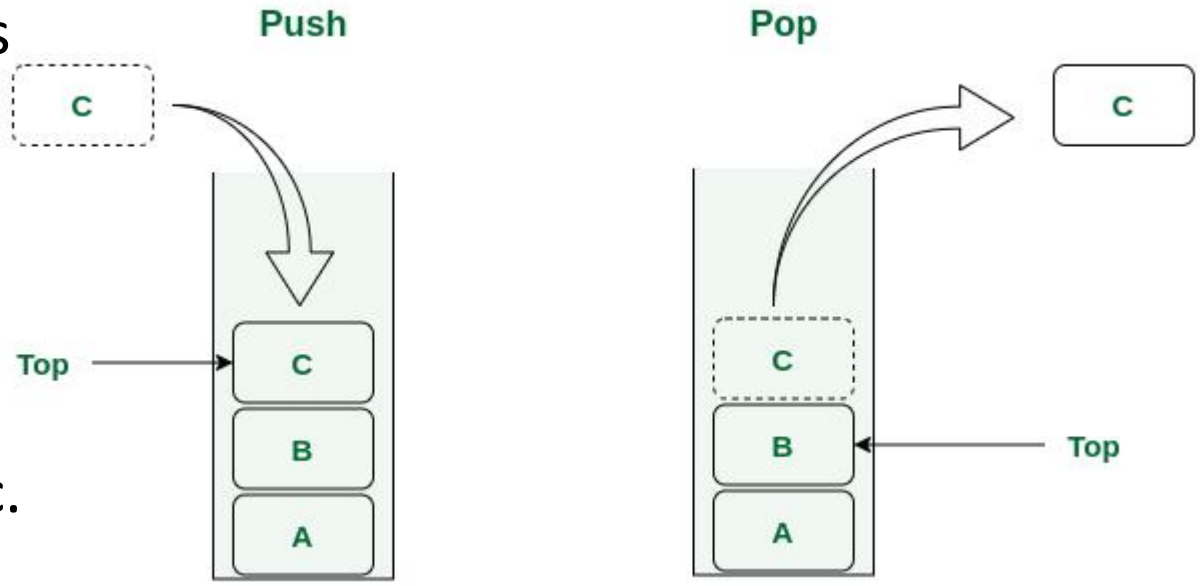
Applications of Circular Linked List

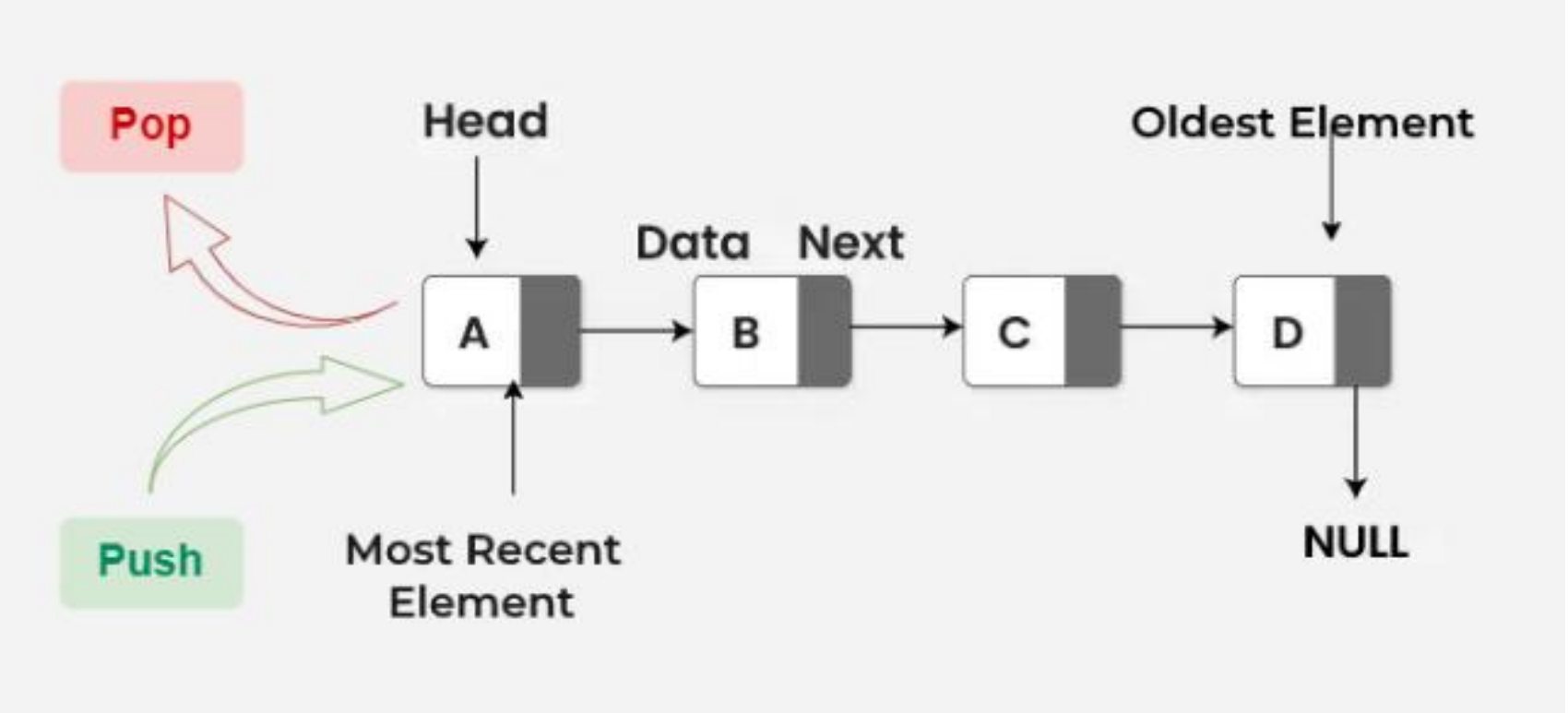
- Round Robin Scheduling: It can be used in the operating system to schedule a process in the cyclic manner.
- FIFO Buffers: It can be used in networking for the buffer management.
- Music Player Playlists: It can be used to cycle through the songs in a playlist.

C program to concatenate 2 lists:

Linked Stacks

- A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle.
- Stack is generally implemented using an array but the limitation of this kind of stack is that the memory occupied by the array is fixed no matter what are the number of elements in the stack.
- It can be implemented using a linked list, where each element of the stack is represented as a node and its size is dynamic.
- The head of the linked list acts as the top of the stack.




Pop
Head
Oldest Element
Data
Next
A
B
C
D
Push
Most Recent Element
NULL

Representation of Linked Stack in C

```
struct Node {
    type data;
    Node* next;
}
```

Basic Operations

- Push: This operation is used to add/insert/push data into the stack.
- Pop: This operation is used to delete/remove/pop data into the stack.
- Peek: This operation returns the top element in the stack.
- isEmpty: Returns true if the stack is empty, false otherwise.

Push Function

Algorithm for Push Function

- Create a new node with the given data.
- Insert the node before the head of the linked list.
- Update the head of the linked list to the new node.

Pop Function

Algorithm for Pop Function

- Check if the stack is empty.
- If not empty, store the top node in a temporary variable.
- Update the head pointer to the next node.
- Free the temporary node.

Peek Function

Algorithm for Peek Function

- Check if the stack is empty.
- If its empty, return -1.
- Else return the head->data.

IsEmpty Function

Algorithm of isEmpty Function

- Check if the top pointer of the stack is NULL.
- If NULL, return true, indicating the stack is empty.
- Otherwise return false indicating the stack is not empty.

C Program to Implement a Stack Using Linked List

```
#include <stdio.h>
#include <stdlib.h>
// Define the structure for a node of the linked list
typedef struct Node {
    int data;
    struct Node* next;
} node;
```

Linked List Create node function

```
node* createNode(int data)
{
   // allocating memory
   node* newNode = (node*)malloc(sizeof(node));

   // if memory allocation is failed
   if (newNode == NULL)
      return NULL;

   // putting data in the node
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}
```

// function to insert data before the head node

```
int insertBeforeHead(node** head, int data)
{
    // creating new node
    node* newNode = createNode(data);

// if malloc fail, return error code
    if (!newNode)
        return -1;

    // if the linked list is empty
    if (*head == NULL) {
        *head = newNode;
        return 0;
    }
    newNode->next = *head;
    *head = newNode;
    return 0;
}
```

// deleting head node

```
int deleteHead(node** head)
{
    node* temp = *head;
   *head = (*head)->next;
   free(temp);
   return 0;
}
```

// Function to check if the stack is empty or not

```
int isEmpty(node** stack)
 {
return *stack == NULL;
 }
```

// Function to push elements to the stack

```
void push(node** stack, int data)
{
      if (insertBeforeHead(stack, data))
   {
       printf("Stack Overflow!\n");
    }
}
```

// Function to pop an element from the stack

```
int pop(node** stack)
{
    if (isEmpty(stack)) {
        printf("Stack Underflow\n");
        return -1;
    }
    deleteHead(stack);
}
```

// Function to return the topmost element of the stack

```
int peek(node** stack)
{

    if (!isEmpty(stack))
        return (*stack)->data;
    else
        return -1;
}
```

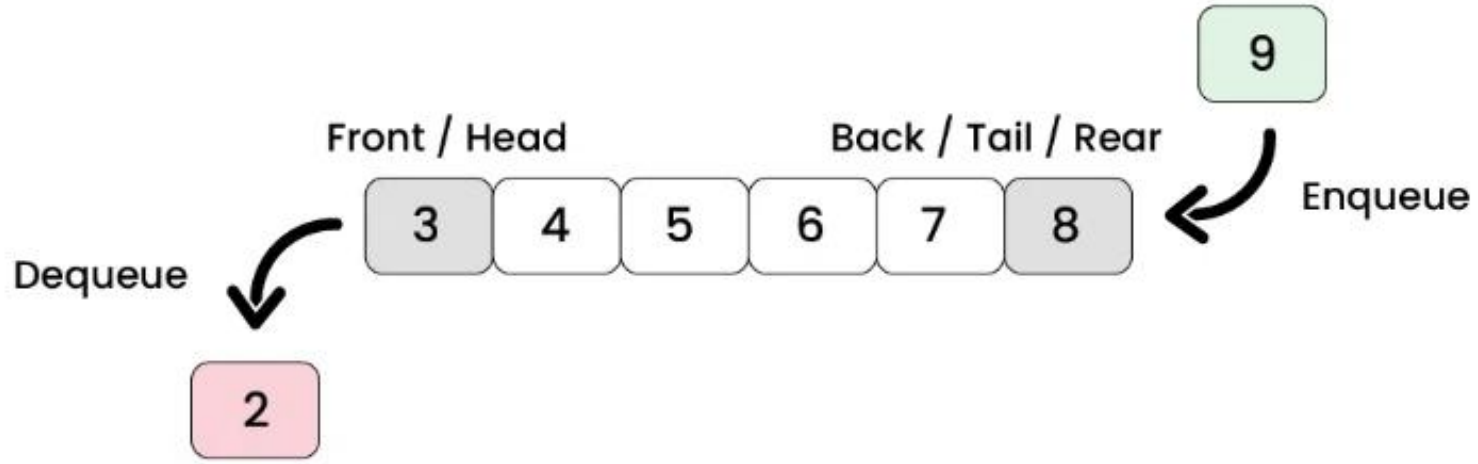
// Function to print the Stack

```
void printStack(node** stack)
{
    node* temp = *stack;
    while (temp != NULL) {
        printf("%d-> ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
int main()
{
    // Initialize a new stack top pointer
    node* stack = NULL;

    // Push elements into the stack
    push(&stack, 10);
    push(&stack, 20);
    push(&stack, 30);
    push(&stack, 40);
    push(&stack, 50);

    // Print the stack
    printf("Stack: ");
    printStack(&stack);

    pop(&stack);
    pop(&stack);

    printf("\nStack: ");
    printStack(&stack);

    return 0;
}
```

Linked Queue

- Queue is a linear data structure that follows the First-In-First-Out (FIFO) order of operations.
- This means the first element added to the queue will be the first one to be removed.



Basic Operations

- isEmpty
- Enqueue
- Dequeue
- Peek

Enqueue Function

Algorithm for Enqueue Function

- Create a new node with the given data.
- If the queue is empty, set the front and rear to the new node.
- Else, set the next of the rear to the new node and update the rear.

Dequeue Function

- Algorithm for Dequeue Function

Check if the queue is empty.

If not empty, store the front node in a temporary variable.

Update the front pointer to the next node.

Free the temporary node.

If the queue becomes empty, update the rear to NULL.

Peek Function

Algorithm for Peek Function

- Check if the queue is empty.
- If empty, return -1.
- Else, return the front->data.

IsEmpty Function

Algorithm of isEmpty Function

- Check if the front pointer of the queue is NULL.
- If NULL, return true, indicating the queue is empty.
- Otherwise, return false, indicating the queue is not empt

C Program to Implement a Queue Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a node of the linked list
typedef struct Node
{
   int data;
   struct Node *next;
} node;

// Define the structure for the queue
typedef struct Queue
{
   node *front;
   node *rear;
} queue;
```

// Function to create a new node

```
node *createNode(int data)
{
   // Allocate memory for a new node
   node *newNode = (node *)malloc(sizeof(node));
   // Check if memory allocation was successful
   if (newNode == NULL)
      return NULL;
   // Initialize the node's data and next pointer
   newNode->data = data;
   newNode->next = NULL;
   return newNode;
}
```

// Function to create a new queue

```
queue *createQueue()
{
   // Allocate memory for a new queue
   queue *newQueue = (queue *)malloc(sizeof(queue));
   // Initialize the front and rear pointers of the queue
   newQueue->front = newQueue->rear = NULL;
   return newQueue;
}
```

// Function to check if the queue is empty

```
int isEmpty(queue *q)
{
   // Check if the front pointer is NULL
   return q->front == NULL;
}
```

// Function to add an element to the queue

```
void enqueue(queue *q, int data)
{
   // Create a new node with the given data
   node *newNode = createNode(data);
   // Check if memory allocation for the new node was
   // successful
   if (!newNode)
   {
      printf("Queue Overflow!\n");
      return;
   }
// If the queue is empty, set the front and rear
   // pointers to the new node
   if (q->rear == NULL)
   {
      q->front = q->rear = newNode;
      return;
   }
   // Add the new node at the end of the queue and
update
   // the rear pointer
   q->rear->next = newNode;
   q->rear = newNode;
}
```

// Function to remove an element from the queue

```
int dequeue(queue *q)
{
   // Check if the queue is empty
   if (isEmpty(q))
   {
      printf("Queue Underflow\n");
      return -1;
   }
```

// Store the front node and update the front pointer

```
    node *temp = q->front;
    q->front = q->front->next;
    // If the queue becomes empty, update the rear pointer
    if (q->front == NULL)
        q->rear = NULL;
    // Store the data of the front node and free its memory
    int data = temp->data;
    free(temp);
    return data;
}
```

// Function to return the front element of the queue

```
int peek(queue *q)
{
   // Check if the queue is empty
   if (isEmpty(q))
      return -1;
   // Return the data of the front node
   return q->front->data;
}
```

// Function to print the queue

```
void printQueue(queue *q)
{
    // Traverse the queue and print each element
    node *temp = q->front;
    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
int main()
{
    // Create a new queue
    queue *q = createQueue();

    // Enqueue elements into the queue
    enqueue(q, 10);
    enqueue(q, 20);
    enqueue(q, 30);
    enqueue(q, 40);
    enqueue(q, 50);

    // Print the queue
    printf("Queue: ");
    printQueue(q);

    // Dequeue elements from the queue
    dequeue(q);
    dequeue(q);

    // Print the queue after deletion of elements
    printf("Queue: ");
    printQueue(q);

    return 0;
}
```

```
Queue: 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Queue: 30 -> 40 -> 50 -> NULL
```

Benefits of Linked List Queue

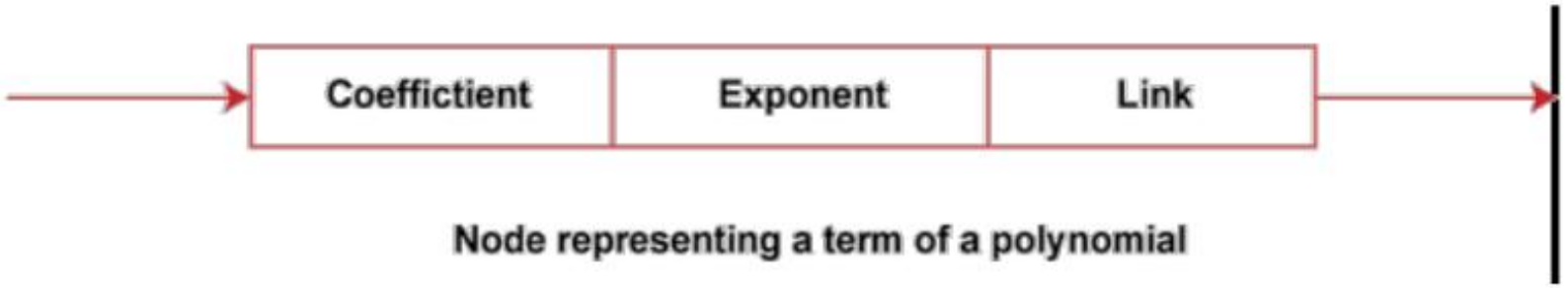
- The dynamic memory management of the linked list provides a dynamic size to the queue that changes with the number of elements.
- Rarely reaches the condition of queue overflow.

Applications of Linked Lists: Polynomials

- A polynomial is basically a collection of terms (coefficient, exponent).
- Example: $7x^4 + 13x^3 - 2x^2 + 9$
- Using a linked list helps store only non-zero terms, saving memory.
- It allows easy insertion of new terms in sorted order.
- Adding two polynomials becomes simple when both are stored in descending order of exponents.

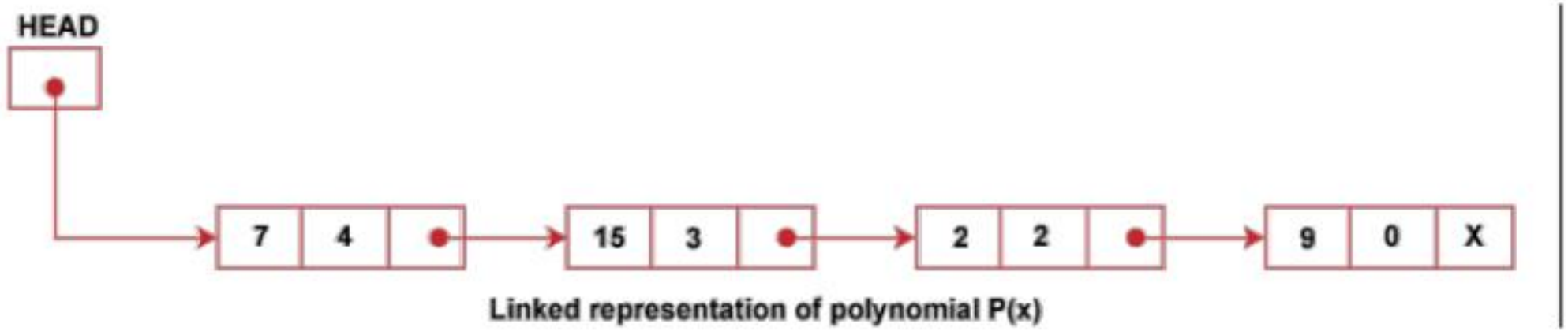
Representation of a Polynomial Node

- Each node contains:
- Coefficient (coef)
- Exponent (exp)
- Pointer to next node



Node representing a term of a polynomial

- Consider a polynomial P(x) = $7x^4 + 15x^3 - 2x^2 + 9$
- Here 7, 15, -2, and 9 are the coefficients, and 4,3,2,0 are the exponents of the terms in the polynomial.
- On representing this polynomial using a linked list, we have

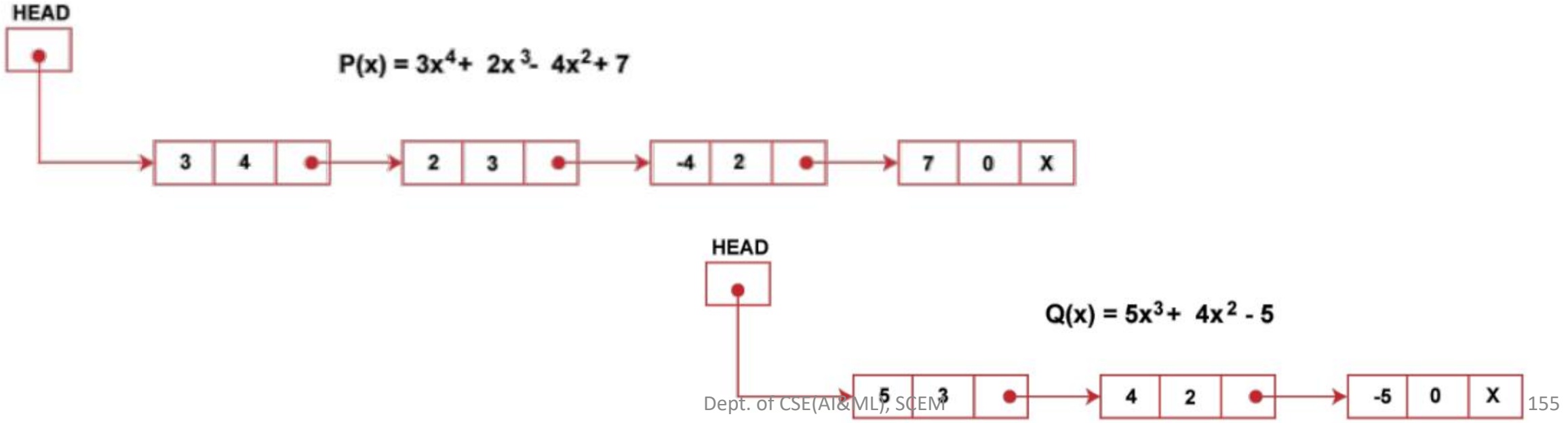


Linked representation of polynomial P(x)

```
struct Node {
    int coef;
    int exp;
    struct Node* next;
};
```

Addition of Polynomials

- Let us consider an example an example to show how the addition of two polynomials is performed
- $P(x) = 3x^4 + 2x^3 - 4x^2 + 7$
- $Q(x) = 5x^3 + 4x^2 - 5$
- These polynomials are represented using a linked list in order of decreasing exponents as follows:



Algorithm: Addition of Two Polynomials Using Linked List

- Input:
- Two polynomial linked lists P(x) and Q(x), each sorted in descending order of exponents.
- Output:
- A new linked list R(x) representing the sum polynomial.

Algorithm Steps

1. Create a new empty linked list

- Initialize newHead = NULL to store the resulting polynomial.

2.Initialize two pointers

- Let
- a → head of list P(x)
- b → head of list Q(x)

3.Traverse both lists until either pointer becomes NULL:

- For each step, compare the exponents of the terms pointed to by a and b.

3 cases:

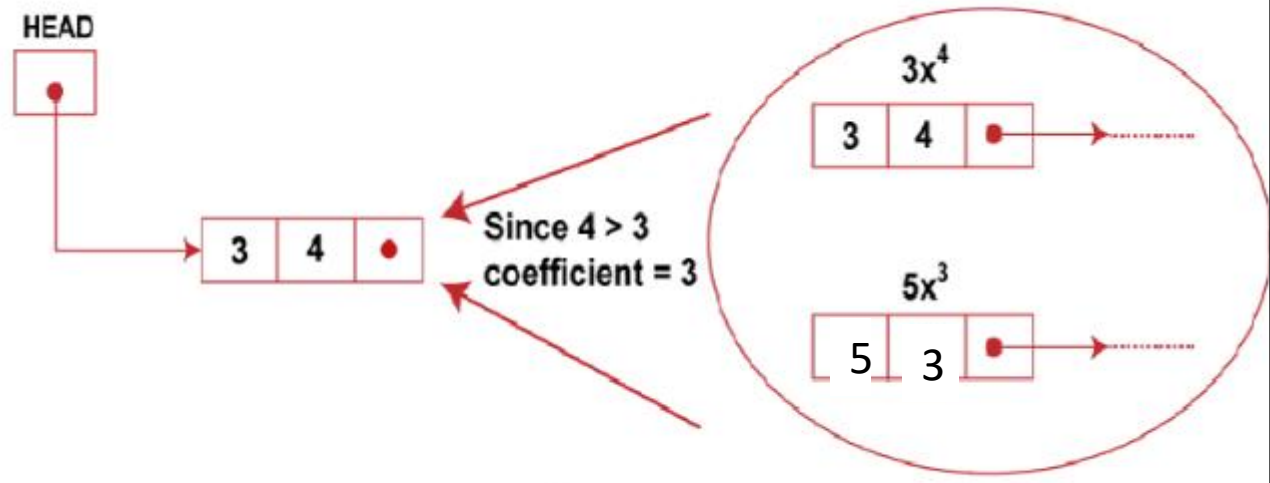
- Create a result list (initially empty).
- Traverse both polynomial lists simultaneously.
- Compare exponents of current nodes:
  - If exp1 > exp2 → add P1 term to result and move P1 ahead.
  - If exp1 < exp2 → add P2 term to result and move P2 ahead.
  - If exp1 == exp2 → add coefficients → insert combined term → move both ahead.
- If one list finishes earlier - append remaining terms from the other list.
- Final list = the result polynomial.

Case 1:exp1 > exp2

$P(x) = 3x^4 + 2x^3 - 4x^2 + 7$

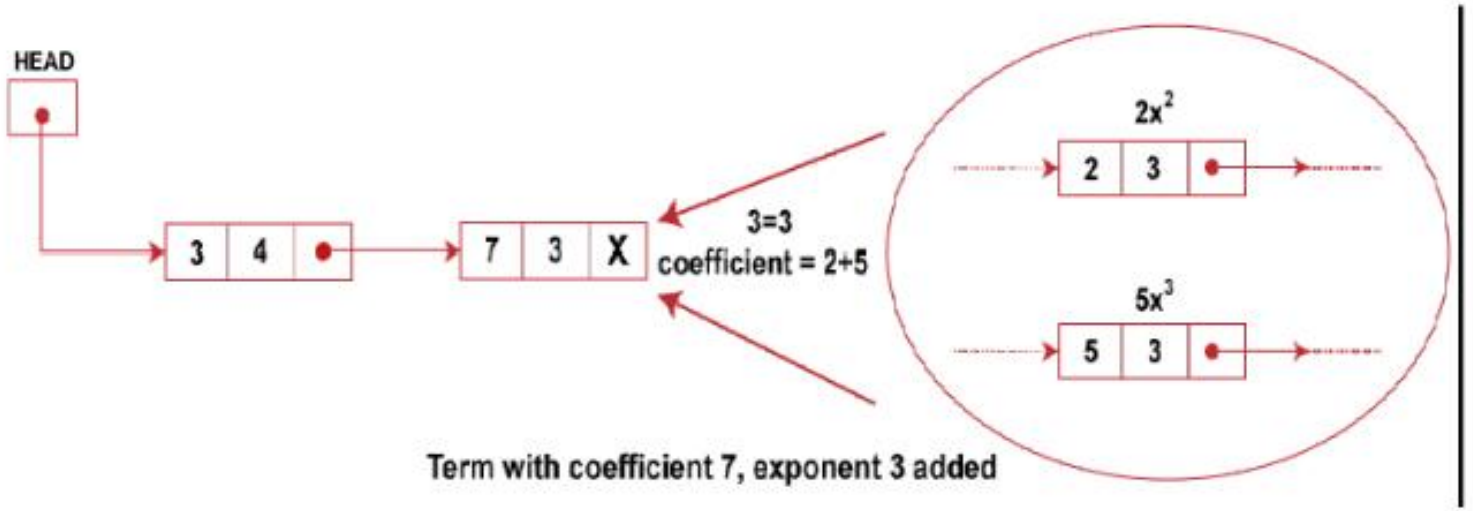
$Q(x) = 5x^3 + 4x^2 - 5$

1. Traverse the two lists P and Q and examine all the nodes.
2. We compare the exponents of the corresponding terms of two polynomials. The first term of polynomials P exponents 4 and 3, respectively. Since the exp term of the polynomial P is greater than the other term having a larger exponent is inserted into the list initially looks as shown below:

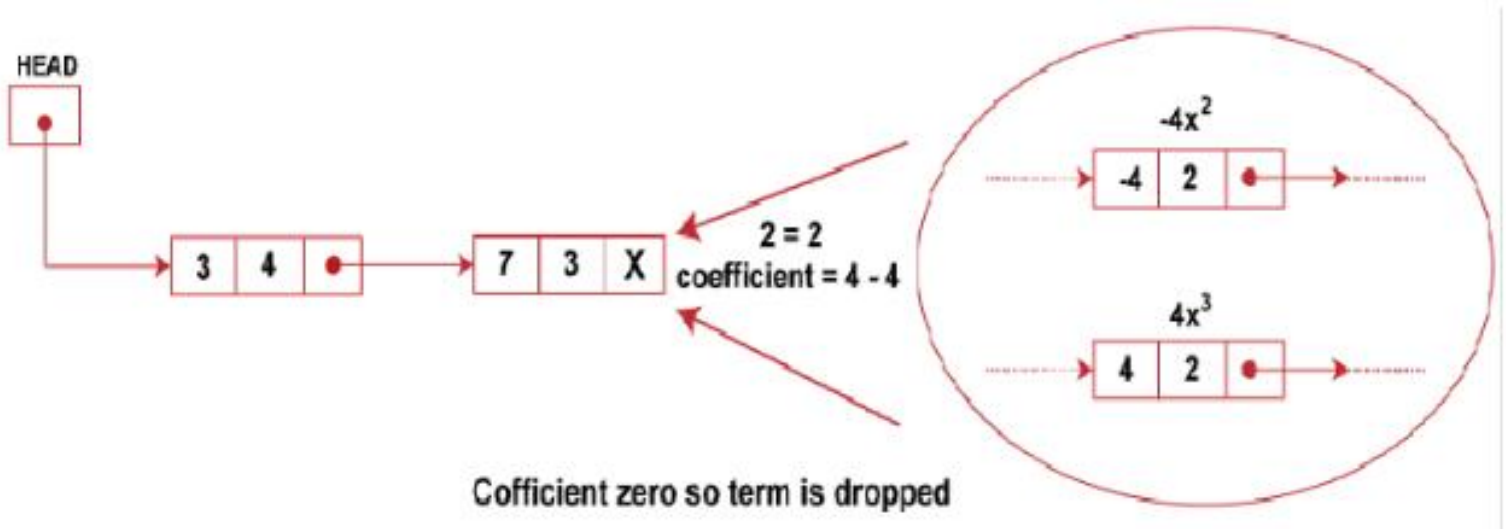


Term with coefficient 3, exponent 4 added

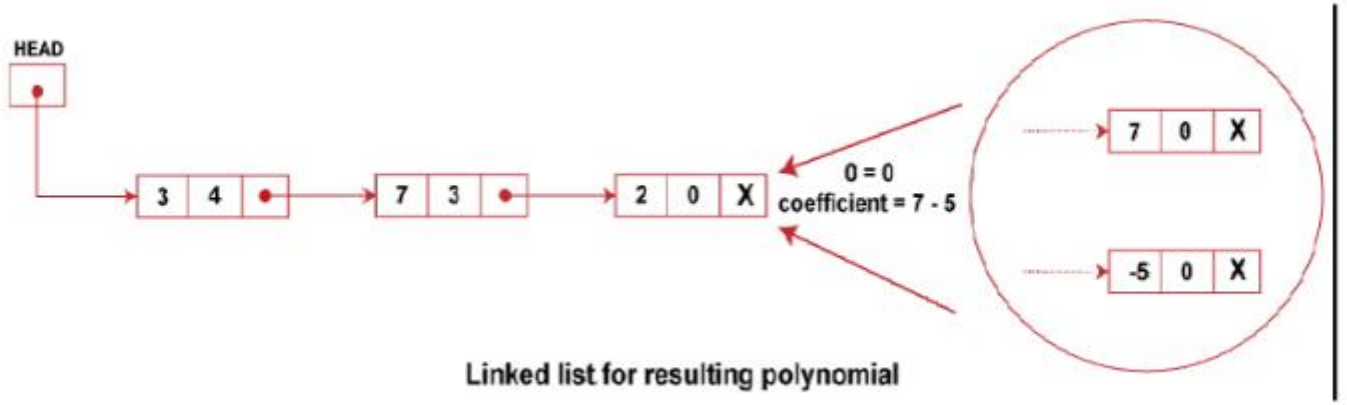
3. We then compare the exponent of the next term of the list P with the exponents of the present term of list Q. Since the two exponents are equal, so their coefficients are added and appended to the new list as follows:



4. Then we move to the next term of P and Q lists and compare their exponents. Since exponents of both these terms are equal and after addition of their coefficients, we get 0, so the term is dropped, and no node is appended to the new list after this,



5. Moving to the next term of the two lists, P and Q, we find that the corresponding terms have the same exponents equal to 0. We add their coefficients and append them to the new list for the resulting polynomial



Linked list for resulting polynomial

Example 2

$$P(x) = 7x^5 + 3x^4 + 2x^3 - 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

$$P(x) = 7x^5 + 3x^4 + 2x^3 - 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

Convert to linked lists (descending exponents):

- **P:** (7,5) → (3,4) → (2,3) → (-4,1) → (6,0)
- **Q:** (5,4) → (-2,3) → (9,2) → (1,0)

Step 1:Compare first terms

P exponent = 5

Q exponent = 4

Since 5 > 4, we apply:

Case 1: exp1 > exp2

Insert $7x^5$ into R(x).

Result list:

R = $7x^5$

Step 2: Move P ahead & compare next

- P exponent = 4
- Q exponent = 4
- Since 4 == 4, we apply:
- Case 3: exp1 == exp2
- Add coefficients:
- 3 + 5 = 8
- Insert $8x^4$.
- Result list:
- R = $7x^5 + 8x^4$

$$P(x) = 7x^5 + 3x^4 + 2x^3 - 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

- Step 3: Move both pointers
- P exponent = 3
- Q exponent = 3
- Equal again.
- Case 3: exp1 == exp2
- Add coefficients:
- 2 + (-2) = 0 → term cancels
- No node added.
- Result list remains:
- R = $7x^5 + 8x^4$

$$P(x) = 7x^5 + 3x^4 + 2x^3 - 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

- Step 4: Next comparison
- P exponent = 1
- Q exponent = 2
- Now:
- Case 2: exp1 < exp2
- Insert Q term $9x^2$.
- Result list:
- $R = 7x^5 + 8x^4 + 9x^2$
- Move Q pointer.

$$P(x) = 7x^5 + 3x^4 + 2x^3 + 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

- Step 5: Compare again
- P exponent = 1
- Q exponent = 0
- Since 1 > 0:
- Case 1: exp1 > exp2
- Insert P term −4x.
- Result list:
- R = $7x^5 + 8x^4 + 9x^2 - 4x$
- Move P pointer.

$$P(x) = 7x^5 + 3x^4 + 2x^3 - 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

- Step 6: Next
- P exponent = 0
- Q exponent = 0
- Equal exponents.
- Case 3: exp1 == exp2
- Add: 6 + 1 = 7
- Insert $7x^0 = 7$
- Result list:
- $R = 7x^5 + 8x^4 + 9x^2 - 4x + 7$

Step 7: Both lists finished

- No remaining terms - we are done.

$R(x) = 7x^5 + 8x^4 + 9x^2 - 4x + 7$

$$P(x) = 7x^5 + 3x^4 + 2x^3 - 4x + 6$$

$$Q(x) = 5x^4 - 2x^3 + 9x^2 + 1$$

Example 3:

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- Linked lists:
- P: (4,6) → (1,4) → (-3,2) → (2,0)
- Q: (5,5) → (2,4) → (1,3) → (-7,1) → (9,0)

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- Step 1
- P exp = 6
- Q exp = 5
- 6 > 5
- Case 1 → Insert $4x^6$
- R = $4x^6$

- Step 2
- P exp = 4
- Q exp = 5
- 4 < 5
- Case 2 → Insert $5x^5$
- R = $4x^6 + 5x^5$

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- Step 3
- P exp = 4
- Q exp = 4
- Equal exponents.
- Case 3 → exp1 == exp2
- 1 + 2 = 3 → insert $3x^4$
- R = $4x^6 + 5x^5 + 3x^4$

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- Step 4
- P exp = 2
- Q exp = 3
- 2 < 3
- Case 2 → Insert $1x^3$

- R = $4x^6 + 5x^5 + 3x^4 + x^3$

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- Step 5
- P exp = 2
- Q exp = 1
- 2 > 1
- Case 1 → Insert $-3x^2$

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- $R = 4x^6 + 5x^5 + 3x^4 + x^3 - 3x^2$

- Step 6
- P exp = 0
- Q exp = 1
- 0 < 1
- Case 2 → Insert −7x

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- $R = 4x^6 + 5x^5 + 3x^4 + x^3 - 3x^2 - 7x$

- Step 7
- P exp = 0
- Q exp = 0
- Equal.

Case 3 → Add

- 2 + 9 = 11 → Insert 11

$$P(x) = 4x^6 + x^4 - 3x^2 + 2$$

$$Q(x) = 5x^5 + 2x^4 + x^3 - 7x + 9$$

- $R = 4x^6 + 5x^5 + 3x^4 + x^3 - 3x^2 - 7x + 11$

Example 4:

$$P(x) = 6x^5 + 3x^3 - 2x$$

$$Q(x) = 4x^6 + 5x^5 + x^4 - 3x^2 + 8$$

Simple C Program to Add Two Polynomials Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Node structure: term = coefficient * x^exponent
struct Node {
    int coef;
    int exp;
    struct Node* next;
};
```

/ /Function to create a new node

```
struct Node* createNode(int c, int e) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->coef = c;
    newNode->exp = e;
    newNode->next = NULL;
    return newNode;
}
```

// Insert at end of list

```
void insertEnd(struct Node** head, int c, int e) {
    struct Node* temp = createNode(c, e);

    if (*head == NULL) {
        *head = temp;
        return;
    }

    struct Node* curr = *head;

    while (curr->next != NULL)
        curr = curr->next;

    curr->next = temp;
}
```

// Add two polynomials

```
struct Node* addPoly(struct Node* p1, struct Node* p2) {
    struct Node* result = NULL;

    while (p1 != NULL && p2 != NULL) {
        // Case 1: p1 exponent > p2 exponent
        if (p1->exp > p2->exp) {
            insertEnd(&result, p1->coef, p1->exp);
            p1 = p1->next;
        }
```

// Case 2: p1 exponent < p2 exponent

```
else if (p1->exp < p2->exp) {
    insertEnd(&result, p2->coef, p2->exp);
    p2 = p2->next;
}
```

// Case 3: exponents equal → add coefficients

```
    else {
        int sum = p1->coef + p2->coef;
        if (sum != 0)
            insertEnd(&result, sum, p1->exp);

        p1 = p1->next;
        p2 = p2->next;
    }
}
```

// Append remaining p1 terms

```
while (p1 != NULL) {
    insertEnd(&result, p1->coef, p1->exp);
    p1 = p1->next;
}
```

// Append remaining p2 terms

```
    while (p2 != NULL) {
        insertEnd(&result, p2->coef, p2->exp);
        p2 = p2->next;
    }

    return result;
}
```

// Print polynomial

```
void printPoly(struct Node* head) {
    while (head != NULL) {
        printf("%dx^%d ", head->coef, head->exp);
        if (head->next != NULL)
            printf("+ ");
        head = head->next;
    }
    printf("\n");
}
```

```
int main() {
    // Polynomial 1: 5x^3 + 4x^2 + 2
    struct Node* P = NULL;
    insertEnd(&P, 5, 3);
    insertEnd(&P, 4, 2);
    insertEnd(&P, 2, 0);
```

```
/ /Polynomial 2: 3x^3 + x + 7
    struct Node* Q = NULL;
    insertEnd(&Q, 3, 3);
    insertEnd(&Q, 1, 1);
    insertEnd(&Q, 7, 0);
```

Output

P(x) = 5x^3 + 4x^2 + 2x^0

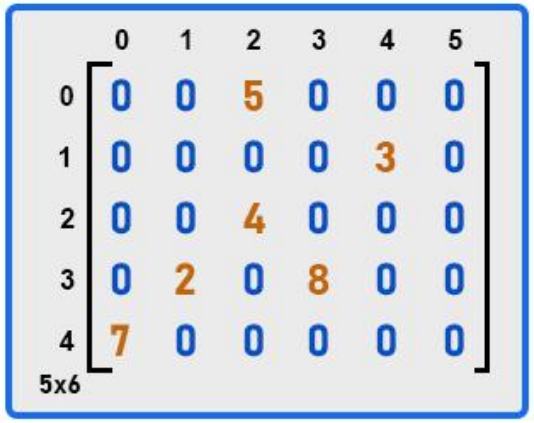
Q(x) = 3x^3 + 1x^1 + 7x^0

R(x) = P(x) + Q(x) = 8x^3 + 4x^2 + 1x^1 + 9x^0

```
 printf("P(x) = ");
    printPoly(P);

    printf("Q(x) = ");
    printPoly(Q);

    struct Node* R = addPoly(P, Q);

    printf("R(x) = P(x) + Q(x) = ");
    printPoly(R);

    return 0;
}
```

Application of linked list: Sparse matrix representation.

- Sparse matrices are those matrices where most elements are ZERO, and only a few are non-zero.



	0	1	2	3	4	5
0	0	0	5	0	0	0
1	0	0	0	0	3	0
2	0	0	4	0	0	0
3	0	2	0	8	0	0
4	7	0	0	0	0	0

5x6

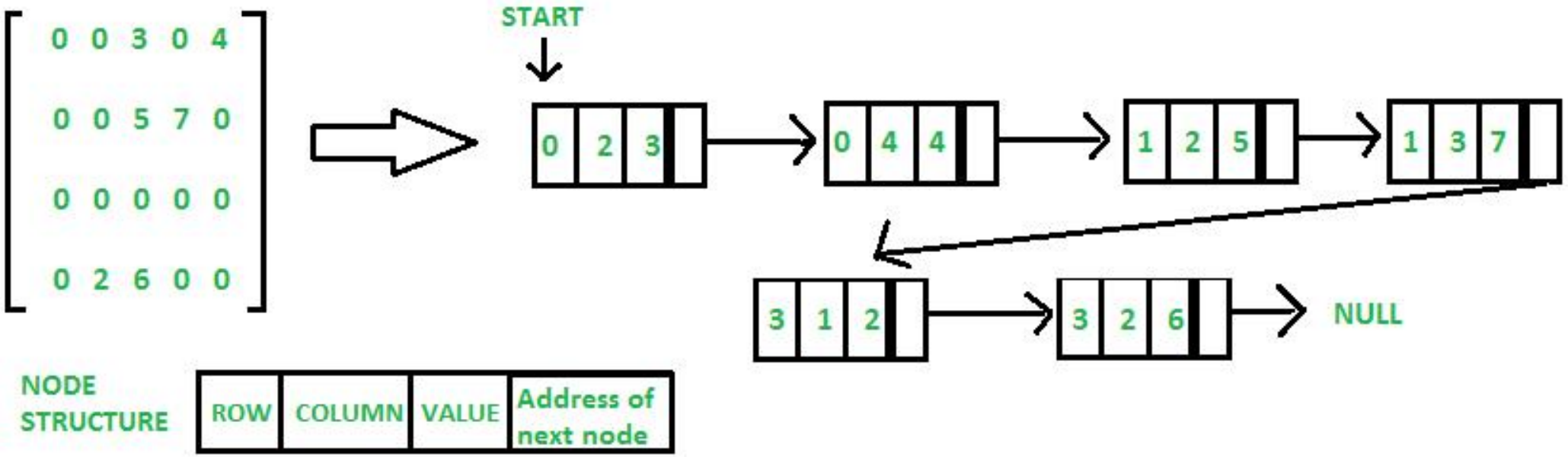
- Storing it in a normal 2D array is a waste of memory.
- So, we use linked lists to store ONLY non-zero elements.

WHY LINKED LISTS FOR SPARSE MATRICES?

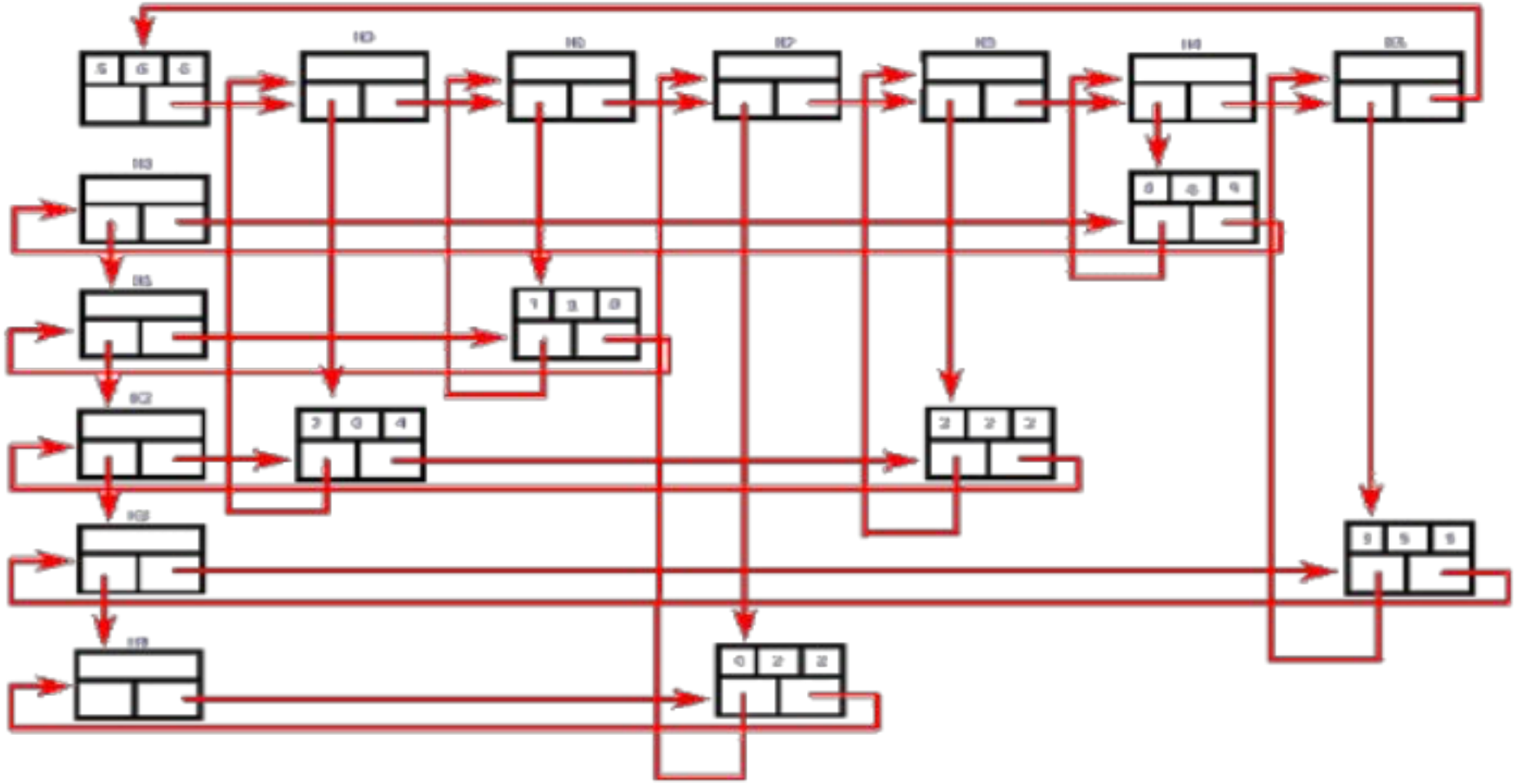
- Memory Efficient
- Dynamic in Size
- Easy to Traverse Row-wise or Column-wise
- Fast Insertion of New Non-zero Elements
- Linked representation supports multi-linked structures

Sparse Matrix Linked List Representation

- Each node stores:
  - row index
  - column index
  - non-zero value
  - pointer to next node

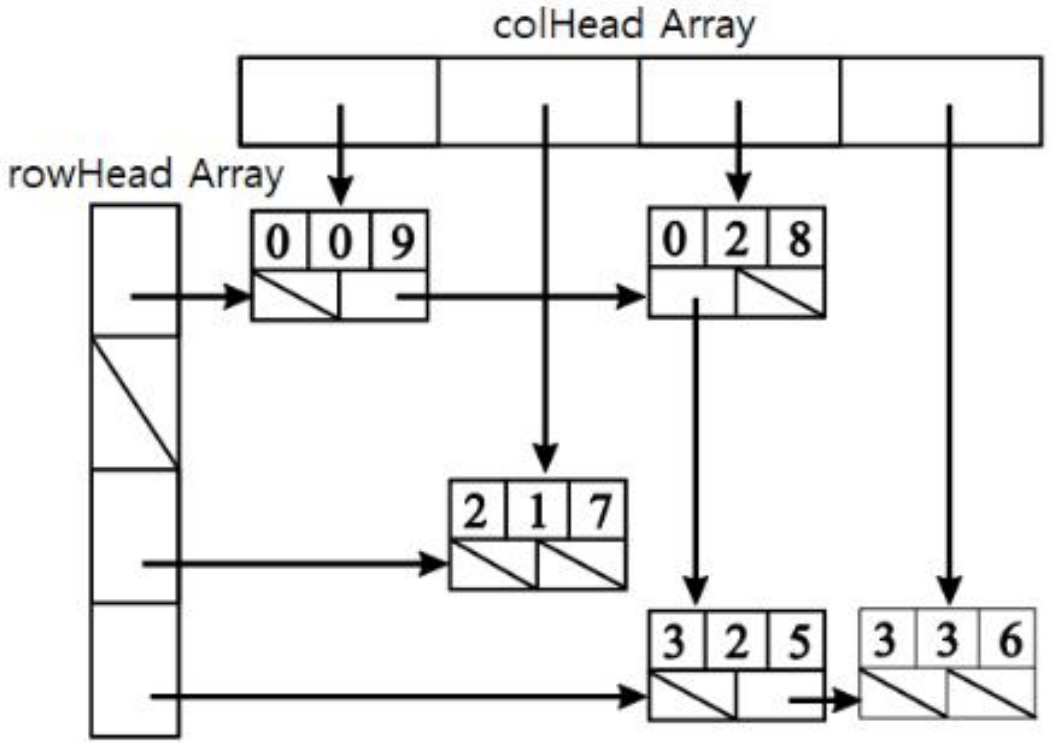

0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0
START
0 2 3
0 4 4
1 2 5
1 3 7
3 1 2
3 2 6
NULL
NODE STRUCTURE
ROW
COLUMN
VALUE
Address of next node

Orthogonal Linked List (OLL)

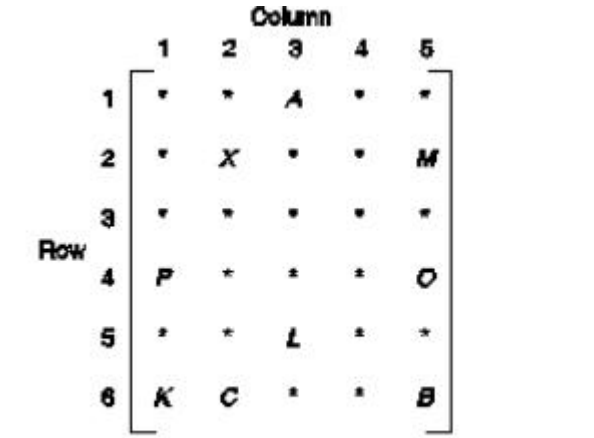


Example 1

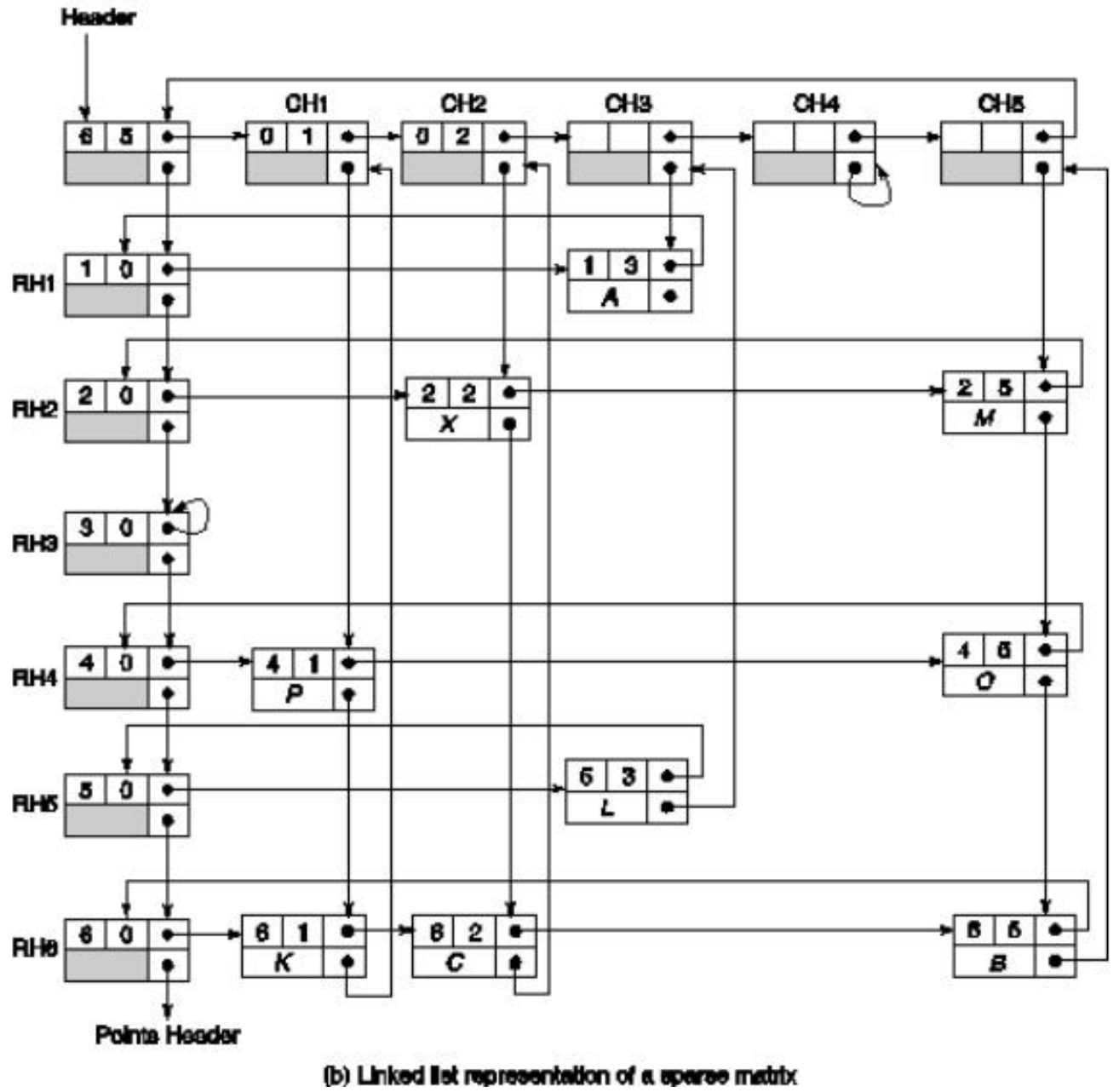
$$\begin{pmatrix} 9 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 7 & 0 & 0 \\ 0 & 0 & 5 & 6 \end{pmatrix}$$



Example 2



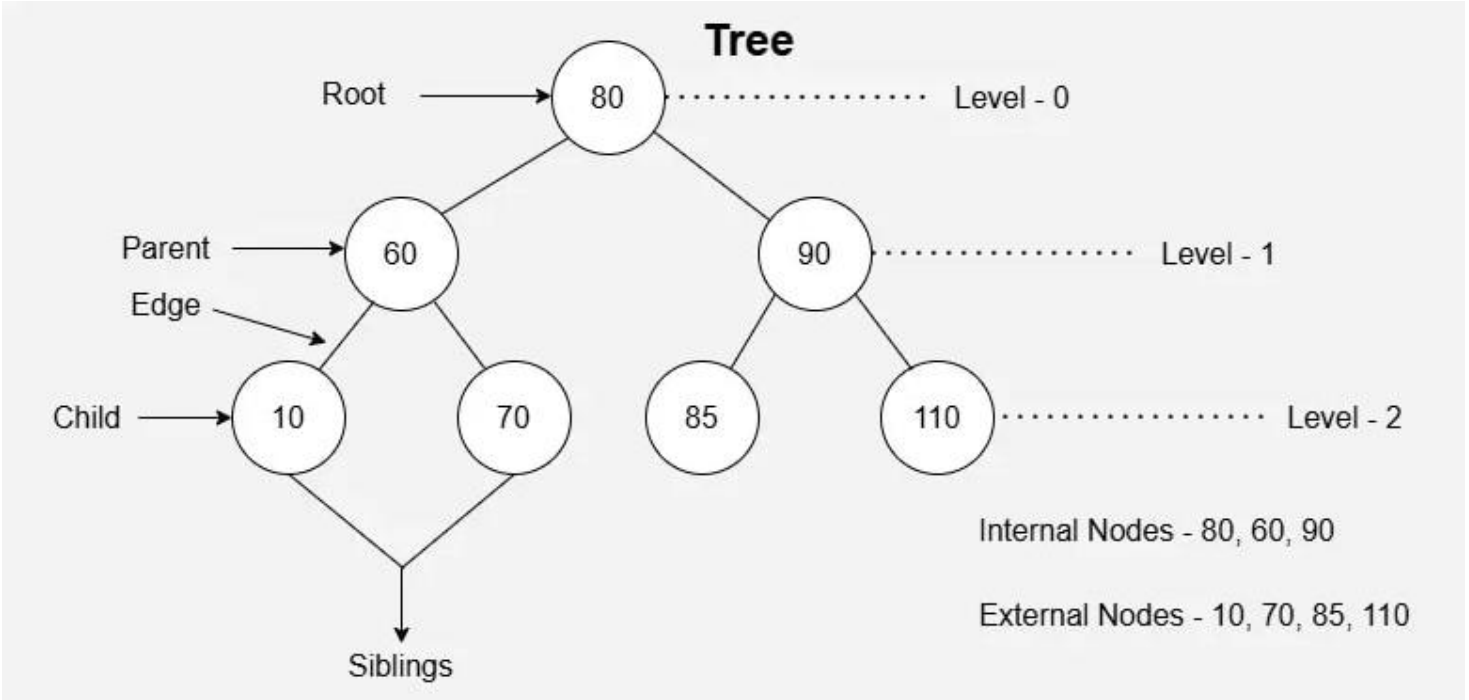
(a) A sparse matrix of order 6 × 5 containing 9 elements



(b) Linked list representation of a sparse matrix

Trees

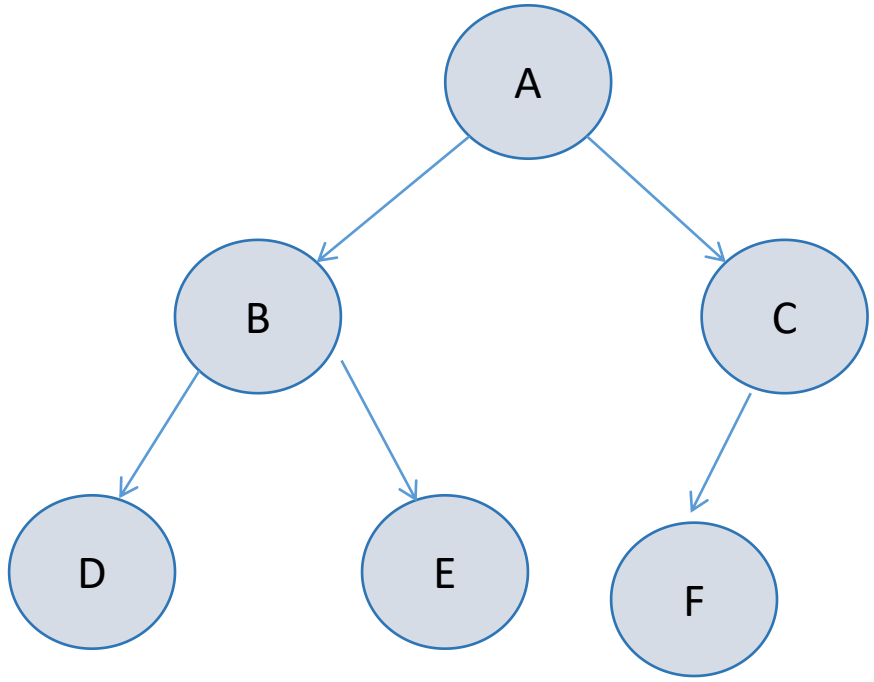
- What is a Tree?
- A hierarchical data structure made of nodes connected by edges.
- Non-linear and is perfect for representing parent–child relationships.



Terminologies

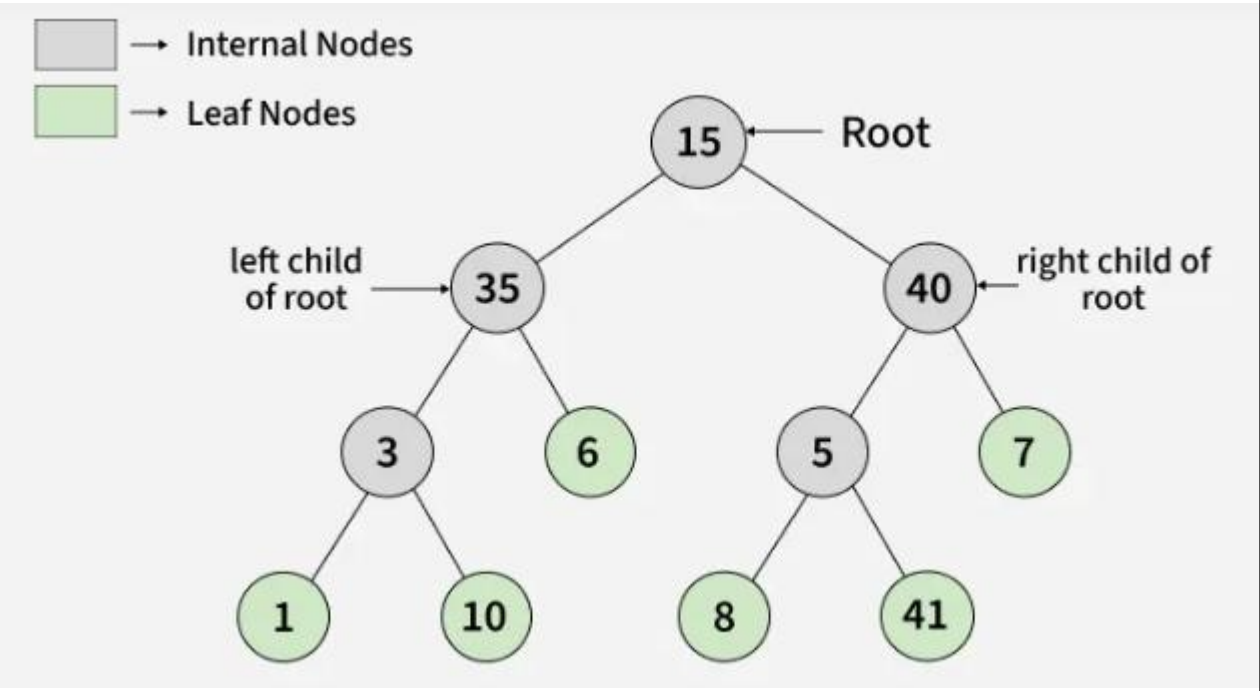
- Node: Fundamental unit carrying data.
- Root: Topmost node; origin of the tree.
- Parent: Node directly above another node.
- Child: Node directly below a parent.
- Siblings: Nodes sharing the same parent.
- Leaf Node: Node with no children → endpoints of the structure.
- Internal Node: Any non-leaf node.
- Edge: Link between nodes.
- Path: Sequence of nodes connected by edges.
- Depth of Node: Distance from root to that node.
- Height of Node: Longest path from that node down to a leaf.
- Height of Tree: Height of the root.
- Subtree: Tree formed by any node and its descendants.

- Node:
- Root:
- Parent:
- Child:
- Siblings:
- Leaf Node:
- Internal Node:
- Edge:
- Path:
- Depth of Node:
- Height of Node:
- Height of Tree:
- Subtree:



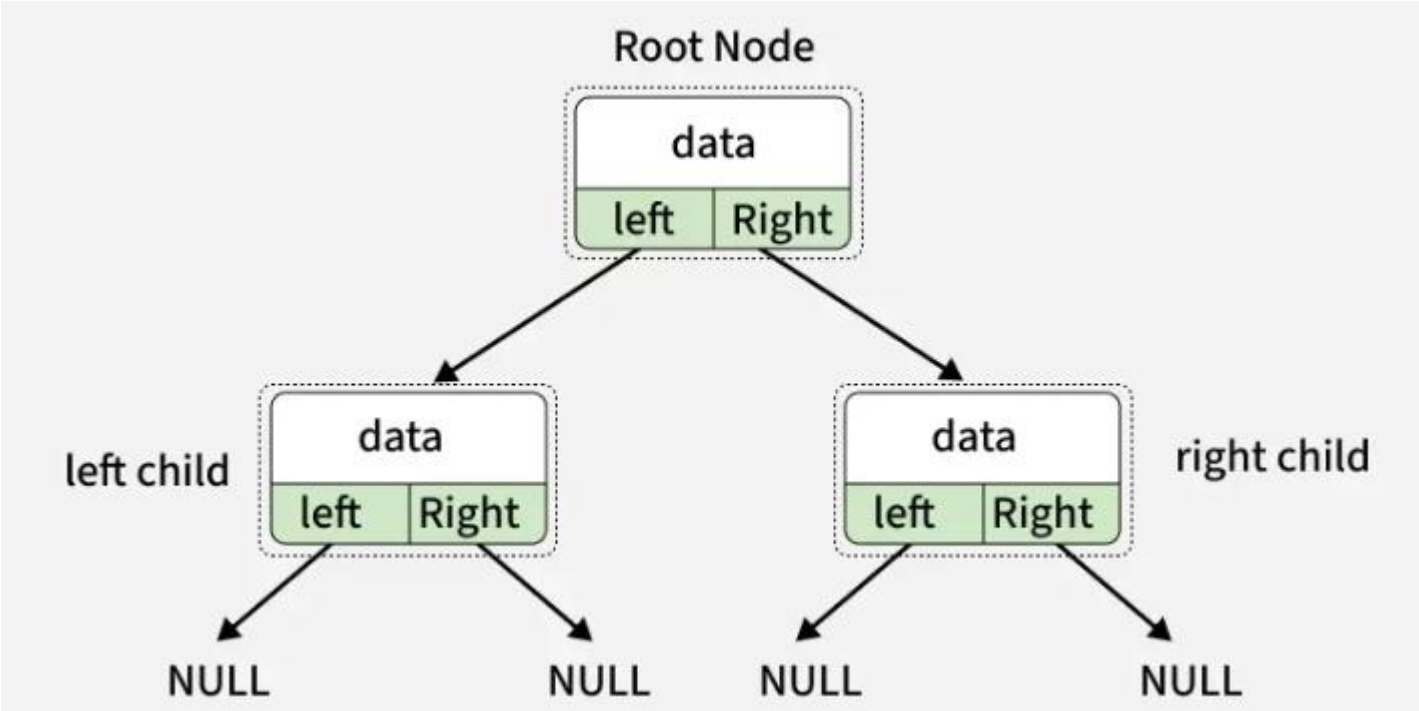
Binary Trees

- Binary Tree is a non-linear and hierarchical data structure where each node has at most two children referred to as the left child and the right child.
- The topmost node in a binary tree is called the root, and the bottom-most nodes(having no children) are called leaves.



Representation of Binary Tree

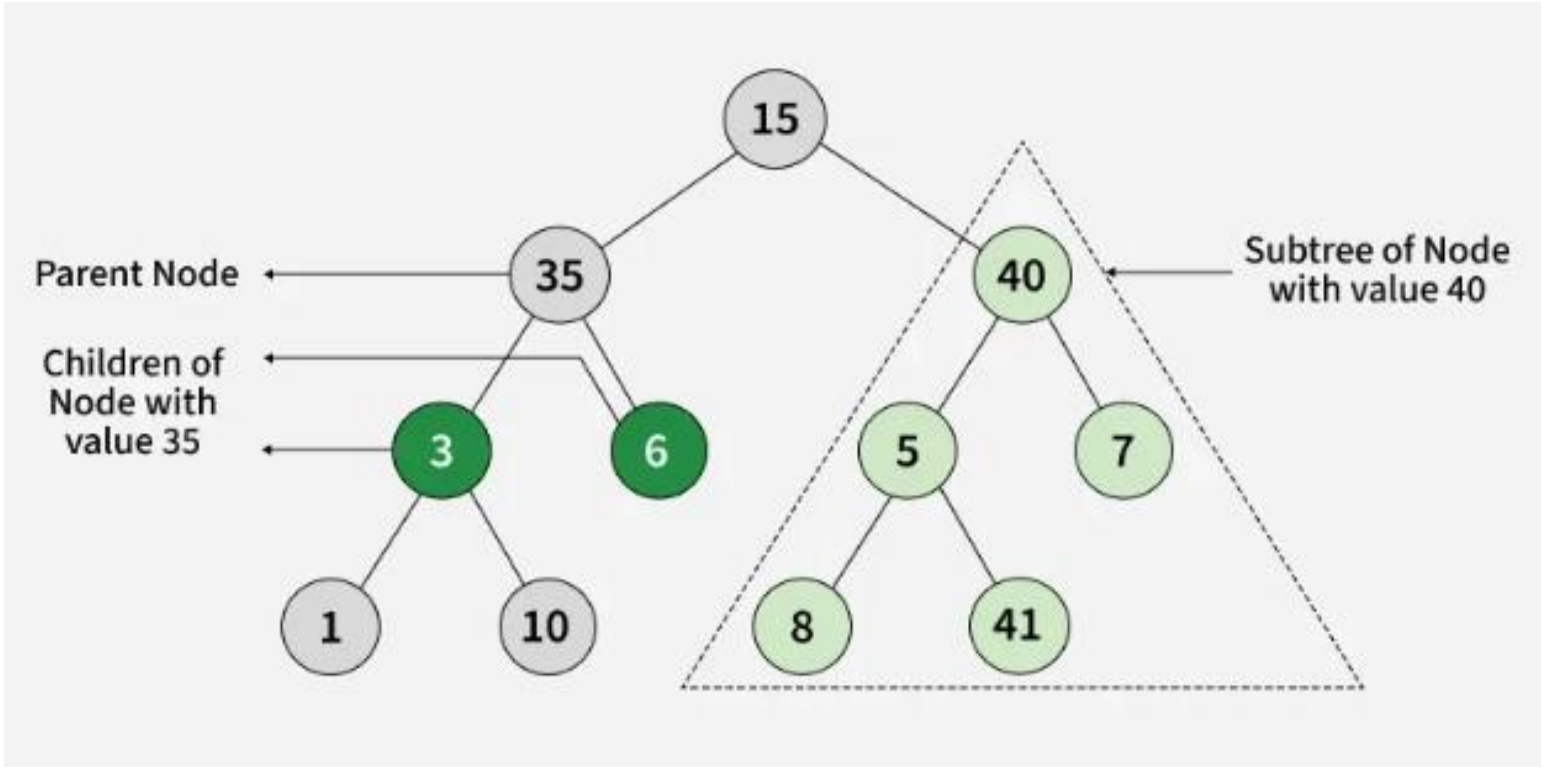
- Each node in a Binary Tree has three parts:
  - Data
  - Pointer to the left child
  - Pointer to the right child

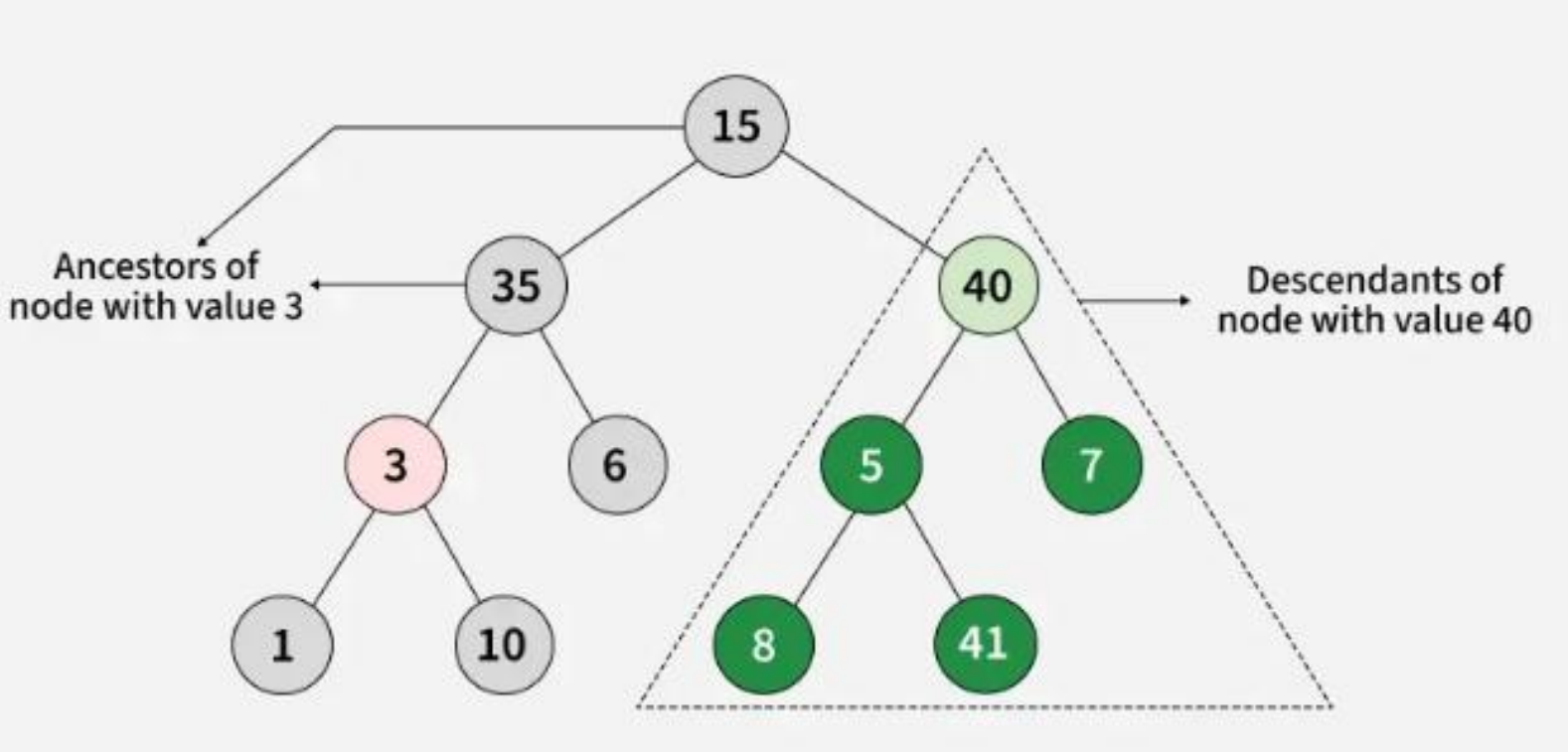


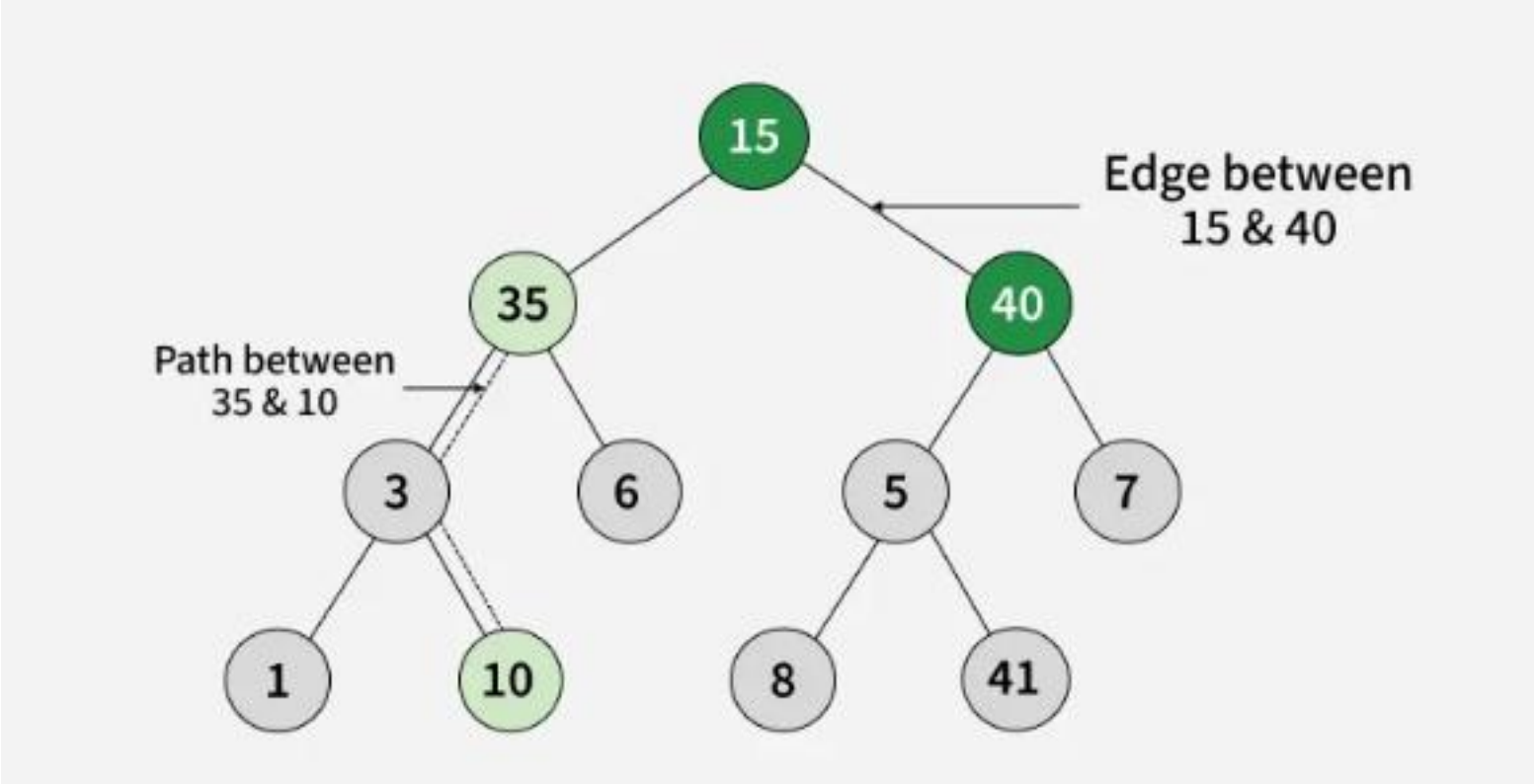
Terminologies in Binary Tree

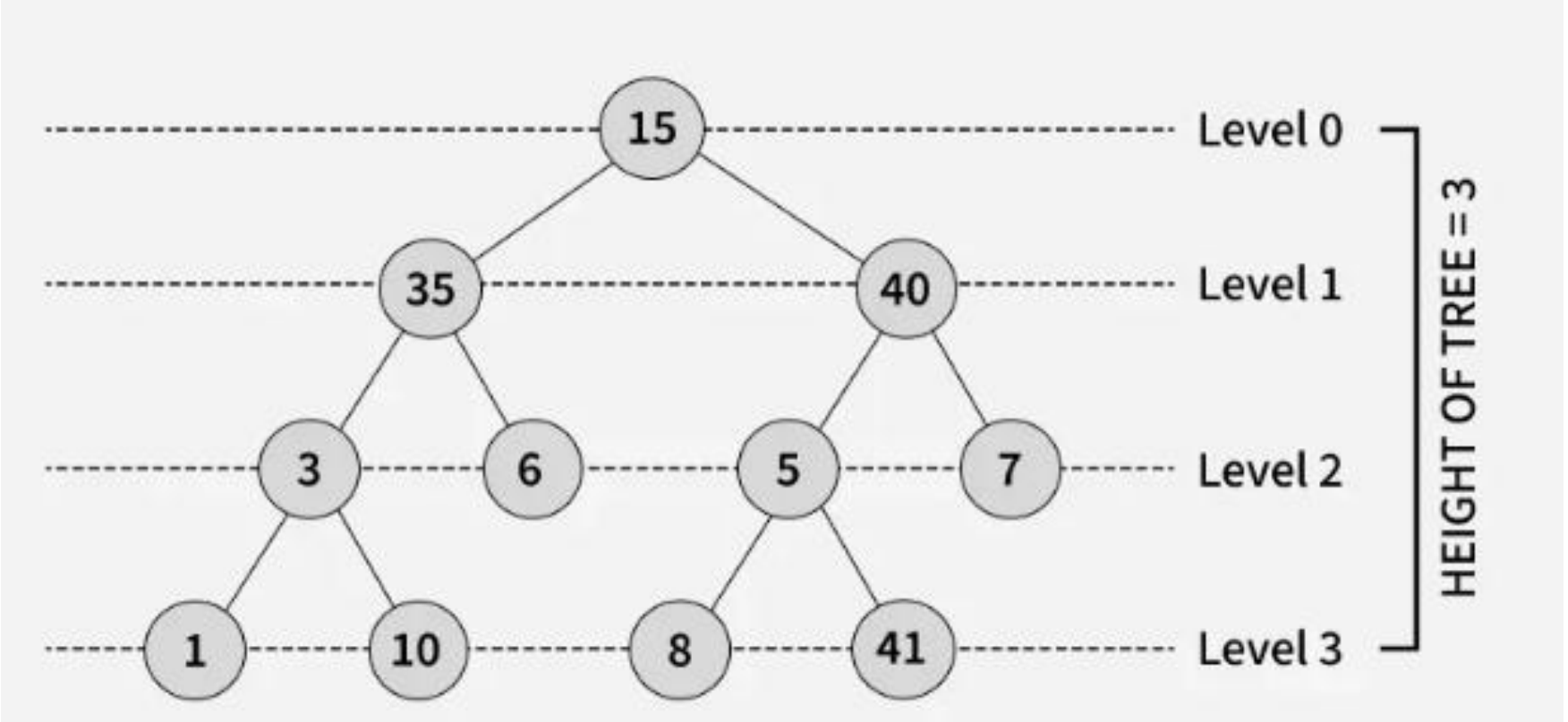
- Parent Node: A node that is the direct ancestor of a node(its child node).
- Child Node: A node that is the direct descendant of another node (its parent).
- Ancestors of a node: All nodes on the path from the root to that node (including the node itself).
- Descendants of a node: All nodes that lie in the subtree rooted at that node (including the node itself).
- Subtree of a node: A tree consisting of that node as root and all its descendants.
- Edge: The link/connection between a parent node and its child node.

- Path in a binary tree: A sequence of nodes connected by edges from one node to another.
- Leaf Node: A node that does not have any children or both children are null.
- Internal Node: A node that has at least one child.
- Depth/Level of a Node: The number of edges in the path from root to that node. The depth/level of the root node is zero.
- Height of a Binary Tree: The number of edges on the longest path from root to a leaf.


15
Parent Node
35
40
Subtree of Node with value 40
Children of Node with value 35
3
6
5
7
1
10
8
41


15
Ancestors of
node with value 3
35
40
Descendants of
node with value 40
3
6
5
7
1
10
8
41


15
Edge between
15 & 40
35
40
Path between
35 & 10
3
6
5
7
1
10
8
41


15
35
40
3
6
5
7
1
10
8
41
Level 0
Level 1
Level 2
Level 3
HEIGHT OF TREE = 3

Properties of Binary Tree

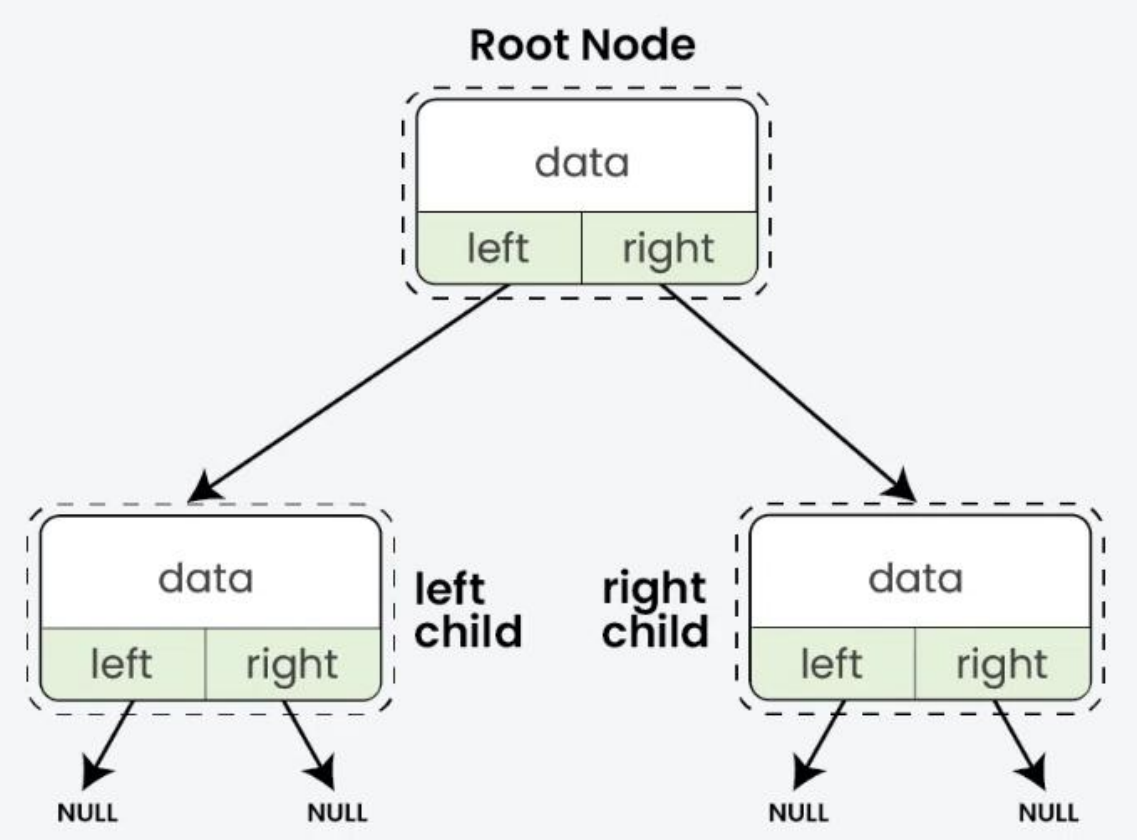
- The maximum number of nodes at level **L** of a binary tree is 2^L.
- The maximum number of nodes in a binary tree of height **H** is $2^{H+1} - 1$.
- Total number of leaf nodes in a binary tree = total number of nodes with 2 children + 1.
- In a Binary Tree with **N** nodes, the minimum possible height or the minimum number of levels is $\lceil \log_2 N \rceil$.
- A Binary Tree with **L** leaves has at least $\lceil \log_2 L \rceil + 1$ levels.

Representation of Binary Trees

- There are two primary ways to represent binary trees
  - Linked Node Representation
  - Array Representation

1. Linked Node Representation

- Each node contains data and pointers to its left and right children.

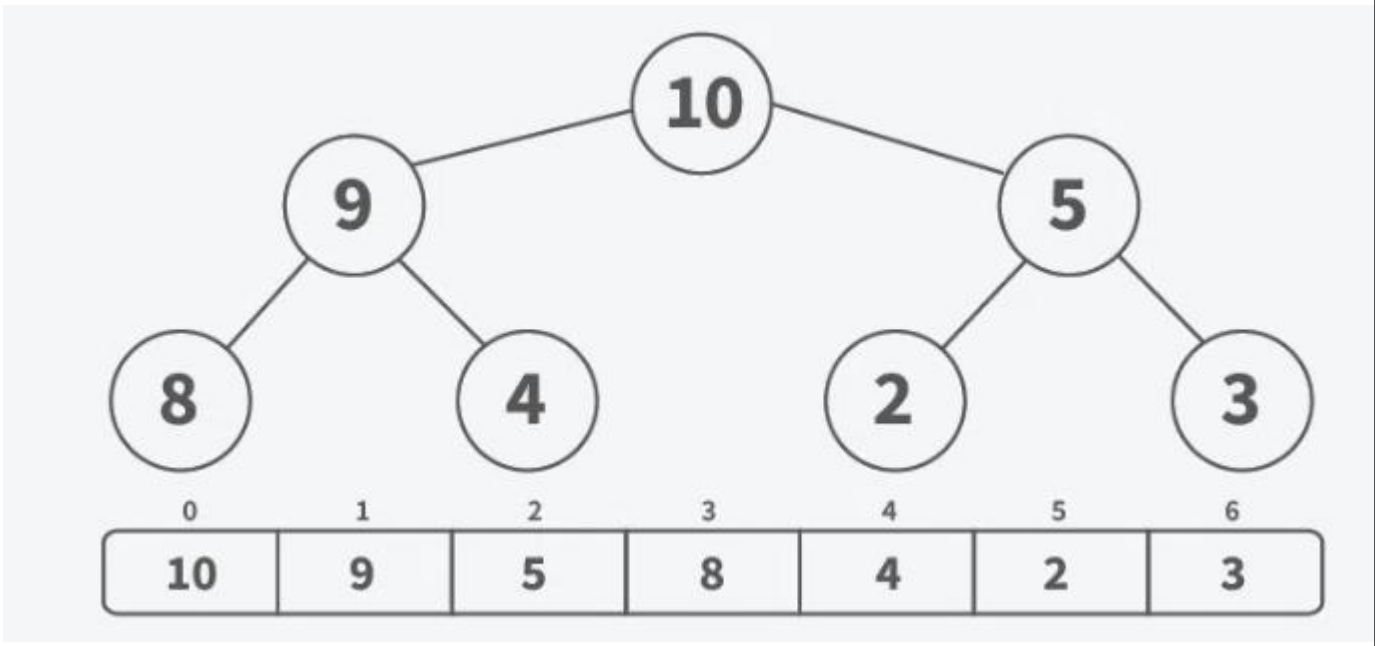


- Advantages:
- It can easily grow or shrink as needed, so it uses only the memory it needs.
- Adding or removing nodes is straightforward and requires only pointer adjustments.
- Only uses memory for the nodes that exist, making it efficient for sparse trees.

- Disadvantages:
- Needs extra memory for pointers.
- Finding a node can take longer because you have to start from the root and follow pointers.

2. Array Representation

- Array Representation is another way to represent binary trees, especially useful when the tree is complete.
- The tree is stored in an array.
- For any node at index i:
- Left Child: Located at 2 * i + 1
- Right Child: Located at 2 * i + 2
- Root Node: Stored at index 0



- Advantages:
- Easy to navigate parent and child nodes using index calculations, which is fast
- Easier to implement, especially for complete binary trees.

- Disadvantages:
- You have to set a size in advance, which can lead to wasted space.
- If the tree is not complete binary tree then then many slots in the array might be empty, this will result in wasting memory
- Not as flexible as linked representations for dynamic trees.

C Program for Binary Tree Representation Using Arrays

```
#include <stdio.h>
#define MAX 20
int tree[MAX];
```

```
// Insert node into array-based tree
void insert(int value, int index) {
    if (index >= MAX) {
        printf("Index out of range!\n");
        return;
    }
    tree[index] = value;
}
```

```
// Display tree
void display() {
    printf("Array Representation:\n");
    for (int i = 0; i < MAX; i++) {
        if (tree[i] != 0)
            printf("Index %d : %d\n", i, tree[i]);
    }
}
```

```
int main() {
 insert(10, 0);       // root
    insert(20, 1);       // left child of root
    insert(30, 2);       // right child
    insert(40, 3);
    insert(50, 4);

    display();
    return 0;
}
```

Binary Tree Representation Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};
```

```
// Create a new node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

```
void preorder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}
```

```
int main() {
struct Node* root = createNode(10);
    root->left = createNode(20);
    root->right = createNode(30);

    root->left->left = createNode(40);
    root->left->right = createNode(50);

    printf("Preorder Traversal: ");
    preorder(root);

    return 0;
}
```

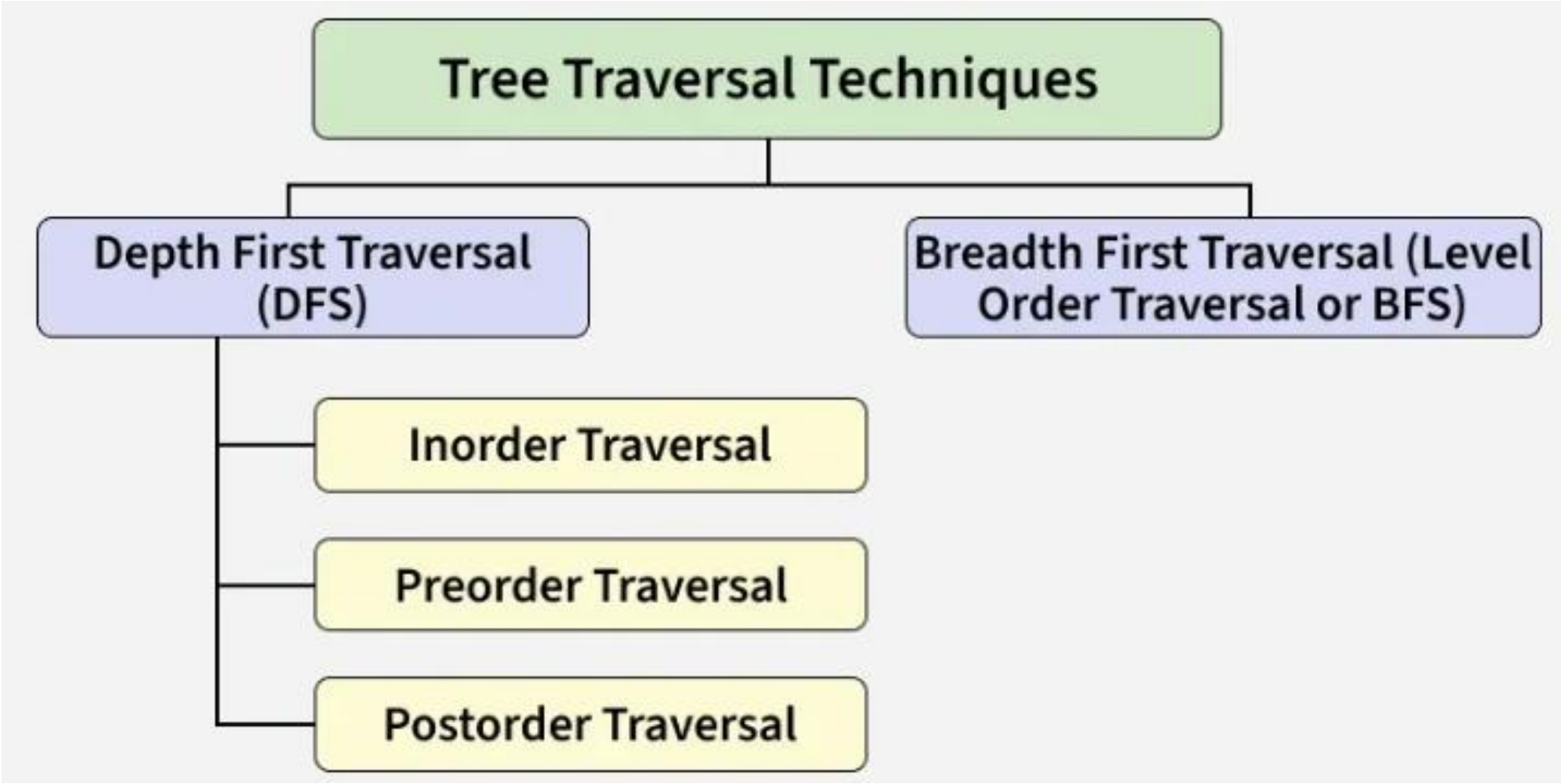
Comparison of Linked list vs Array Representation of Binary Tree

Feature	Linked Node Representation	Array Representation
Memory Usage	Uses extra memory for pointers	Efficient for complete trees
Ease of Implementation	Simple and intuitive	Simple but limited to complete trees
Dynamic Size	Easily grows and shrinks	Not flexible for dynamic sizes
Access Time	Slower access for specific nodes	Fast access using indices
Suitable For	Any binary tree structure	Complete binary tree

Binary Tree Traversals

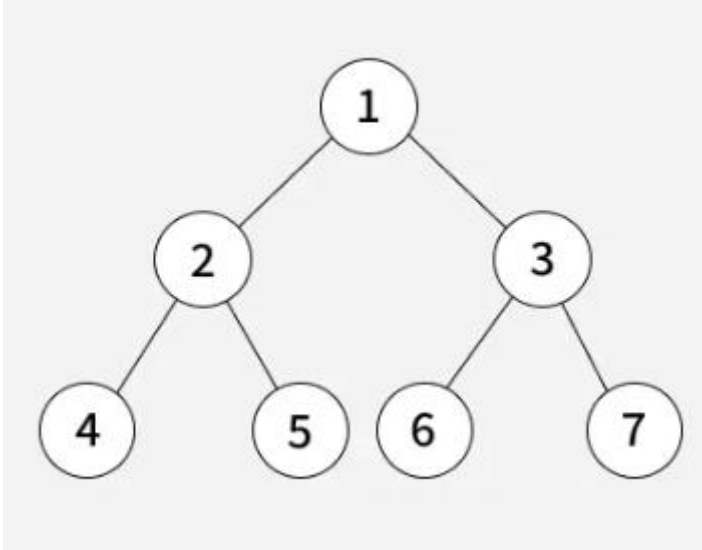
What is a Binary Tree Traversal?

- Tree traversal refers to the process of visiting or accessing each node of a tree exactly once in a specific order.
- Order matters: different traversals give different outputs


Tree Traversal Techniques
Depth First Traversal (DFS)
Breadth First Traversal (Level Order Traversal or BFS)
Inorder Traversal
Preorder Traversal
Postorder Traversal

Breadth-First Traversal (BFT)

- Explores nodes level by level from top to bottom.
- Type: Level Order Traversal.
- The level order traversal of the above tree is 1, 2, 3, 4, 5, 6, and 7.

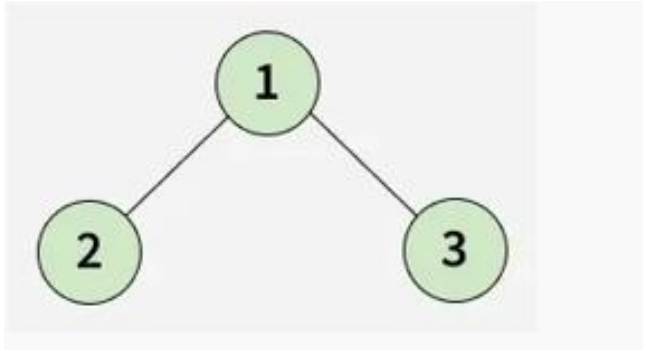


Depth-First Traversal (DFT)

- Explores as far as possible along a branch before exploring the next branch.
- Types:
- Inorder(Left -> Root -> Right)
- Preorder(Root -> Left -> Right)
- Postorder(Left -> Right -> Root)

Inorder Traversal

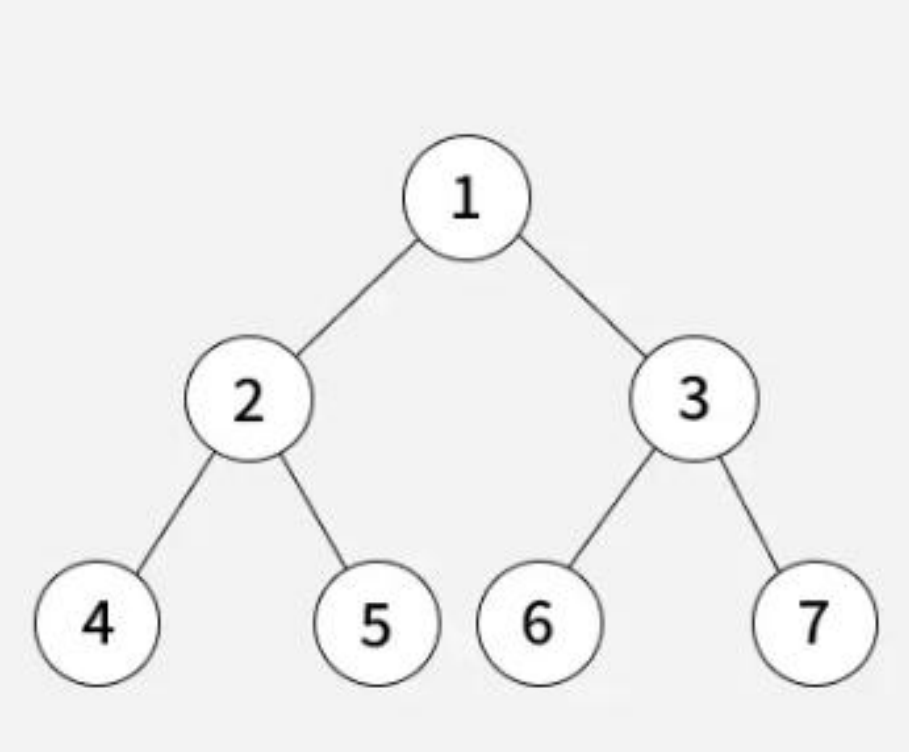
- Inorder traversal visits the node in the order: Left -> Root -> Right
- Algorithm for Inorder Traversal
- Inorder(tree)
  - Traverse the left subtree, i.e., call Inorder(left->subtree)
  - Visit the root.
  - Traverse the right subtree, i.e., call Inorder(right->subtree)



Inorder:
Left → Root → Right
2, 1, 3

Postorder
Left → Right → Root
2, 3, 1

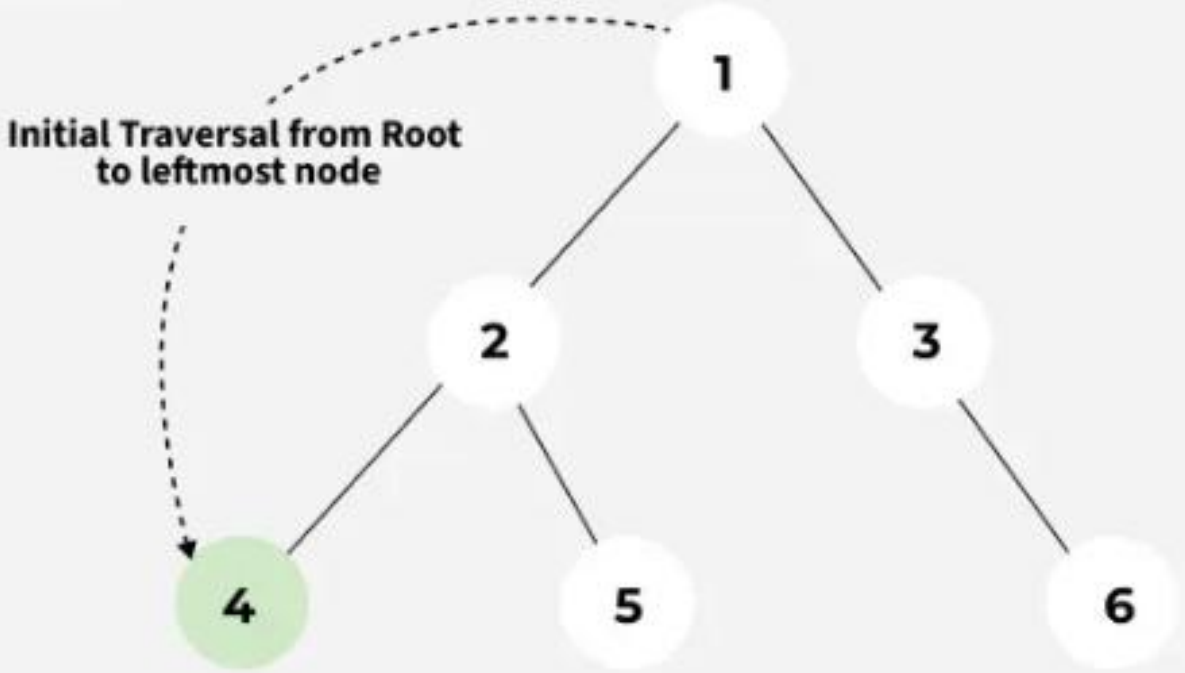
Preorder
Root → Left → Right
1, 2, 3


1
2
3
4
5
6
7


Inorder Traversal
4
2
5
1
6
3
7

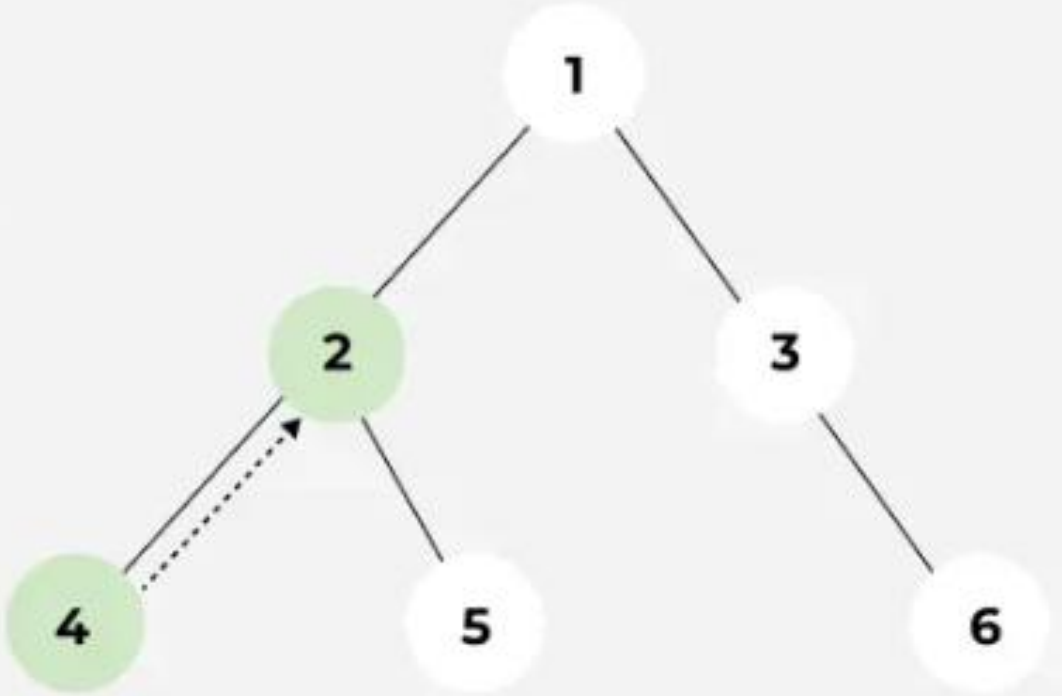
01 Step

The traversal will go from 1 to its left subtree i.e., 2, then from 2 to its left subtree root, i.e., 4. Now 4 has no left subtree, so it will be visited. It also does not have any right subtree. So no more traversal from 4



02 Step | As the left subtree of 2 is visited completely, now it visits node 2 before moving to its right subtree.

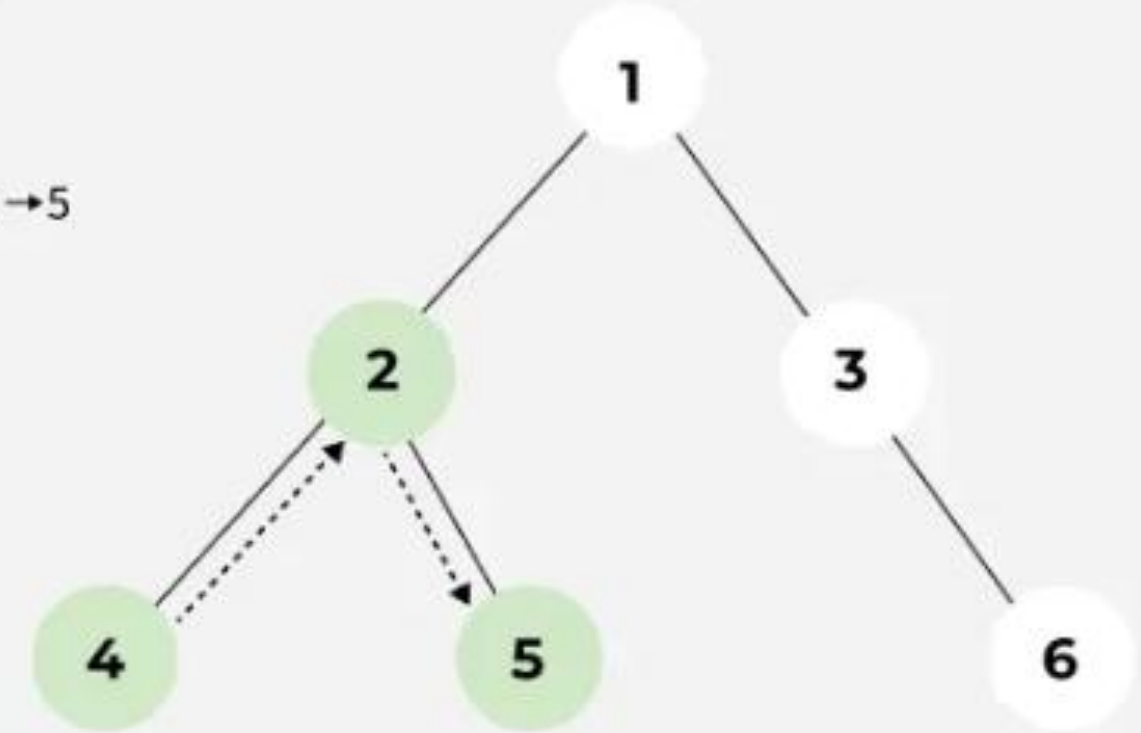
Traversal Order: 4→2



03 Step

Now the right subtree of 2 will be traversed i.e., move to node 5. For node 5 there is no left subtree, so it gets visited and after that, the traversal comes back because there is no right subtree of node 5.

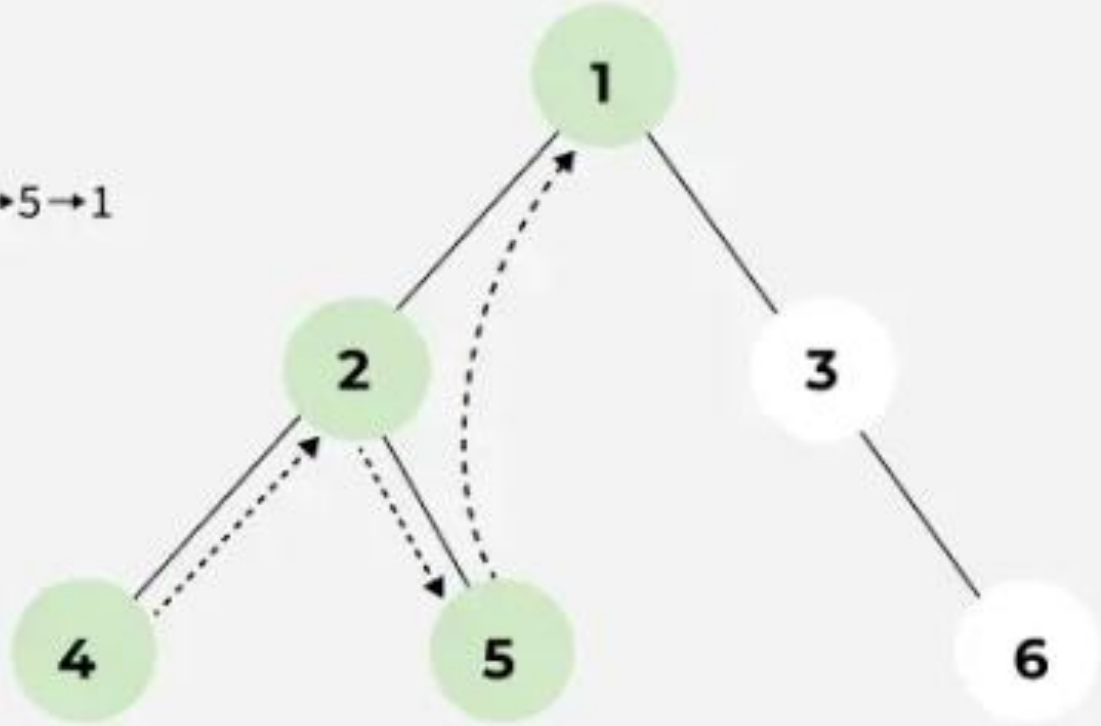
Traversal Order: 4 → 2 → 5



04 Step

As the left subtree of node 1 is completely visited the root itself, i.e., node 1 will be visited.

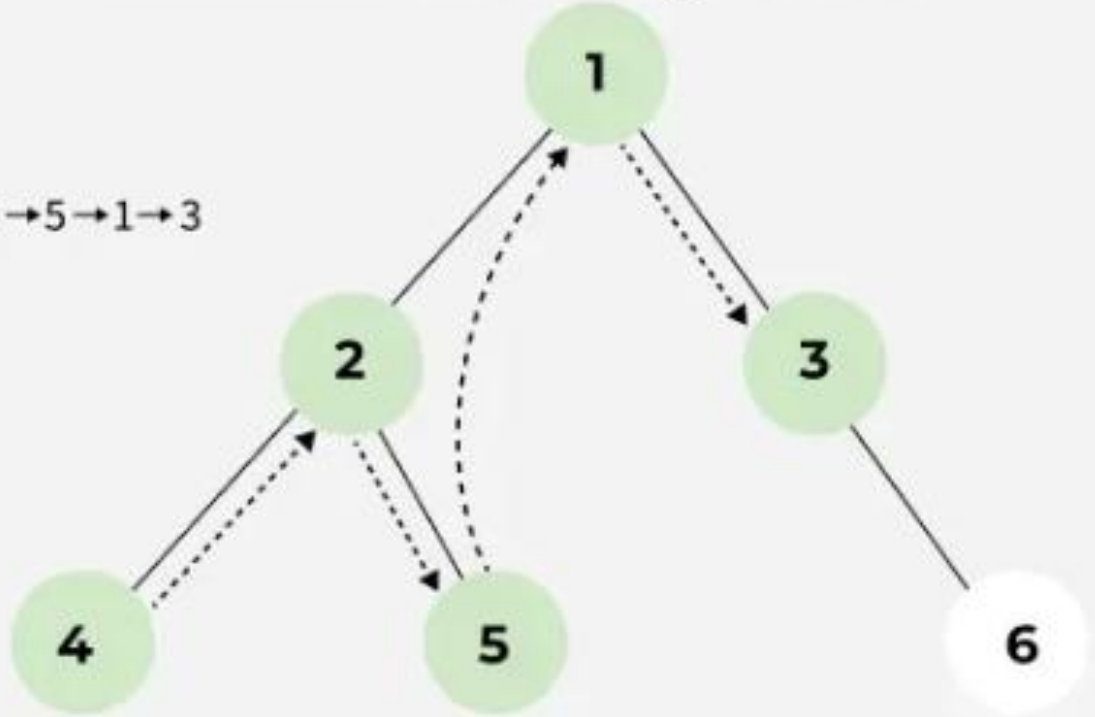
Traversal Order: 4→2→5→1



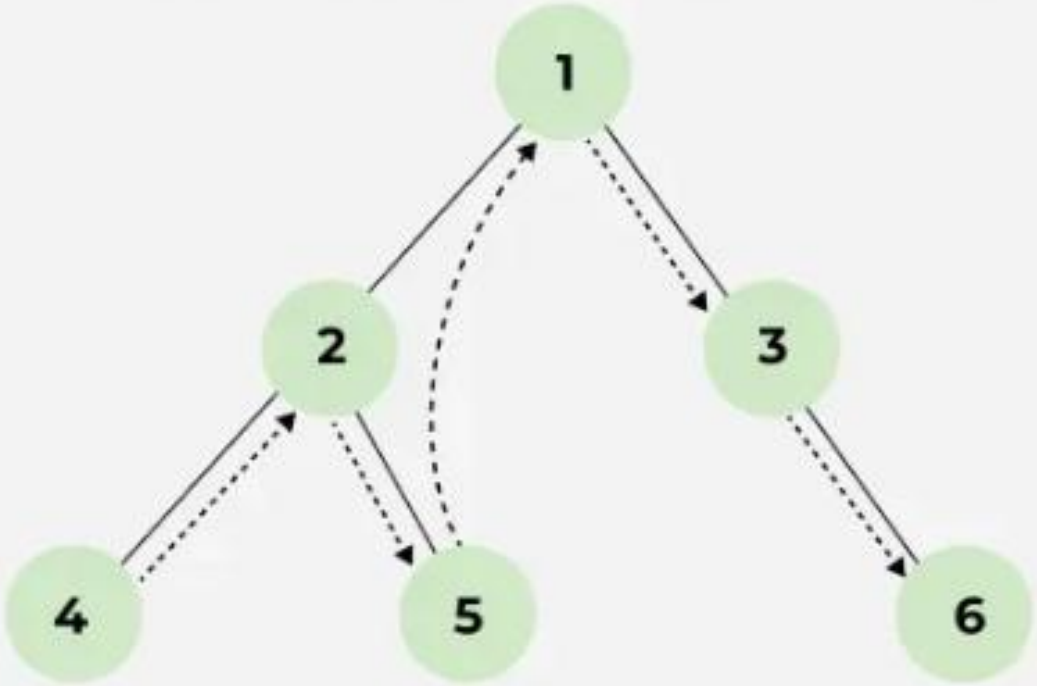
05 Step

Left subtree of node 1 and the node itself is visited. So now the right subtree of 1 will be traversed i.e., move to node 3. As node 3 has no left subtree so it gets visited.

Traversal Order: 4→2→5→1→3



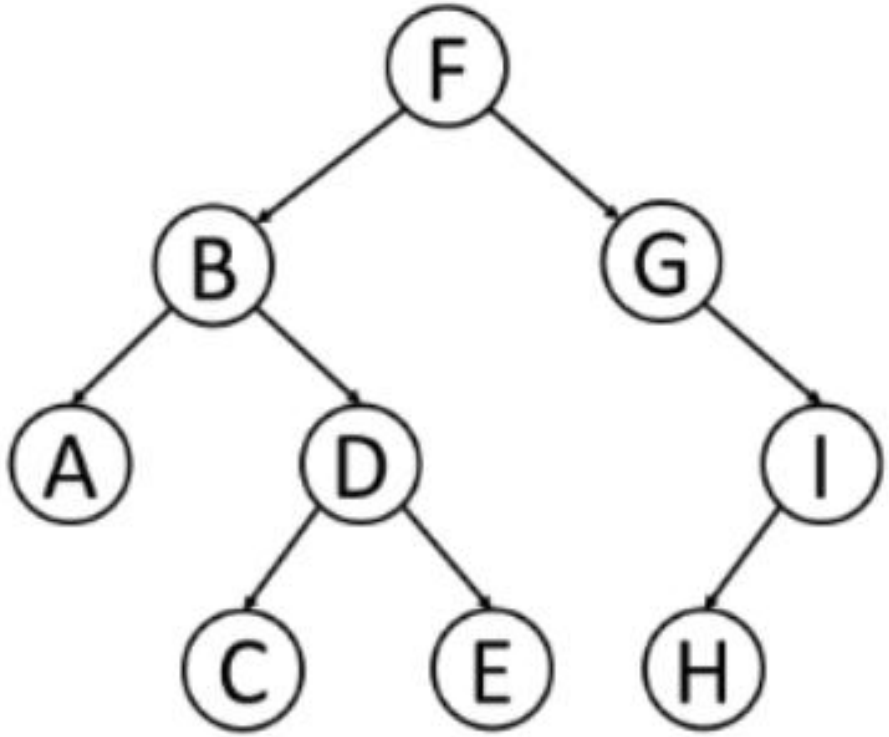
06 Step The left subtree of node 3 and the node itself is visited. So traverse to the right subtree and visit node 6. Now the traversal ends as all the nodes are traversed.



So the order of traversal of nodes is
4→2→5→1→3→6.

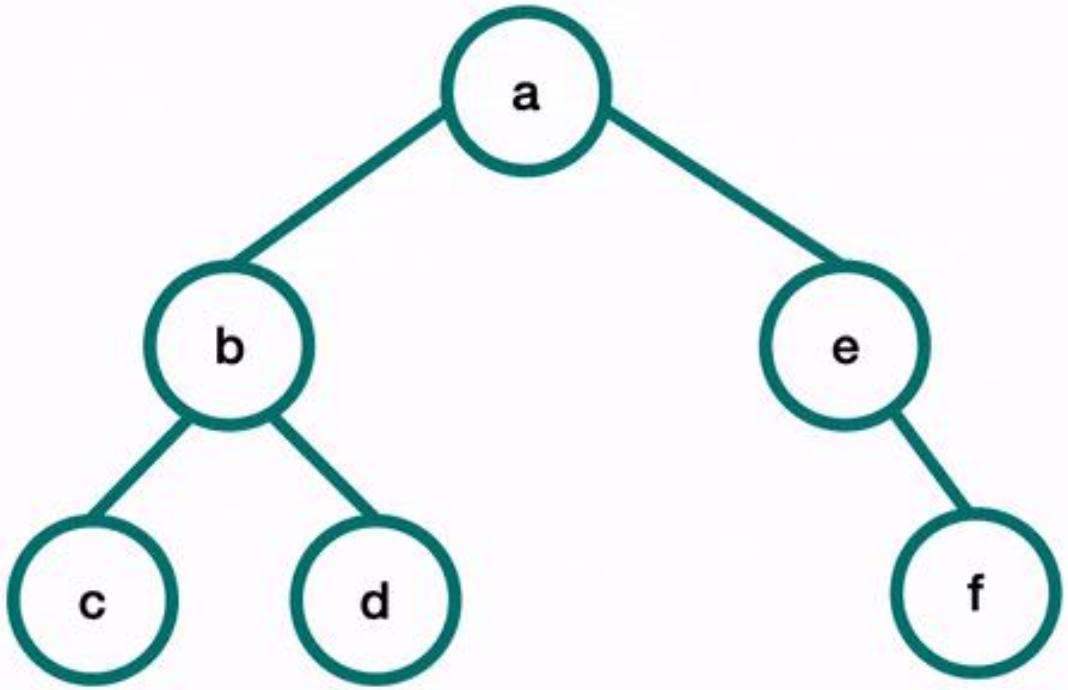
Uses of Inorder Traversal

- In the case of binary search trees (BST), Inorder traversal gives nodes in non-decreasing order.
- To get nodes of BST in non-increasing order, a variation of Inorder traversal where Inorder traversal is reversed can be used.
- Inorder traversal can be used to evaluate arithmetic expressions stored in expression trees.



Inorder:

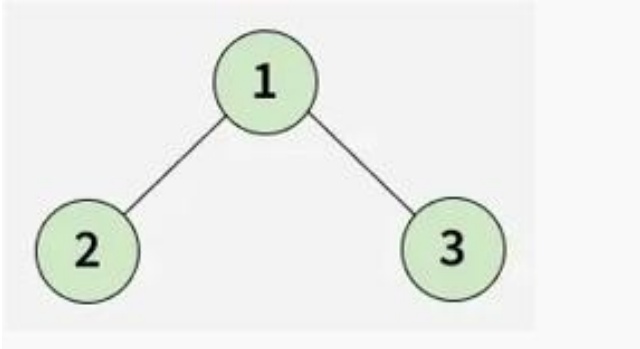
In-Order Traversal



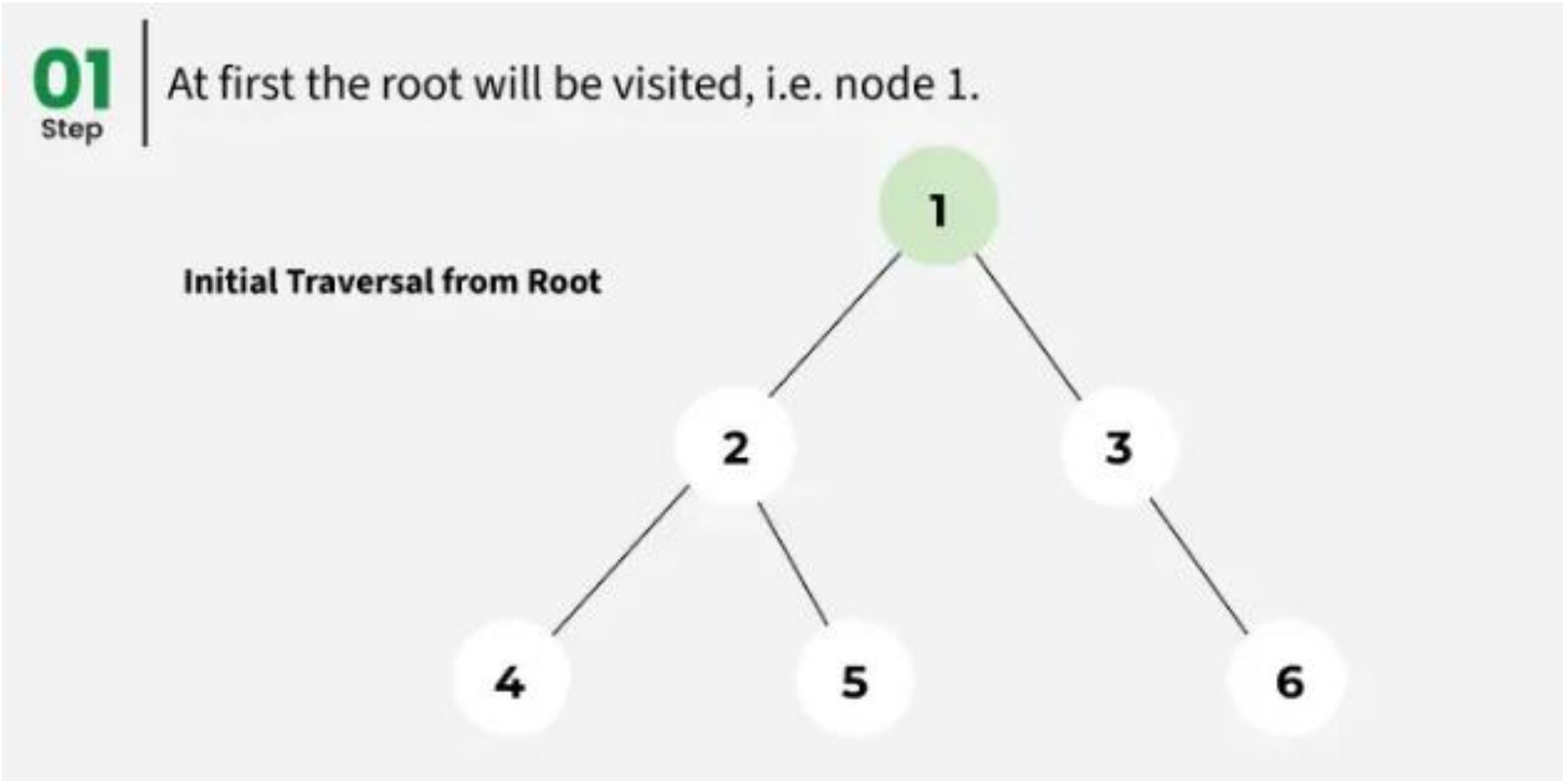
Print “”

Preorder Traversal

- Preorder traversal visits the node in the order: Root -> Left -> Right
- Algorithm for Preorder Traversal
- Preorder(tree)
  - Visit the root.
  - Traverse the left subtree, i.e., call Preorder(left->subtree)
  - Traverse the right subtree, i.e., call Preorder(right->subtree)


1
2
3

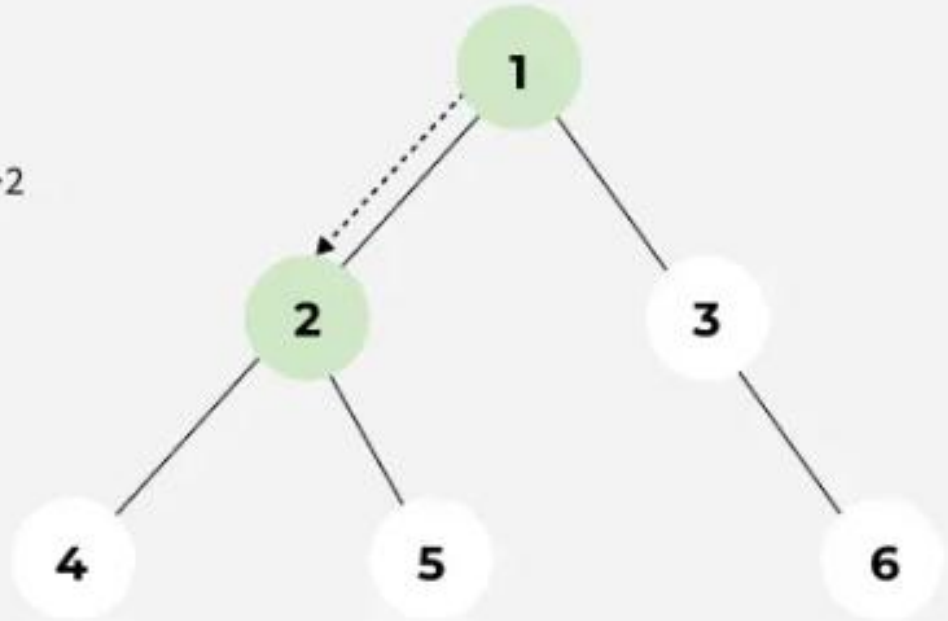
01 Step | At first the root will be visited, i.e. node 1.



02 Step

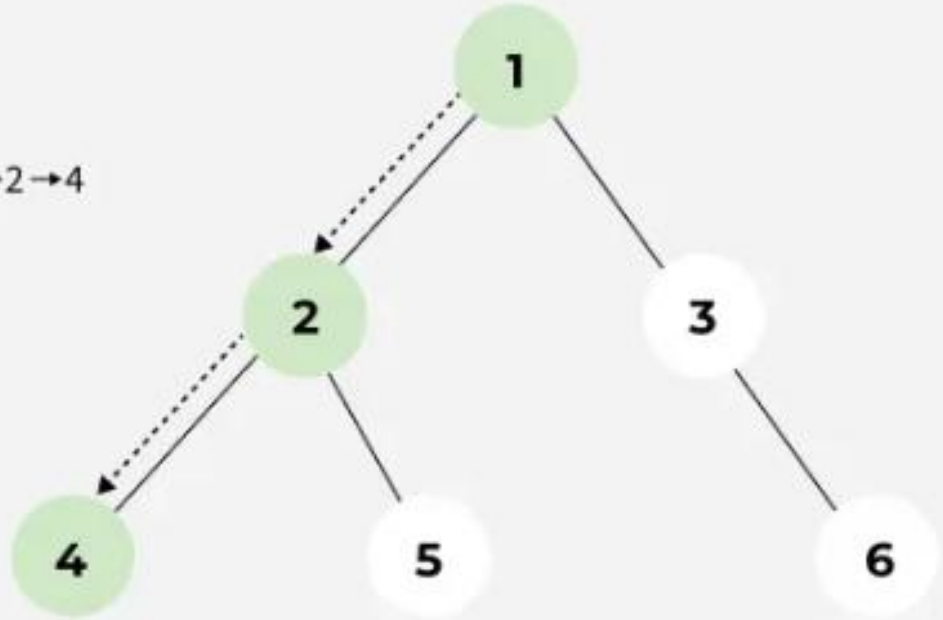
After this, traverse in the left subtree. Now the root of the left subtree is visited i.e., node 2 is visited.

Traversal Order: 1 → 2



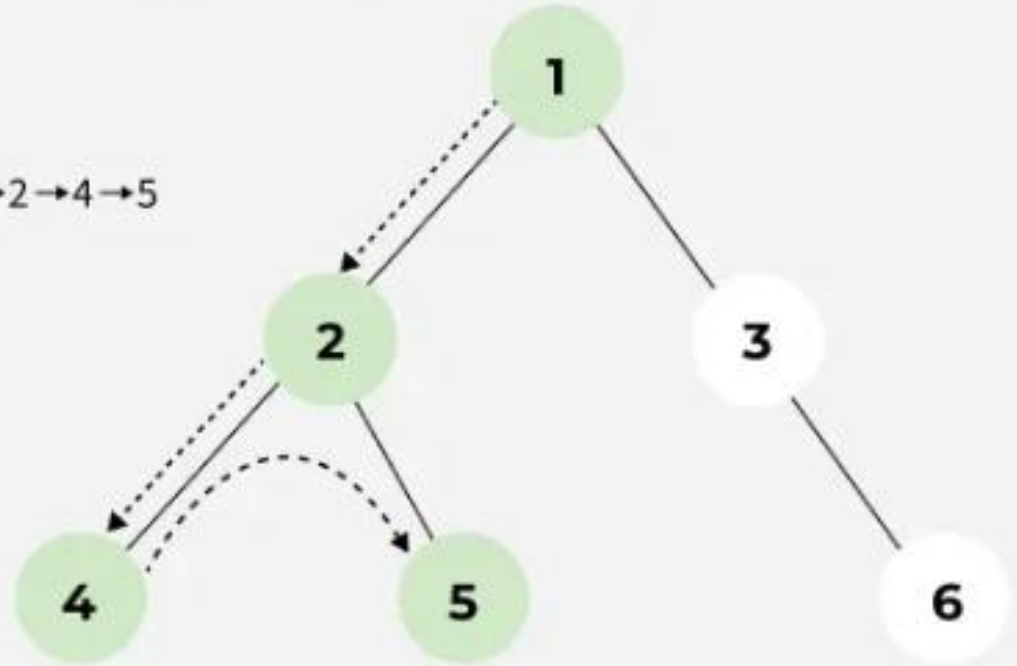
03 Step | Again the left subtree of node 2 is traversed and the root of that subtree i.e., node 4 is visited.

Traversal Order: 1→2→4



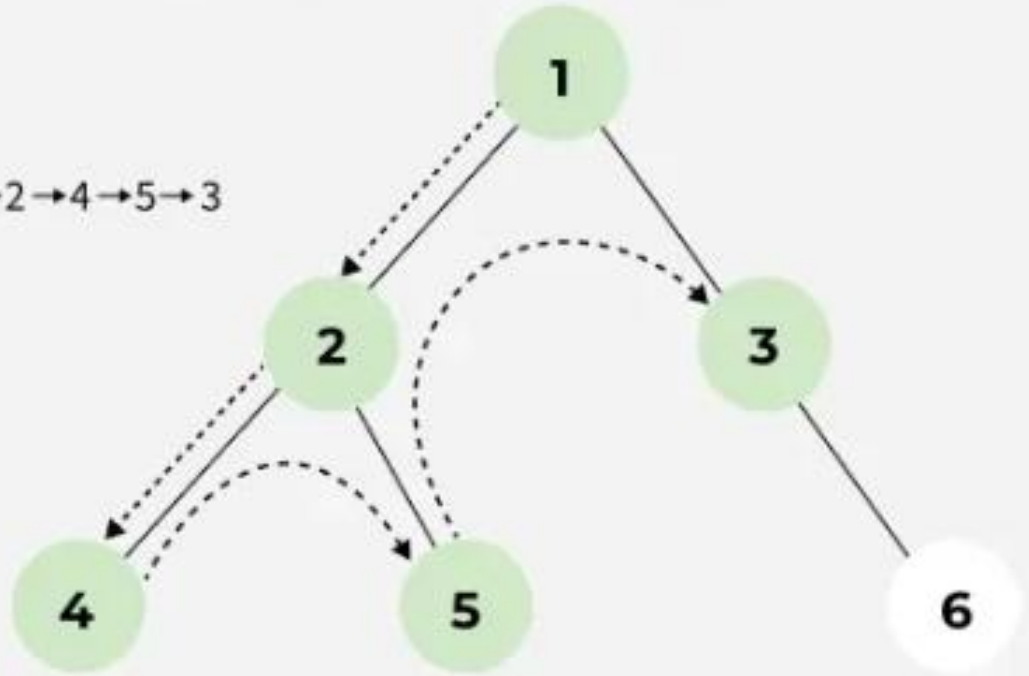
04 Step | There is no subtree of 4 and the left subtree of node 2 is visited. So now the right subtree of node 2 will be traversed and the root of that subtree i.e., node 5 will be visited.

Traversal Order: 1 → 2 → 4 → 5

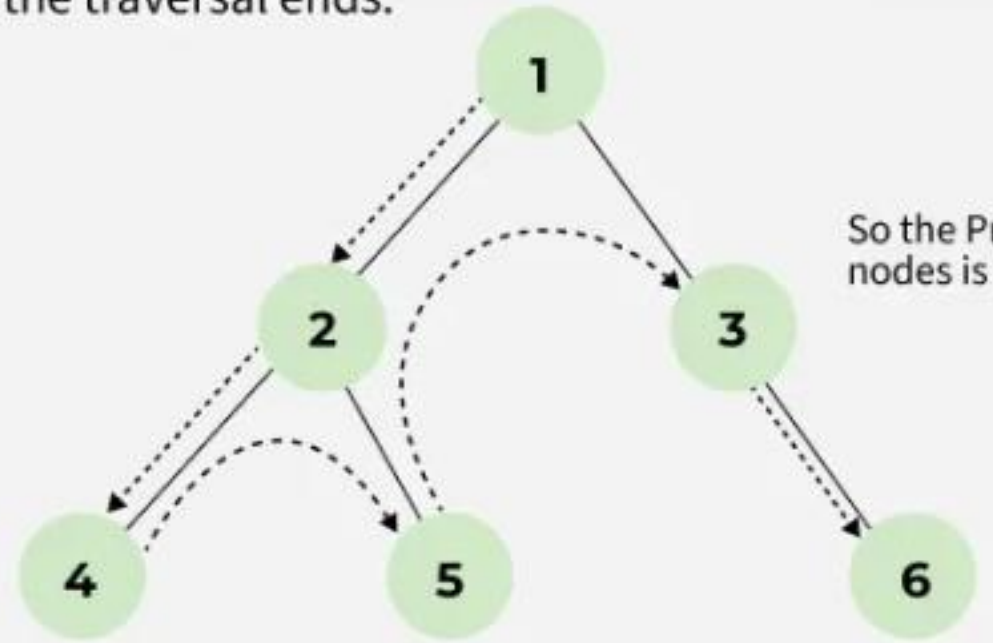


05 Step | The left subtree of node 1 is visited. So now the right subtree of node 1 will be traversed and the root node i.e., node 3 is visited.

Traversal Order: 1→2→4→5→3



06 Step | Node 3 has no left subtree. So the right subtree will be traversed and the root of the subtree i.e., node 6 will be visited. After that there is no node that is not yet traversed. So the traversal ends.

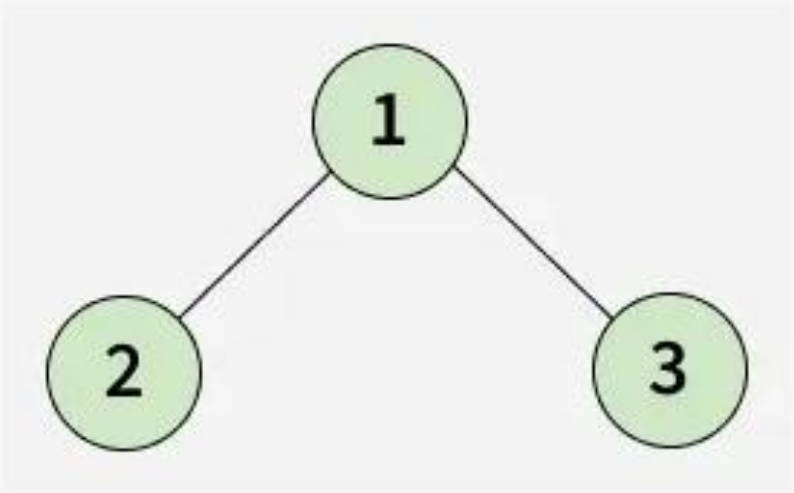


So the Preorder traversal of nodes is 1→2→4→5→3→6.

Postorder Traversal

- Postorder traversal visits the node in the order: Left -> Right -> Root
- Algorithm for Postorder Traversal:
- Postorder(tree)
  - Traverse the left subtree, i.e., call Postorder(left->subtree)
  - Traverse the right subtree, i.e., call Postorder(right->subtree)
  - Visit the root

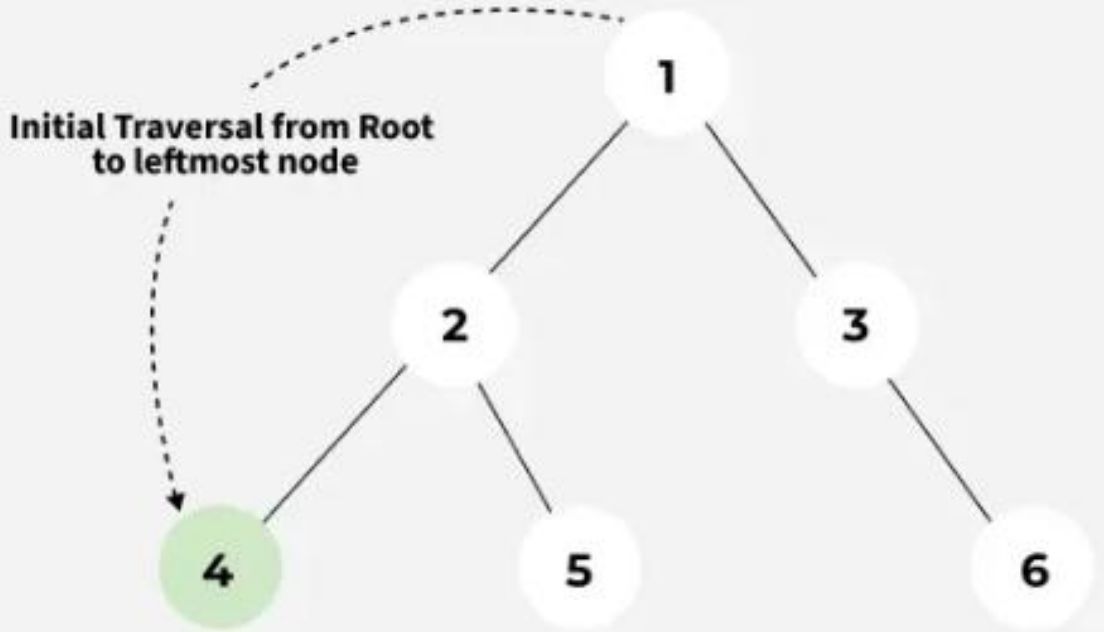
Input:



Output: *[2, 3, 1]*

01 Step

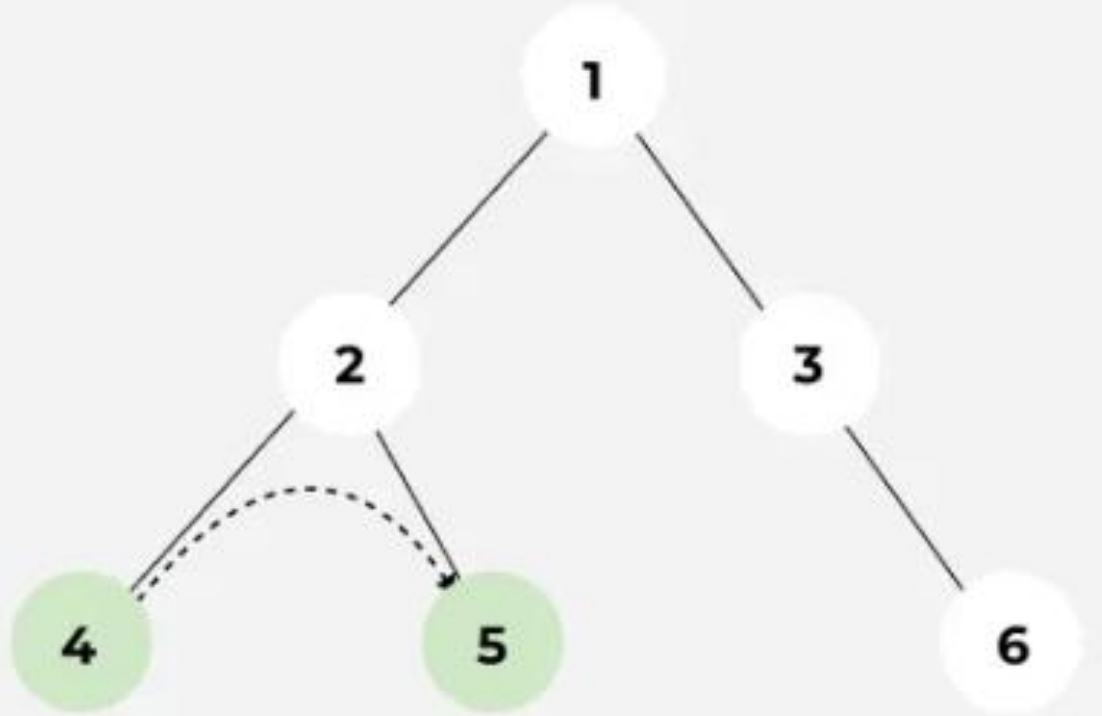
The traversal will go from 1 to its left subtree i.e., 2, then from 2 to its left subtree root, i.e., 4. Now 4 has no subtree, so it will be visited.



02 Step

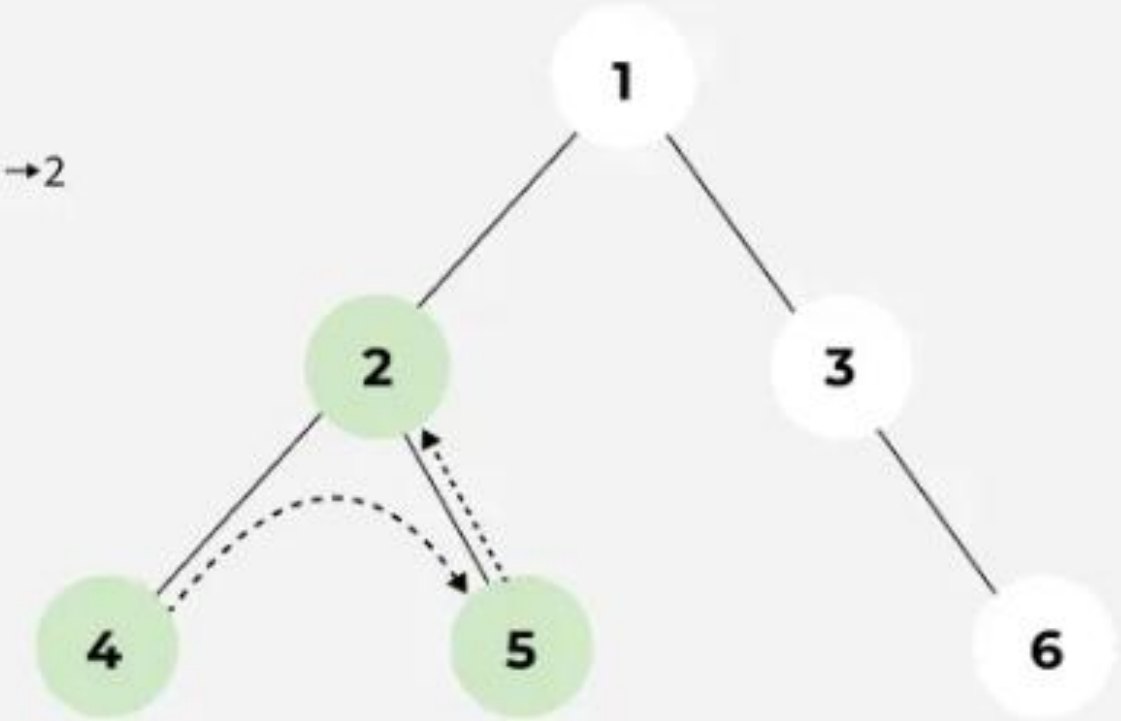
As the left subtree of 2 is visited completely, now it will traverse the right subtree of 2 i.e., it will move to 5. As there is no subtree of 5, it will be visited.

Traversal Order: 4 → 5



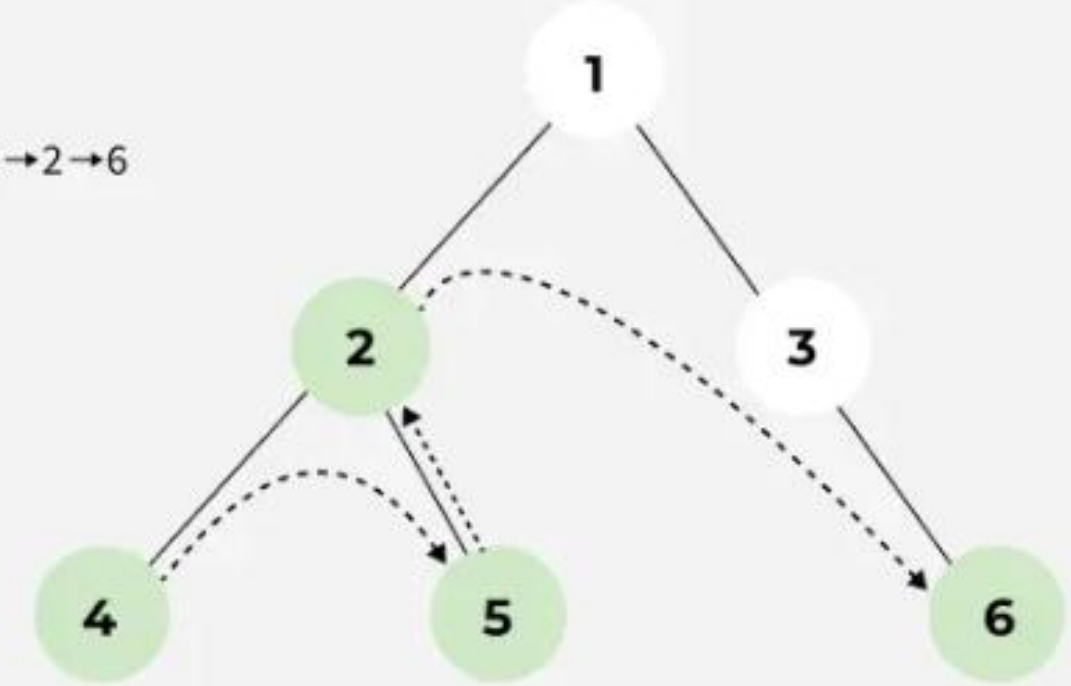
03 Step | Now both the left and right subtrees of node 2 are visited. So now visit node 2 itself.

Traversal Order: 4 → 5 → 2



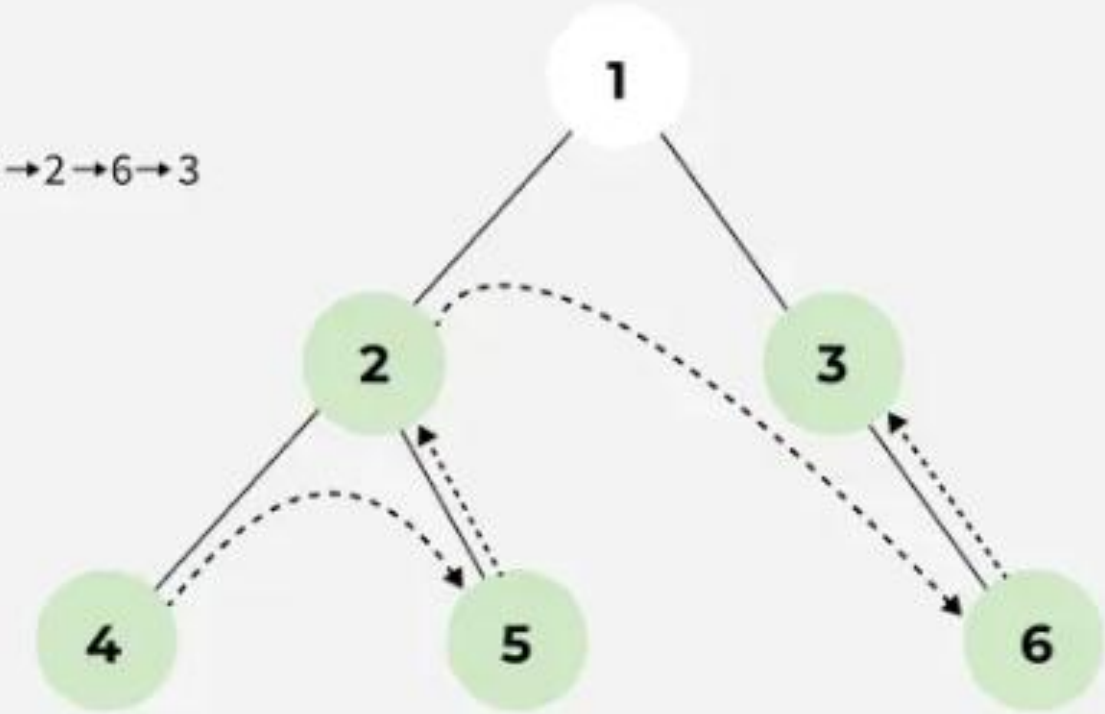
04 Step | As the left subtree of node 1 is traversed, it will now move to the right subtree root, i.e., 3. Node 3 does not have any left subtree, so it will traverse the right subtree i.e., 6. Node 6 has no subtree and so it is visited.

Traversal Order: 4→5→2→6



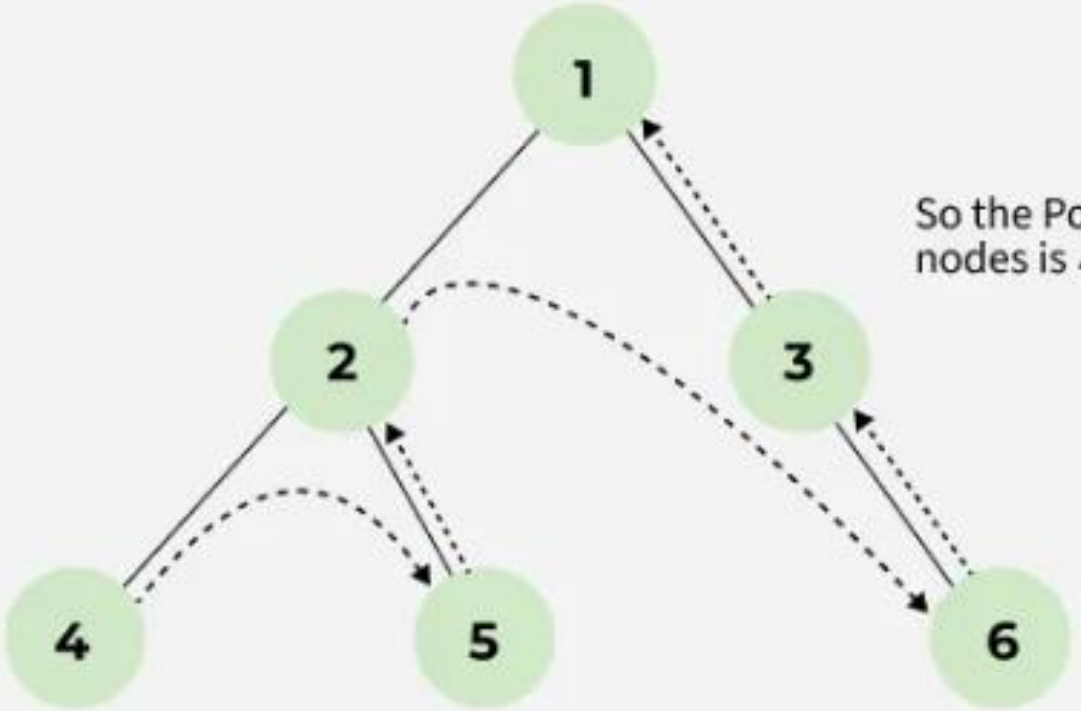
05 Step | All the subtrees of node 3 are traversed. So now node 3 is visited.

Traversal Order: 4→5→2→6→3

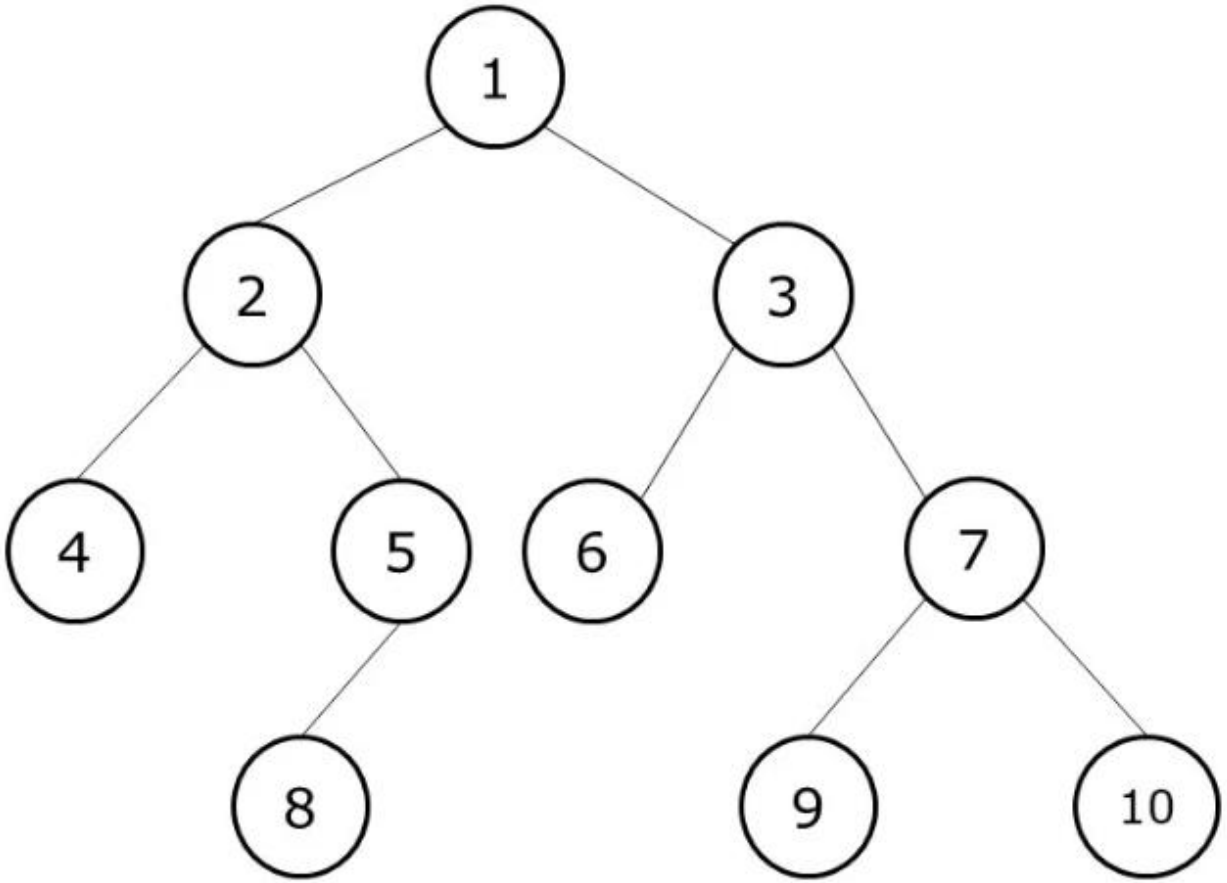


06 Step

As all the subtrees of node 1 are traversed, now it is time for node 1 to be visited and the traversal ends after that as the whole tree is traversed.



So the Postorder traversal of nodes is 4→5→2→6→3→1.

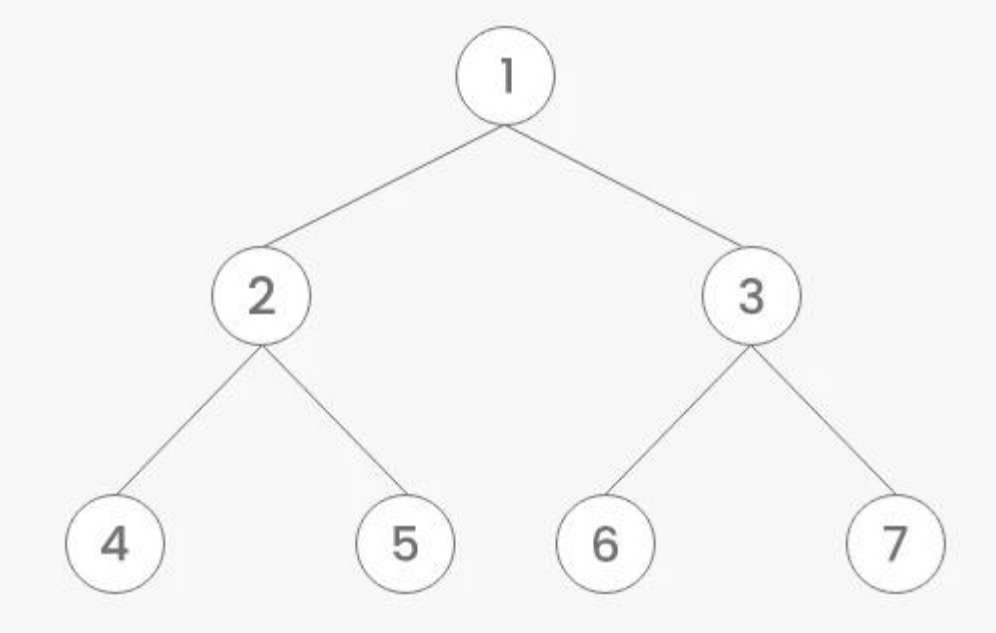


Preorder Traversal:

[root, left, right]

1	2	4	5	8	3	6	7	9	10

Examples



Inorder Traversal

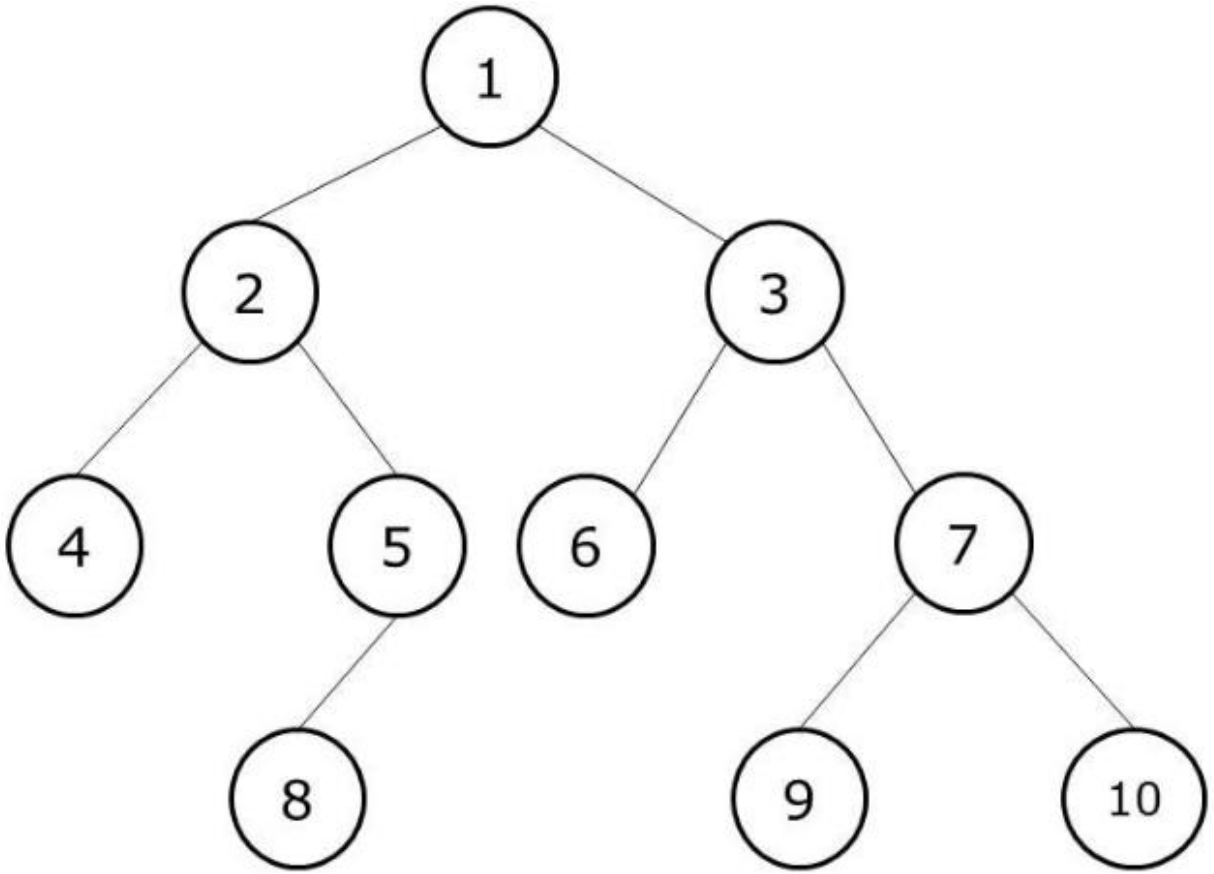
4	2	5	1	6	3	7

Preorder Traversal

1	2	4	5	3	6	7

Postorder Traversal

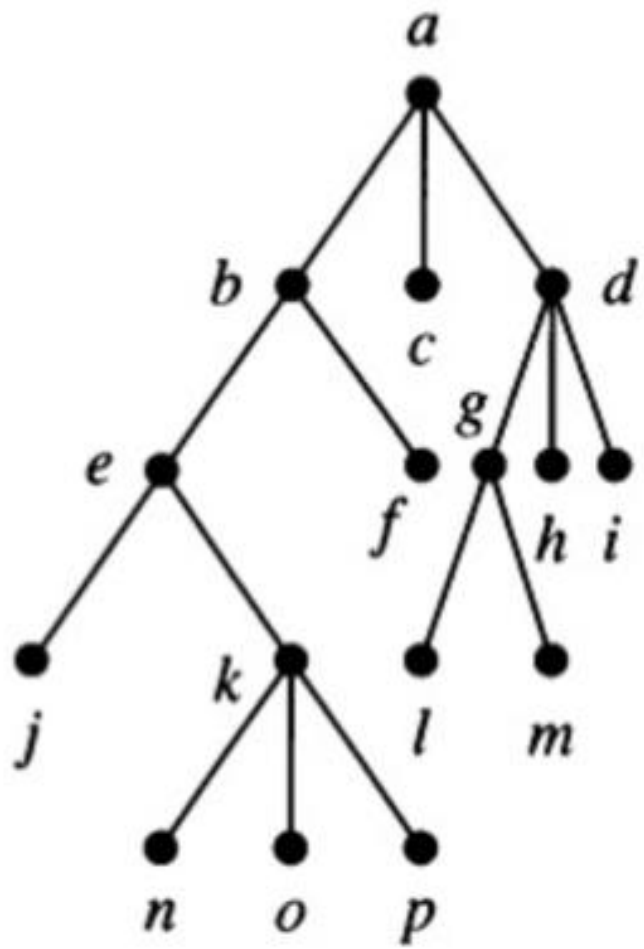
4	5	2	6	7	3	1

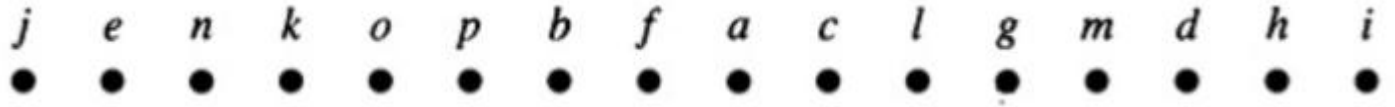


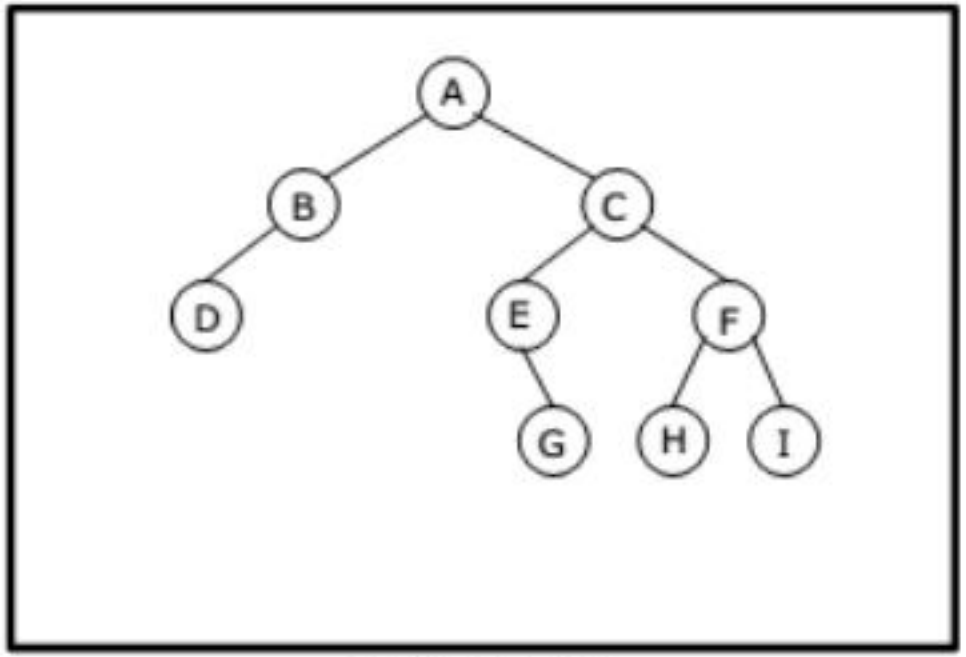
Inorder Traversal:

[left,root,right]

4	2	8	5	1	6	3	9	7	10


a
b
c
d
e
f
g
h
i
j
k
l
m
n
o
p

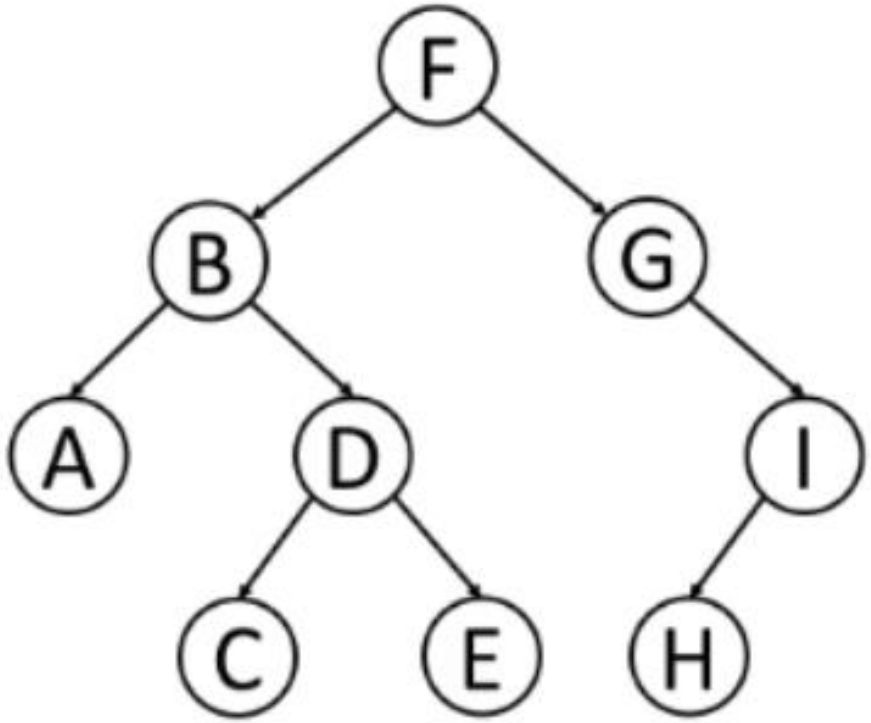

j e n k o p b f a c l g m d h i



Binary Tree

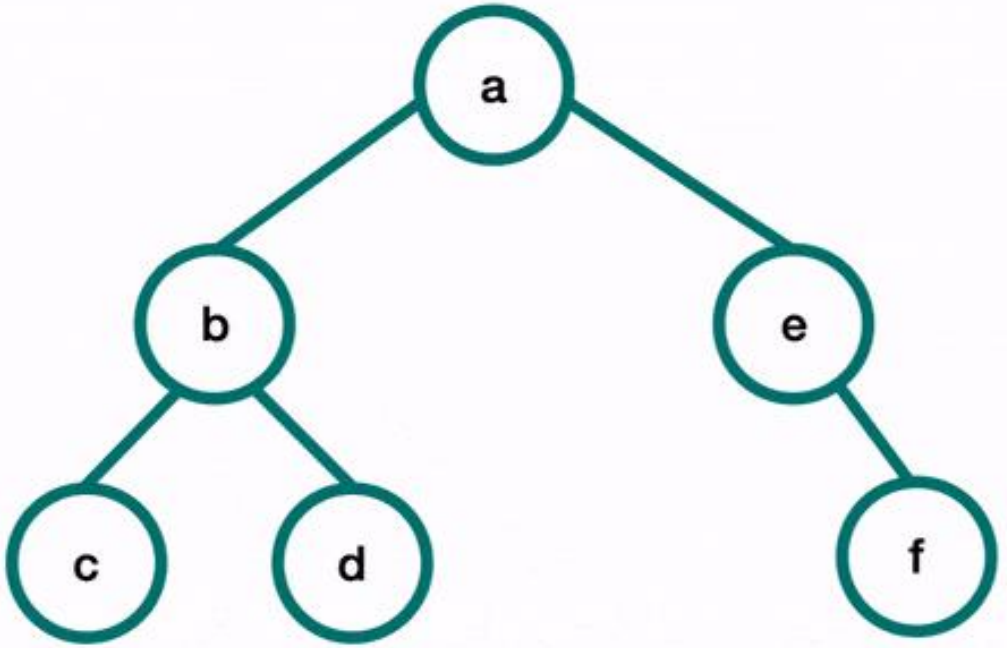
- Preorder traversal yields:
  A, B, D, C, E, G, F, H, I
- Postorder traversal yields:
  D, B, G, E, H, I, F, C, A
- Inorder traversal yields:
  D, B, A, E, G, C, H, F, I
- Level order traversal yields:
  A, B, C, D, E, F, G, H, I

Pre, Post, Inorder and level order Traversing



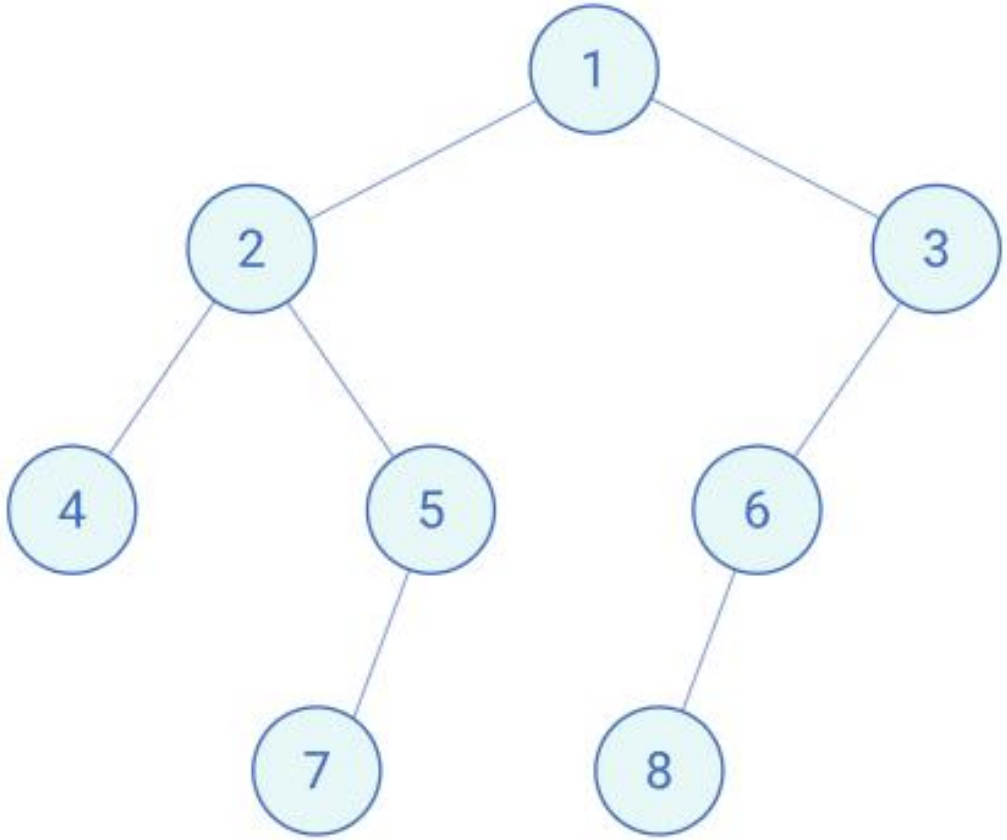
Preorder:

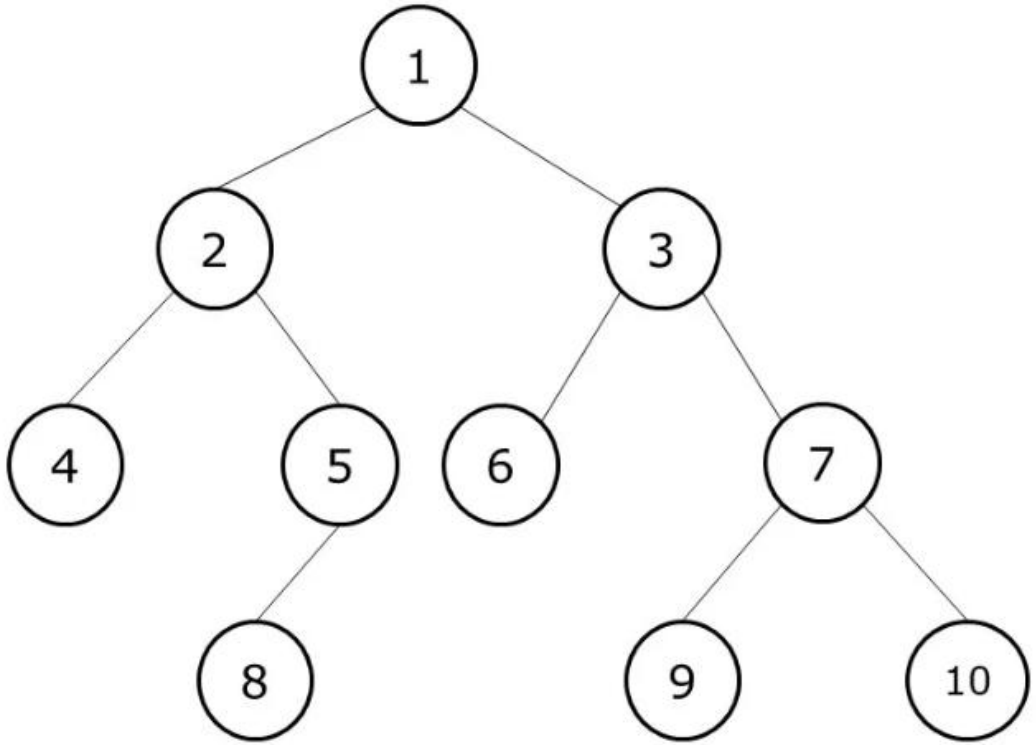
Pre-Order Traversal



Print “”

Preorder: [1, 2, 4, 5, 7, 3, 6, 8]
Inorder: [4, 2, 7, 5, 1, 8, 6, 3]





Postorder Traversal:

[left, right, root]

4	8	5	2	6	9	10	7	3	1

InOrder(root) visits nodes in the following order:

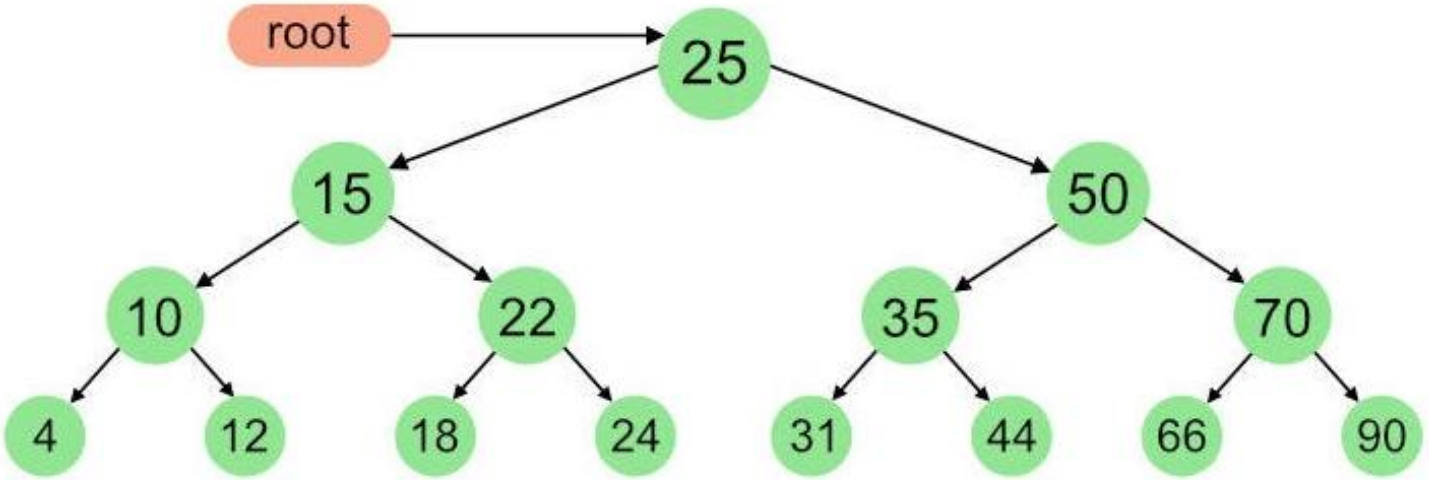
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

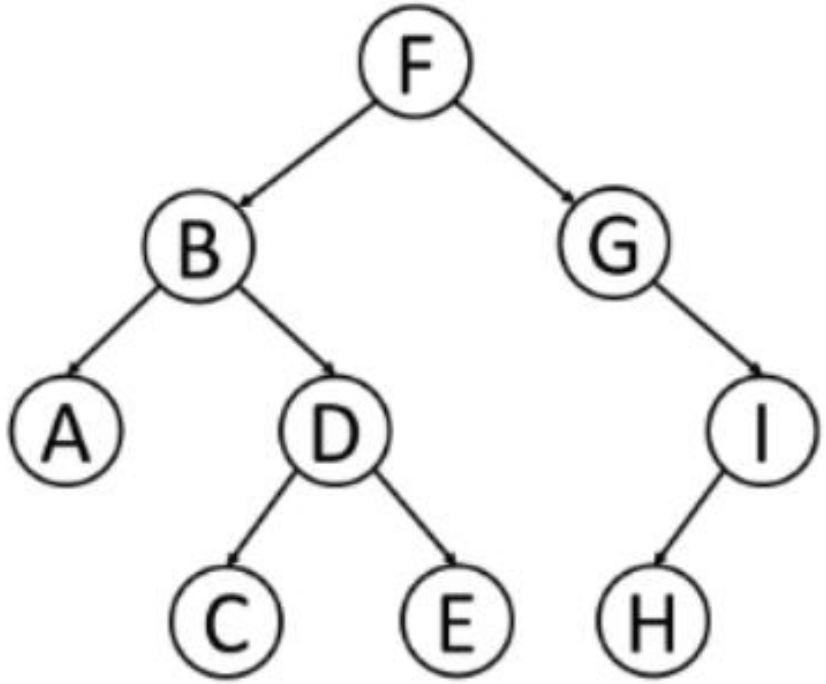
A Pre-order traversal visits nodes in the following order:

25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

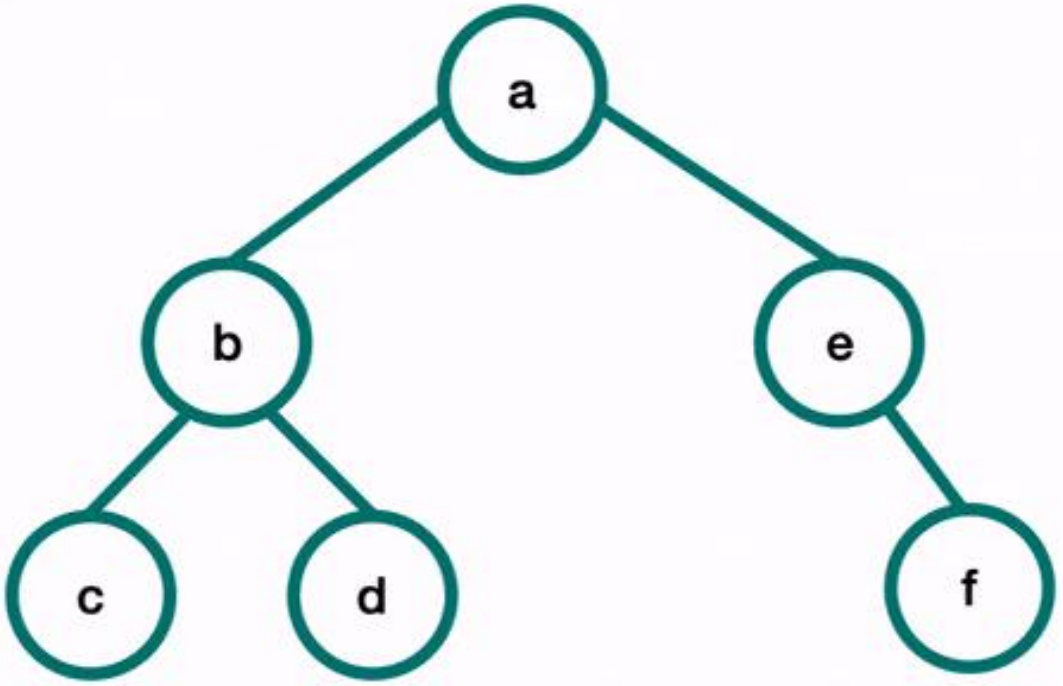
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



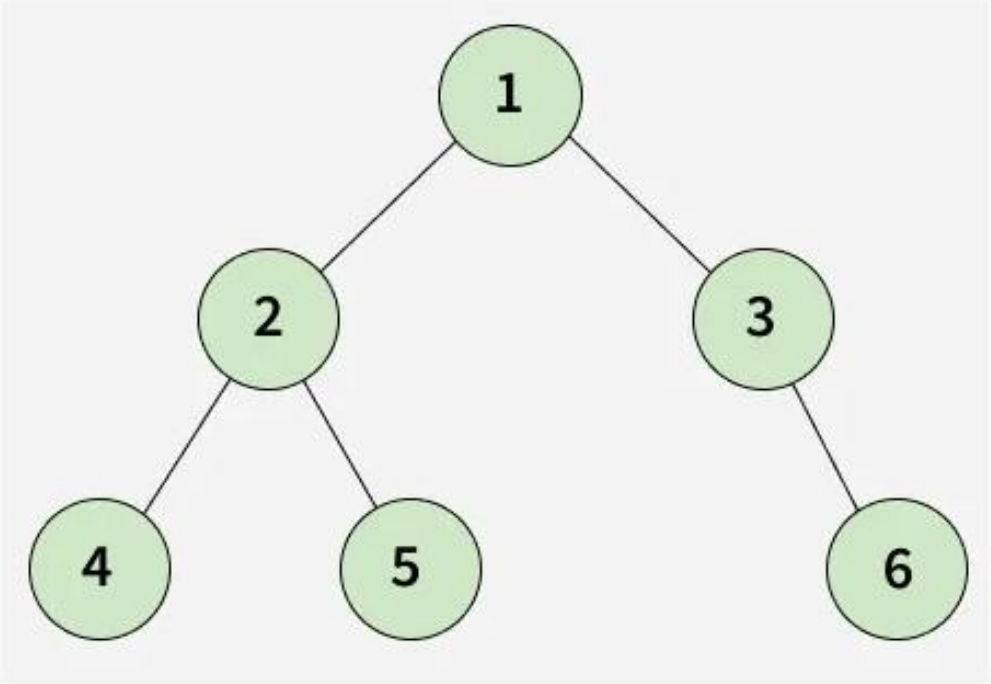


Postorder:

Post-Order Traversal



Print “”



Output: *[4, 5, 2, 6, 3, 1]*
Explanation: *Postorder Traversal visits the nodes in the following order: Left, Right, Root. Therefore resulting is 4 , 5, 2, 6, 3, 1.*

C Program For Preorder Traversal

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};
```

```
// Function to create a new node
struct Node* newNode(int value) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = value;
    temp->left = temp->right = NULL;
    return temp;
}
```

```
// Preorder Traversal
void preorder(struct Node* root) {
    if (root == NULL) return;
    printf("%d ", root->data);    // Root
    preorder(root->left);         // Left
    preorder(root->right);        // Right
}
```

```
int main() {
    struct Node *root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->left->right->left = newNode(7);
    root->right->right = newNode(6);
    root->right->right->right = newNode(8);
    printf("Preorder Traversal: ");
    preorder(root);
    return 0;
}
```

C Program For Preorder Traversal

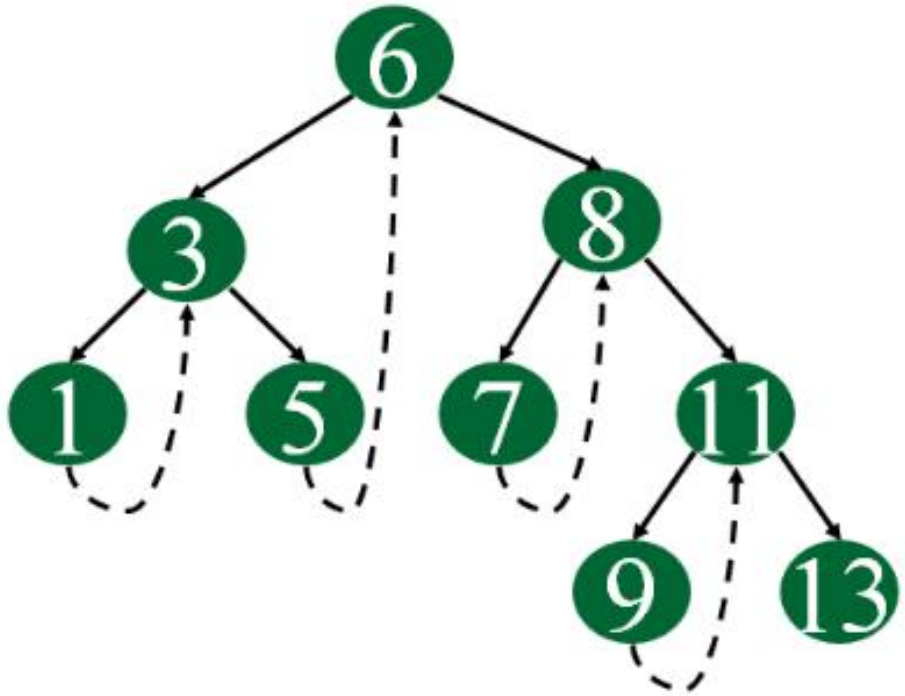
C Program For Postorder Traversal

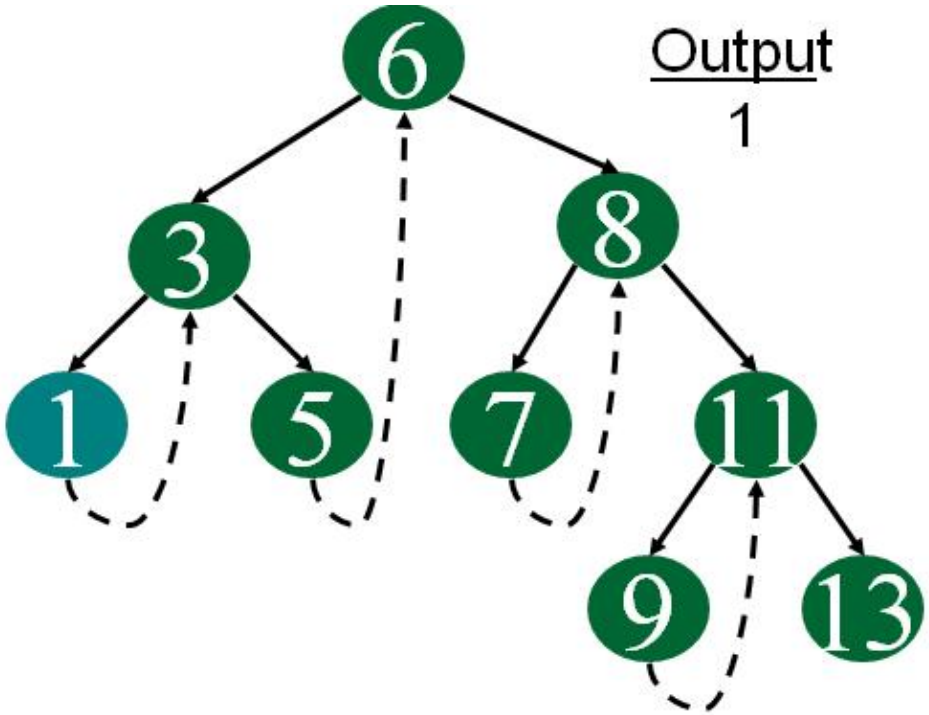
Threaded binary trees

- A threaded binary tree is a type of binary tree data structure where the empty left and right child pointers in a binary tree are replaced with threads that link nodes directly to their in-order predecessor or successor, thereby providing a way to traverse the tree without using recursion or a stack.
- There are two types of threaded binary trees.
  - Single Threaded: Where a NULL right pointers is made to point to the inorder successor (if successor exists)
  - Double Threaded: Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively.

Following diagram demonstrates inorder order traversal using threads.

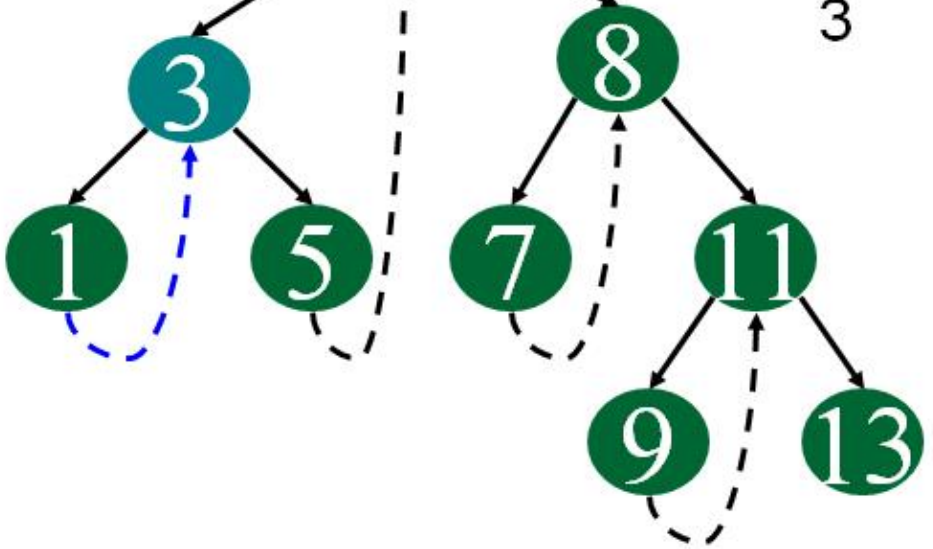
- The dotted lines represent threads.





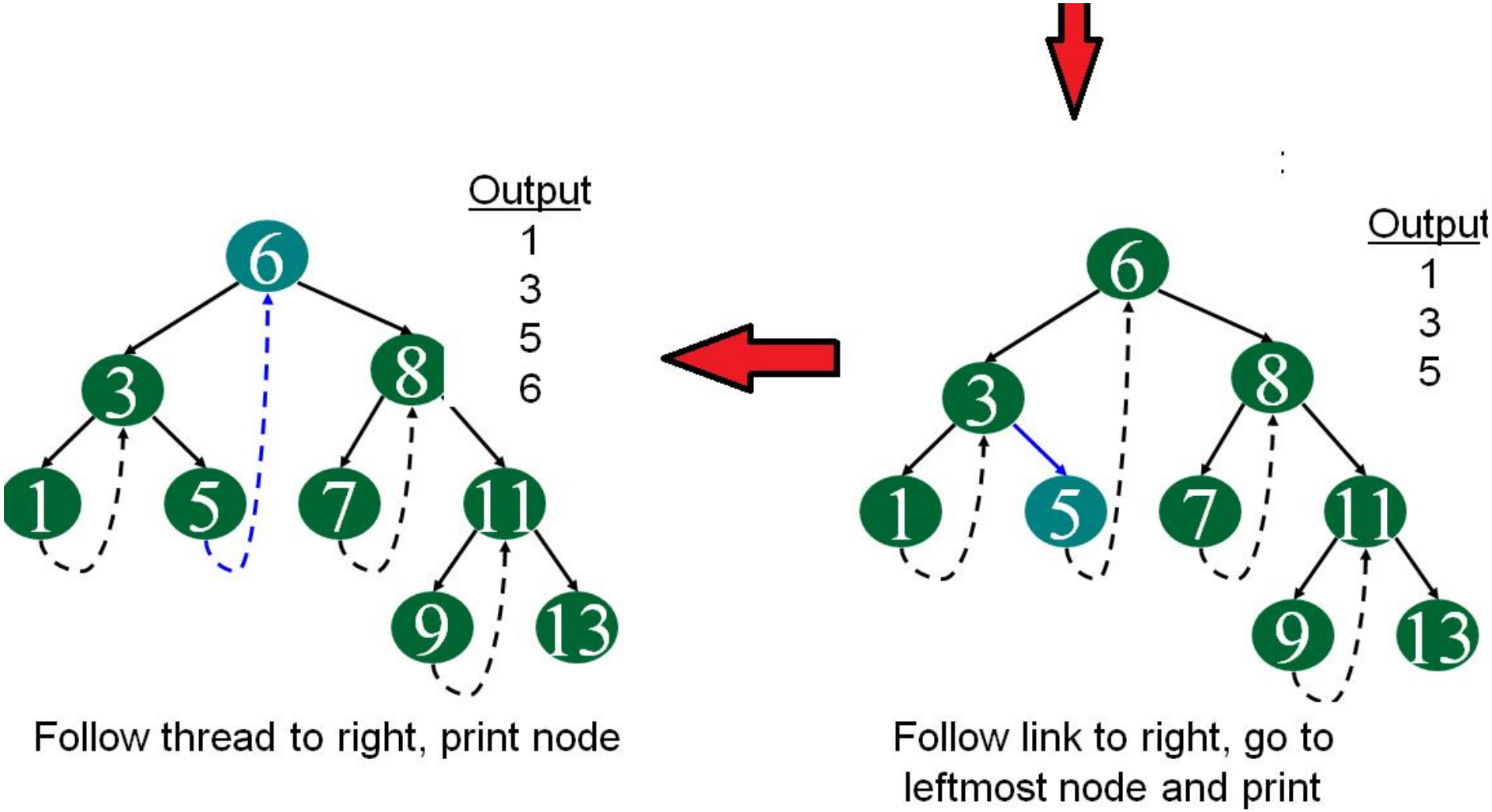
Output
1

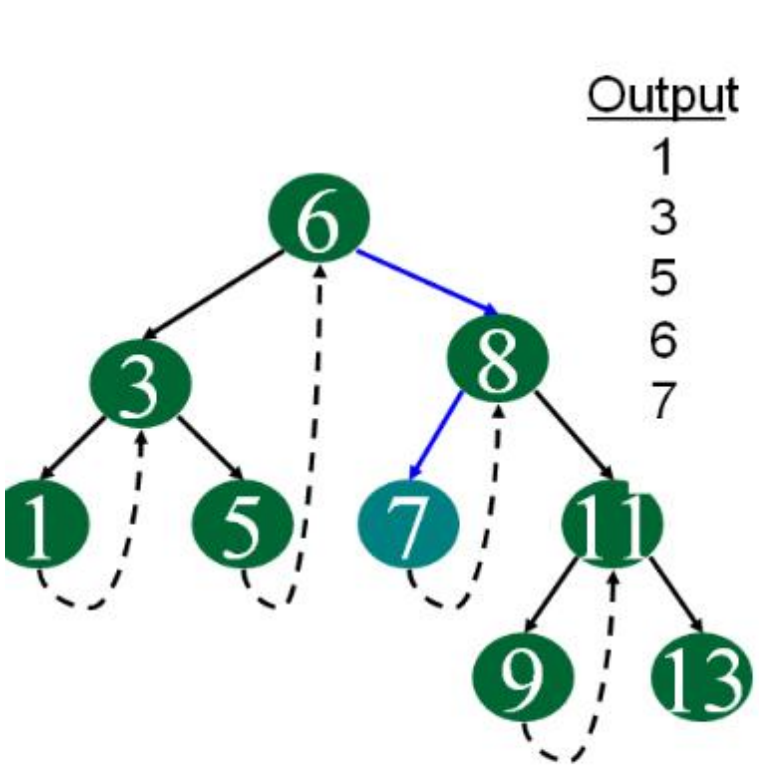
Start at leftmost node, print it



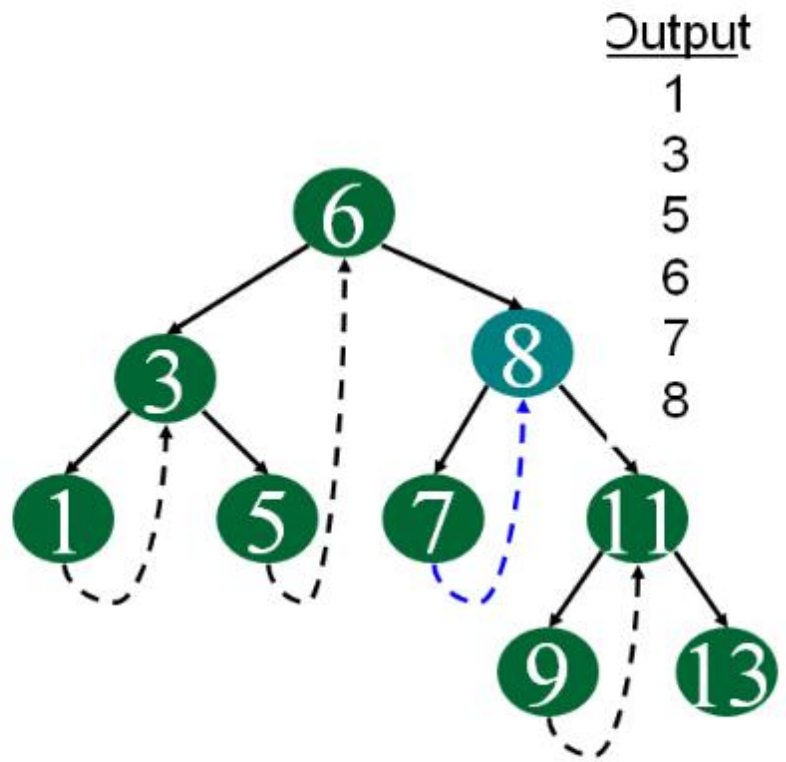
Output
1
3

Follow thread to right, print node


Output
1
3
5
6
Output
1
3
5
Follow thread to right, print node
Follow link to right, go to leftmost node and print



Follow link to right, go to leftmost node and print



Follow thread to right, print node

continue same way for remaining node.....

C Program: Inorder Threaded Binary Tree

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
    int rightThread; // 1 => right pointer is a thread; 0 => right pointer is a child
};
```

```
struct Node* createNode(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    newNode->rightThread = 0;
    return newNode;
}
```

// Finds inorder successor

```
struct Node* inorderSuccessor(struct Node *ptr) {
    if (ptr->rightThread)
        return ptr->right;
    ptr = ptr->right;
    while (ptr->left)
        ptr = ptr->left;
    return ptr;
}
```

// Insert a node in Threaded BST

```
struct Node* insert(struct
Node *root, int data) {
    struct Node *parent = NULL,
*curr = root;


    while (curr != NULL) {
        if (data == curr->data) {
            printf("Duplicate keys
not allowed!\n");
            return root;
        }
```

```
parent = curr;
        if (data < curr->data) {
            curr = curr->left;
        } else {
            if (curr->rightThread == 0)
                curr = curr->right;
            else
                break;
        }
    }
```

```
struct Node *newNode = createNode(data);
    if (parent == NULL) {
        root = newNode;
        newNode->rightThread = 1;
        newNode->right = NULL;
    }
                else if (data < parent->data) {
                    newNode->left = parent->left;
                    newNode->right = parent;
                    newNode->rightThread = 1;
                    parent->left = newNode;
                }
 else {
        newNode->right = parent->right;
        newNode->rightThread = parent->rightThread;
        parent->right = newNode;
        parent->rightThread = 0;
        newNode->left = NULL;
    }
    return root;
}
```

// Inorder traversal using threads

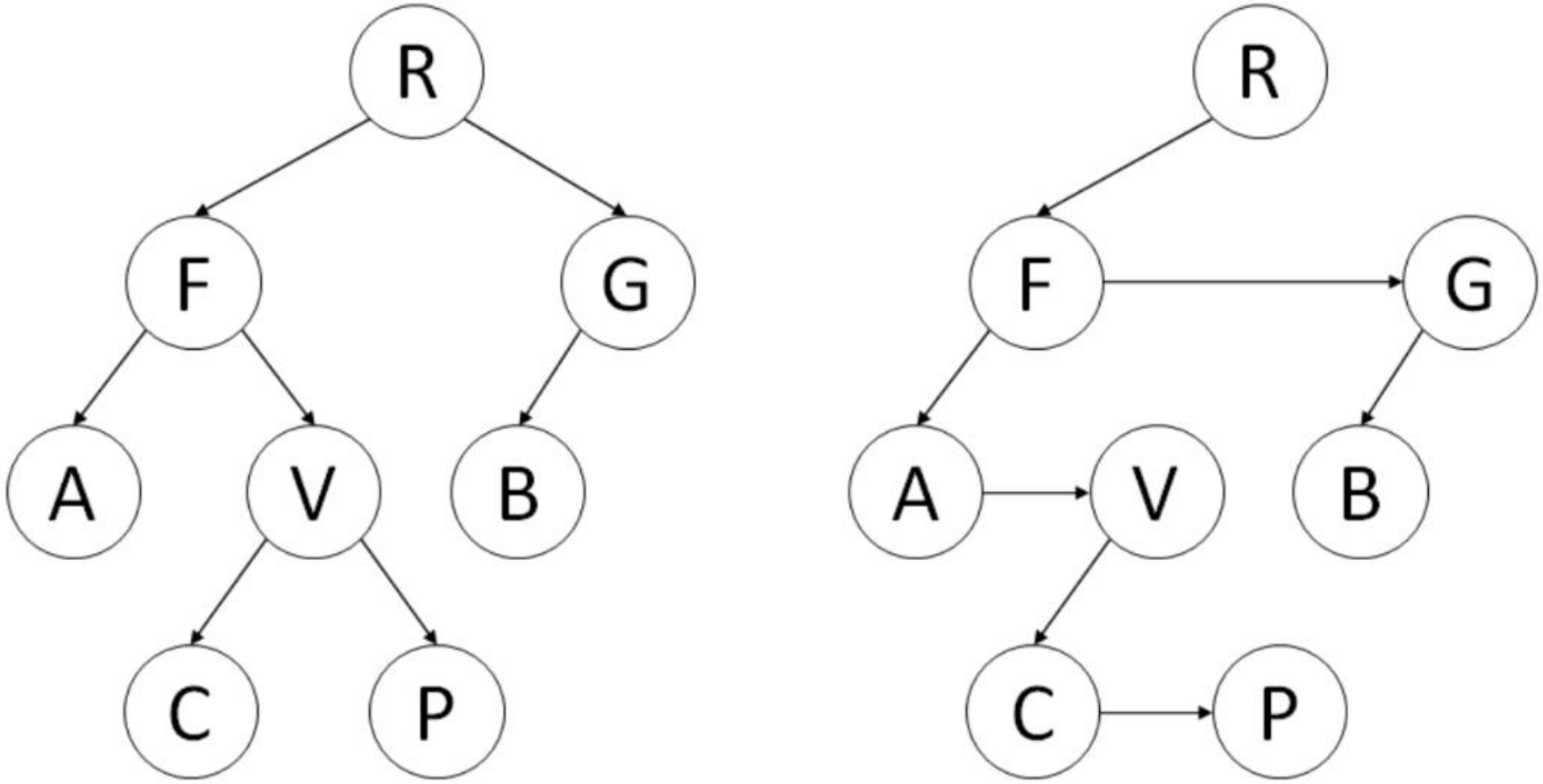
```
void inorder(struct Node *root) {
    if (root == NULL) return;

    struct Node *curr = root;
    while (curr->left)
        curr = curr->left;

    while (curr != NULL) {
        printf("%d ", curr->data);
        curr = inorderSuccessor(curr);
    }
}
```

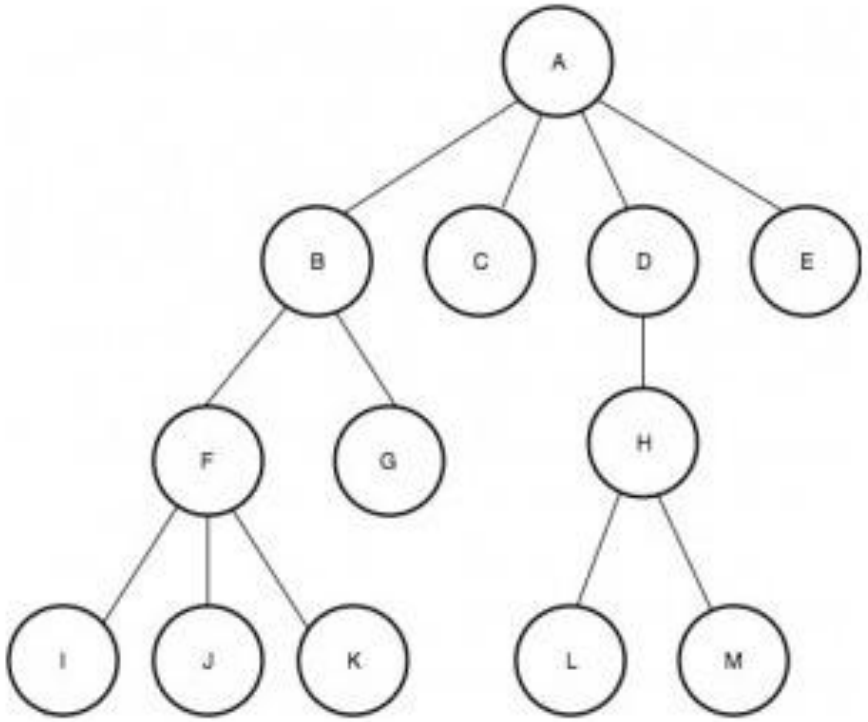
```
int main() {
    struct Node *root = NULL;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Inorder Traversal (Threaded): ");
    inorder(root);
    printf("\n");

    return 0;
}
```

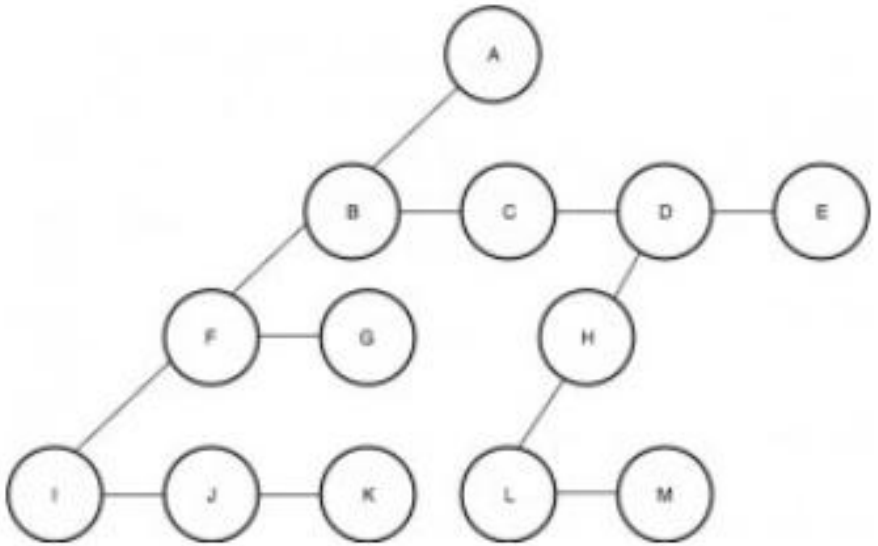
Left-Child Right-Sibling Representation of Tree

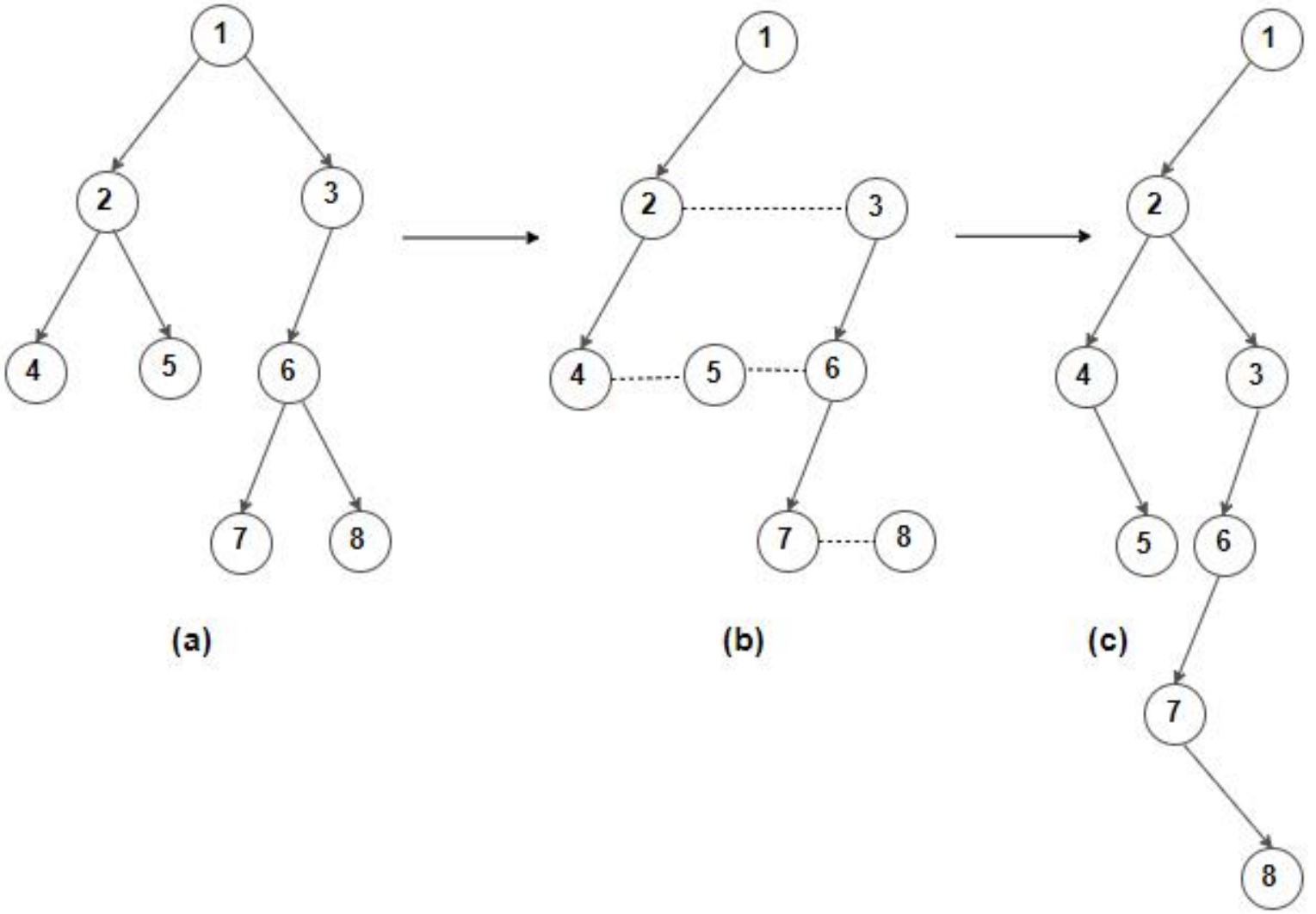
- It is a different representation of an n-ary tree where instead of holding a reference to each and every child node, a node holds just two references, first a reference to it’s first child, and the other to it’s immediate next sibling.

Consider the following family tree.

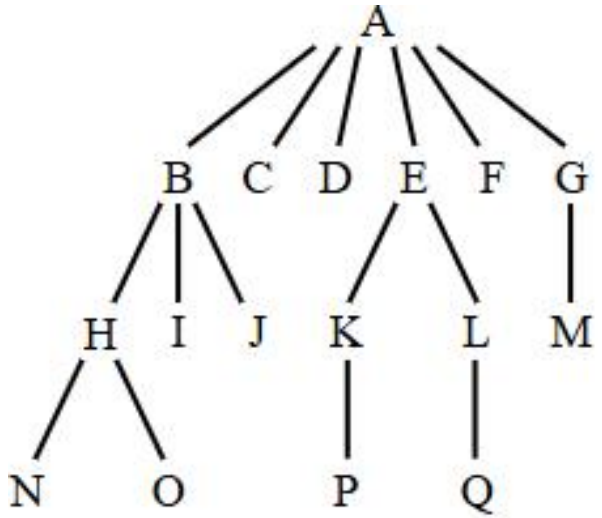


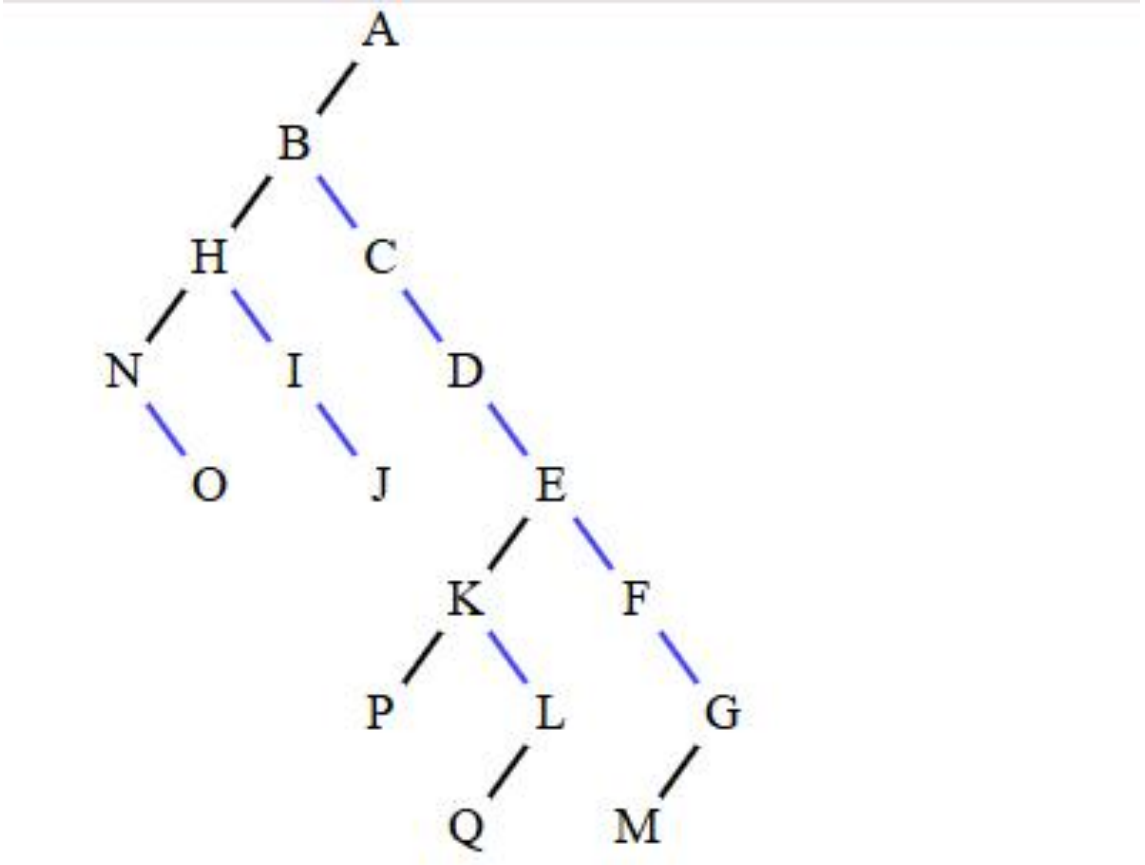

A
B
C
D
E
F
G
H
I
J
K
L
M

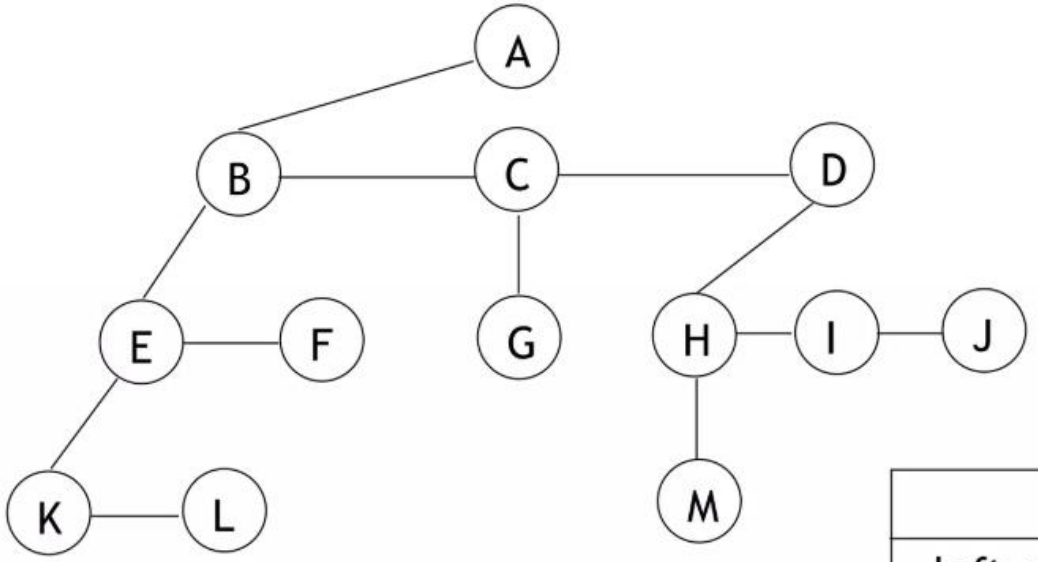

A
B
C
D
E
F
G
H
I
J
K
L
M


1
2
3
4
5
6
7
8
(a)
(b)
(c)

Left child right child tree representation







data	
left child	right sibling